Technical Design Document (TDD)

TaskFlow – Intelligent Task Allocation & Tracking System

© 2026 Jojo George. All rights reserved.

Field	Value
Document	Technical Design Document
Version	2.1
Status	Updated to match implementation; aligned with the approved BRD set (signed off 31-Oct-2026) and the FRD / HLD / LLD set
Basis	Approved BRD set v1.0 (`TF - Documents/TF - BRD` , overall + 16 sub-modules, signed off 31-Oct-2026) + FRD / HLD / LLD v1.0 (`TF - Documents/TF - FRD` , `TF - HLD` , `TF - LLD`) + the original BRD v1.0 (retired from the repository) + `TF - Documents/Screens Wise Feature Development Plan/01..20-*.md` + the TaskFlow source code (`TaskFlow Proj/backend` , `TaskFlow Proj/frontend`)
Location	`TF - Documents/TF - TDD/TDD-TaskFlow.md` (+ `.pdf`)
Stack	ASP.NET Core Web API on .NET 10 (`net10.0`), EF Core 10.0.12 + SQLite, FluentValidation 12.2.0, JwtBearer 10.0.12, SpaProxy 10.0.12; Angular 19.2 (standalone), TypeScript 5.7, Chart.js 4 + ng2-charts 8, Angular CDK 19, qrcode
Project type	Personal Project (Skill Development)
Date	31-Oct-2026
Author	Jojo George (jojogeorge3344@gmail.com)

> This TDD describes TaskFlow **as implemented**. The code is the source of truth for *how*; the BRD is the source of truth for *what was asked*. Where the two differ, this document describes the code and records the difference in §23 Deviations from BRD. Since 31-Oct-2026 the approved BRD set (§1.5) is the requirement baseline; §23 records the differences between the code and the **original** BRD v1.0, which the approved BRD set has already absorbed. The BRD recommended Python Flask + SQLAlchemy + APScheduler; the implementation uses ASP.NET Core + EF Core + Angular (mapping in §4.3).

Revision History

Version	Date	Author	Summary
1.0	2026-10-11	Jojo George	Initial design: BRD re-targeted onto ASP.NET Core, EF Core, SQLite and Angular; 4-project backend layout (Api/Application/Domain/Infrastructure); fixed Administrator/Manager/Employee roles; SMTP email notifications.
2.0	31-Oct-2026	Jojo George	Synced to the implementation. Single `TaskFlow.Api` project, layer-first with module sub-folders, enforced by architecture tests and `npm run check:structure`. Permission-based authorization (70 permission keys, read-only system roles, configurable custom roles, `TaskAccess` resource check, Own/Team/Organization scopes). Mandatory MFA (TOTP + 10 backup codes) and password recovery via authenticator code. Internal Mail replaces SMTP email (BRD Screen 14 superseded). Projects, positions and allocations added; `InQA` task status; comment edit/delete and hours logging. TFA AI assistant (Ollama/Gemini/OpenAI). Data folder `C:\TaskFlow`. Legal pages. API documented as 114 actions over 83 routes. Serilog and Playwright removed from the stack (not installed). New sections §15, §16, §23 and Appendix C.
2.1	31-Oct-2026	Jojo George	Aligned with the new document set. Added §1.5 Reference Documents (approved BRD set, FRD, HLD, LLD, Development Plan, permission documents, workflow docs, developer docs, OpenSpec, skills) and §1.6 Document Hierarchy & Traceability (document chain, ID schemes, sub-module document map, original-screen → sub-module mapping). §5.1 repository layout updated to the `TF - Documents` folder. §13 linked to the sub-module documents. §21 maps each criterion to the approved BRD acceptance IDs (AC-01 – AC-26) and adds AC-02 – AC-06 and AC-21 – AC-26. §22.5 lists where open items are tracked in the document set. Design content and section numbers unchanged; code facts re-verified (114 actions, 83 routes, 70 keys, 31 entities).

---

1. Introduction

1.1 Purpose

This document specifies the architecture, data model, API, module design and cross-cutting concerns of TaskFlow as it is implemented with ASP.NET Core (.NET 10), EF Core on SQLite and Angular 19. It keeps traceability to the BRD and the 20 screen specifications.

1.2 Intended Audience

Reader	Uses this document to
Developer / maintainer	Understand the structure, conventions and where each feature lives
Reviewer / technical lead	Check the design against the BRD and the acceptance criteria (§21)
Analyst / designer	Move between this TDD and the BRD, FRD, HLD and LLD of a sub-module (§1.5, §1.6)
Portfolio reviewer	Evaluate the design, its decisions (§2.3) and its deviations from the BRD (§23)
AI / automation contributor	Follow the layer and folder rules (§5) enforced by tests

1.3 Scope

In scope (BRD): the 20 screens, the automatic deadline-notification workflow, business rules, non-functional requirements, database requirements and acceptance criteria defined in the BRD.

Implemented beyond the BRD:

Capability	Summary	Section
Roles and permissions	Fixed permission catalog (70 keys), read-only system roles, custom roles (Role Configuration screen)	§9
Two-step verification (MFA)	Mandatory TOTP authenticator + 10 one-time backup codes; Security Setup screen; admin MFA reset	§9, §14
Password recovery	Self-service reset with an authenticator code (Account Recovery screen); no emailed passwords	§9
Internal Mail	Mailbox between TaskFlow users (folders, drafts, reply/forward, attachments); replaces SMTP email	§15
Projects, positions, allocations	Project Management with members, positions and allocation percentages; departments with members/managers	§10
TFA assistant	AI chat panel (Ollama, Gemini, OpenAI) with confirm-before-act actions, audit, rate limit, voice	§16
Dashboard analytics	Analytics and filters per dashboard scope	§8.3
Legal pages	Copyright, terms, privacy, disclaimer, licences; ownership footer	§12
Data folder	All local data under `C:\TaskFlow` (database, key ring, attachments)	§2.2, §18

Out of scope: items listed in §24 Future Enhancements.

1.4 Definitions

Term	Meaning
SPA	Single Page Application: the Angular 19 frontend
API	The ASP.NET Core Web API (`TaskFlow.Api`), REST/JSON under `/api/v1`
Scheduler	`ReminderSchedulerService` , the `BackgroundService` that generates deadline reminders
Recipient	A user configured to receive reminders for a task (`TaskRecipient`), in addition to the assignee
Permission key	A dotted string such as `tasks.create` from `PermissionKeys.cs` ; each maps to the policy `perm:<key>`
Permission catalog	The fixed list of permissions with role defaults and assignability (`PermissionCatalog.cs`)
System role	Administrator, Manager or Employee (`SystemRoles.cs`); created by the application, read-only
Custom role	A role an Administrator creates from catalog permissions under Settings → Role Configuration
Scope	Data visibility of a dashboard or task list: Own, Team or Organization
MFA	Multi-factor (two-step) authentication: password plus a one-time code
TOTP	Time-based One-Time Password (6-digit authenticator code), implemented in `Totp`
Backup code	One of 10 single-use recovery codes issued at MFA setup, stored only as hashes (`MfaBackupCode`)
Security stamp	Per-user value (`User.SecurityStamp`) embedded in tokens; a change invalidates existing sessions
TFA	"TaskFlow Assistant", the built-in AI chat assistant
Pending action	An assistant action (`AssistantPendingAction`) that runs only after the user confirms it
Key ring	ASP.NET Core Data Protection keys in `dp-keys` , used to encrypt AI provider keys and MFA secrets
Internal Mail	TaskFlow's own user-to-user mailbox; nothing leaves the application

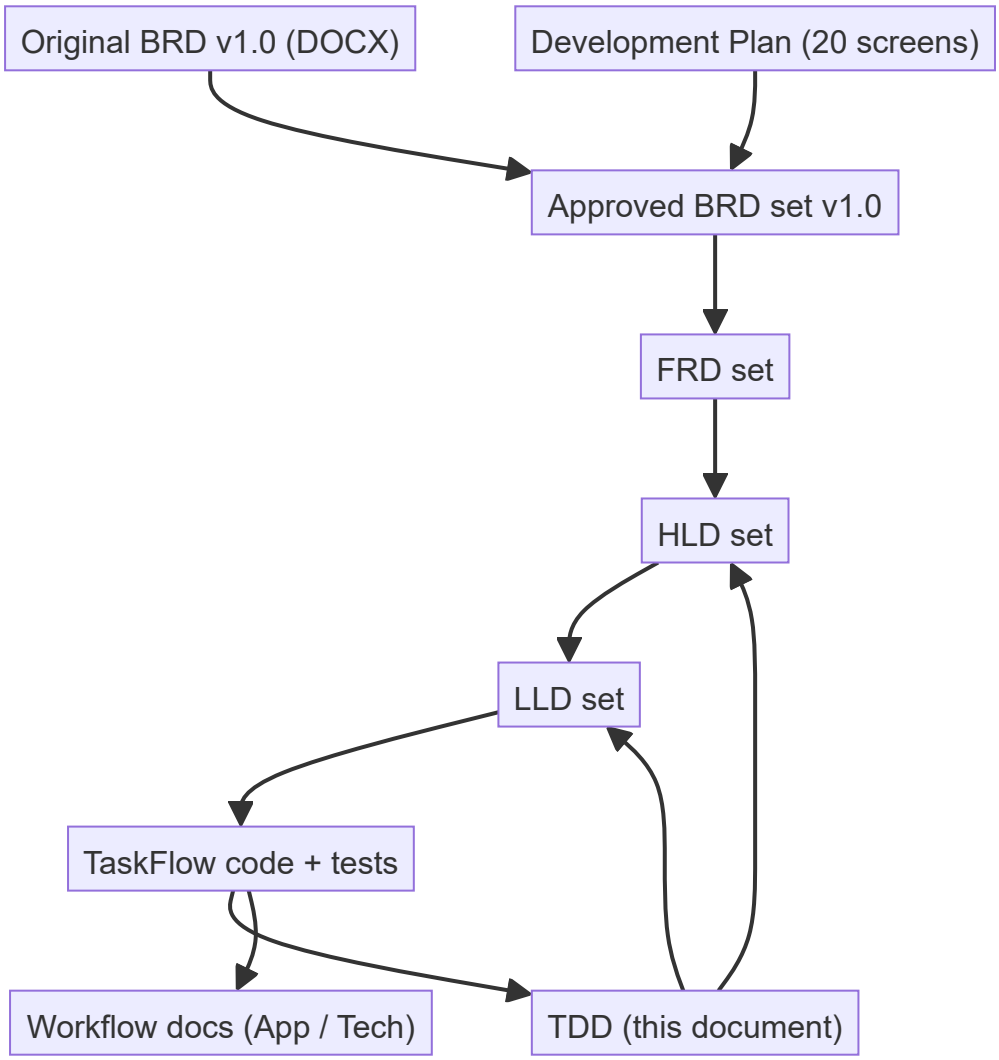
1.5 Reference Documents

All paths are relative to the project root (`TaskFlow Project/`). Every Markdown document has a PDF with the same name in the same folder.

Document	Purpose	Location	Version / status
BRD – Business Requirements Document (overall + 16 sub-modules)	Business goals, roles, scope, workflows, screen requirements (`REQ-…`), business rules, NFRs, acceptance criteria	`TF - Documents/TF - BRD/<module>/<sub-module>/BRD-<Sub-module>.md` · overall: `TF - Documents/TF - BRD/00-Overall/BRD-TaskFlow-Overall.md`	v1.0 · **Approved**, signed off 31-Oct-2026 (requirement baseline)
FRD – Functional Requirements Document (overall + 16)	Testable functional requirements (`FR-…`), use cases (`UC-…`), validations, messages, permissions; traces to the BRD	`TF - Documents/TF - FRD/…/FRD-<Sub-module>.md`	v1.0 · Draft
HLD – High-Level Design (overall + 16)	Context, components, data architecture, key flows, security, deployment, design decisions (`D-…`)	`TF - Documents/TF - HLD/…/HLD-<Sub-module>.md`	v1.0 · Draft
LLD – Low-Level Design (overall + 16)	Classes and files, every controller action, DTOs, validators, tables, sequence diagrams, error handling, FR → code → test	`TF - Documents/TF - LLD/…/LLD-<Sub-module>.md`	v1.0 · Draft (verified against the code: 114 actions, 70 keys, 31 entities)
TDD – this document	Whole-system technical design synced to the code	`TF - Documents/TF - TDD/TDD-TaskFlow.md`	v2.1
Original BRD	First requirement statement (20 screens); superseded by the approved BRD set	Retired – no longer in the repository	v1.0 · superseded
Development Plan	Screen-by-screen feature plan, 20 screens (`01-login-screen.md` … `20-application-settings-screen.md`)	`TF - Documents/Screens Wise Feature Development Plan/`	Reference
Permission documents	All permissions with purpose, and the Administrator, Manager and Employee permissions module-, sub-module- and screen-wise	`TF - Documents/Permission Requirement/Permissions in Excel/01-All-Permissions.xlsx` … `04-Employee-Permissions.xlsx` · PDF copies in `Permissions in PDF/`	Matches Appendix C
Application workflow docs	Per module and child module: purpose, screens, who can see and use each feature, permissions, step-by-step flows	`TaskFlow Proj/TFWorkflow/App WF in MD/` (`01-Auth` … `10-Legal`) · PDF in `App WF in PDF/`	Verified from the code
Technical workflow docs	Per module and child module: endpoints, services, data, permissions checks, known gaps	`TaskFlow Proj/TFTechnicalWorkflow/Tech WF in MD/` · PDF in `Tech WF in PDF/`	Verified from the code (source of §22.4)
README	Project overview, technologies, setup, run, documentation index	`README.md`	–

Document	Purpose	Location	Version / status
Developer overview / contributor rules	Developer orientation; layer and folder rules for AI and automation contributors	`TaskFlow Proj/project.md`, `TaskFlow Proj/agent.md`	–
OpenSpec	Change proposals, designs, tasks and the main specs	`openspec/changes/`, `openspec/specs/`	–
Documentation skills	Templates and procedures that produce and maintain these documents: `brd-writer`, `frd-writer`, `hld-writer`, `lld-writer`, `module-docs`, `tdd-sync` (fact extraction + `verify_tdd.py`), `requirements-to-markdown`	`.claude/skills/<skill>/SKILL.md`	–

1.6 Document Hierarchy & Traceability



The BRD states *what* is required, the FRD *how it behaves*, the HLD *how it is structured*, the LLD *how each part is built*. The TDD is the single whole-system technical reference that the HLD and LLD cite by section number; its section numbers are therefore kept stable.

ID schemes used across the set (`<CODE>` is the sub-module code in the table below):

Document	Identifiers
BRD – overall	`OBJ-nn`, global rules `BR-01 – BR-29`, `NFR-01 – NFR-20`, `AC-01 – AC-26`, `REQ-AUTH-nn`, `REQ-LEGAL-nn`
BRD – sub-module	`REQ-<CODE>-nn`, `BR-<CODE>-nn`, `AC-<CODE>-nn`, `OBJ-<CODE>-nn`, `NFR-<CODE>-nn`, `R-<CODE>-nn`
FRD	`FR-<CODE>-nn` (traces to `REQ-<CODE>-nn`), use cases `UC-<CODE>-nn`; overall `FR-AUTH-nn`, `FR-LEGAL-nn`
HLD	design decisions `D-<CODE>-n` (overall `D-nn`); traceability `FR-…` → component
LLD	traceability `FR-…` → endpoint / class → test
TDD (this document)	`BRD §x`, `Screen n`, `BR-n`, `FR-n` refer to the **original** BRD v1.0 numbering; use the tables below to reach the approved documents

Sub-module document map. Each sub-module has a BRD, FRD, HLD and LLD with the same stem (`TF - Documents/TF - <TYPE>/<Folder>/<TYPE>-<Stem>.md`).

Code	Sub-module	Folder	Stem	Original BRD screens	TDD sections	Workflow docs (App / Tech)
ALL	Overall (incl. Authentication, Legal)	`00-Overall`	`TaskFlow-Overall`	1; Account Recovery, Security Setup, Legal pages (added)	§1 – §5, §8.2, §9, §12, §14, §17 – §24	`01-Auth`, `10-Legal`
DASH	Dashboard	`01-Dashboard/01.1-Dashboard`	`Dashboard`	2, 3, 4	§8.3, §7.5	`02-Dashboard`
TASK	Task Management	`02-TaskFlow-Monitoring/02.1-Task-Management`	`Task-Management`	7, 8, 9, 11, 15, 18	§8.5, §10.1 – §10.10, Appendix A	`04-Monitoring` (04.1, 04.2)
WORK	Employee Workload	`02-TaskFlow-Monitoring/02.2-Employee-Workload`	`Employee-Workload`	16	§8.5	`04-Monitoring` (04.4)
BOARD	Board	`02-TaskFlow-Monitoring/02.3-Board`	`Board`	10	§8.5, §10.2, Appendix A	`04-Monitoring` (04.3)
EMP	Employee Management	`03-TaskFlow-Management/03.1-Employee-Management`	`Employee-Management`	5	§8.4, §10.11	`03-Management` (03.1)
PROJ	Project Management	`03-TaskFlow-Management/03.2-Project-Management`	`Project-Management`	– (added)	§8.4, §10.13	`03-Management` (03.3)
DEPT	Department Management	`03-TaskFlow-Management/03.3-Department-Management`	`Department-Management`	6	§8.4, §10.12	`03-Management` (03.2)
MAIL	Mail	`04-Communication/04.1-Mail`	`Mail`	14 (superseded by internal Mail)	§8.6, §15	`05-Communication` (05.2)

Code	Sub-module	Folder	Stem	Original BRD screens	TDD sections	Workflow docs (App / Tech)
NOTIF	Notification (incl. reminder engine)	`04-Communication/04.2-Notification`	`Notification`	12	§8.6, §11, Appendix B	`05-Communication` (05.1)
RPT	Reports	`05-Reports/05.1-Reports`	`Reports`	17	§8.7	`06-Reports`
REM	Reminder Settings	`06-Settings/06.1-Reminder-Settings`	`Reminder-Settings`	13	§8.9, §11.3, Appendix B	`08-Settings` (08.2)
APP	Application Settings	`06-Settings/06.2-Application-Settings`	`Application-Settings`	20	§8.9, §18.3	`08-Settings` (08.1)
TFAS	TFA Settings	`06-Settings/06.3-TFA-Settings`	`TFA-Settings`	– (added)	§8.9, §16.4	`08-Settings` (08.4)
ROLE	Role Configuration	`06-Settings/06.4-Role-Configuration`	`Role-Configuration`	– (added)	§8.9, §9.8, §9.9, Appendix C	`08-Settings` (08.3)
PROF	My Profile	`07-My-Profile/07.1-My-Profile`	`My-Profile`	19	§8.8, §9.3, §9.7, §10.14	`07-Profile`
TFA	TFA Assistant	`08-TFA-Assistant/08.1-Assistant`	`Assistant`	– (added)	§8.10, §16	`09-Assistant`

Example: original Screen 15 (Task Reassignment) → `TF - Documents/TF - BRD/02-TaskFlow-Monitoring/02.1-Task-Management/BRD-Task-Management.md`, the matching `FRD-`, `HLD-` and `LLD-Task-Management.md`, and TDD §10.5.

---

2. Design Goals & Constraints

2.1 Goals (from BRD §1.2)

1. Simplify employee task allocation.
2. Monitor progress of assigned tasks.
3. Reduce missed deadlines via automatic notifications.
4. Notify assigned employee and selected recipients about approaching deadlines.
5. Help managers identify pending and overdue tasks.
6. Improve manager–employee communication (comments).
7. Maintain a history of task updates and changes.
8. Generate reports for planning; support CSV export.

2.2 Constraints

Constraint	How the implementation meets it	Source
Single-server deployment, SQLite file database	One API process; database `taskflow.db` via EF Core SQLite	BRD §7
One local data folder	`C:\TaskFlow` on Windows (`TaskFlow` under the common application-data folder elsewhere), override `Storage:Root`; holds `taskflow.db`, `dp-keys`, `mail-attachments`. `StorageRoot.EnsureAvailable` creates it and stops startup with a clear message if it is not writable	`Persistence/StorageRoot.cs`
Single instance	The scheduler and the attempt limiters (in-memory sliding window, reset on restart) run in-process; no distributed coordination	`ReminderSchedulerService`, `AttemptLimiter`
Demonstrable offline	In-app notifications and internal Mail need no external server; Ollama runs locally; only Gemini/OpenAI need the internet	BRD §3.4
Consistent application timezone	`Application:Timezone` (default `Asia/Kolkata`) seeds the application setting used by `AppClock` for all deadline comparisons	BRD BR-16
Scheduler resumes after restart without duplicates	Stateless pass at startup and every `Reminders:TickSeconds` (60 s); duplicates prevented by the database (see §11)	BRD §5.4, §3.5
No cross-user access to tasks, notifications, mail	Permission policies + `TaskAccess` resource check; mailbox items owned per user	BRD §5.2, BR-13
Schema created automatically	`Database.MigrateAsync()` and seeders run at startup	BRD §9

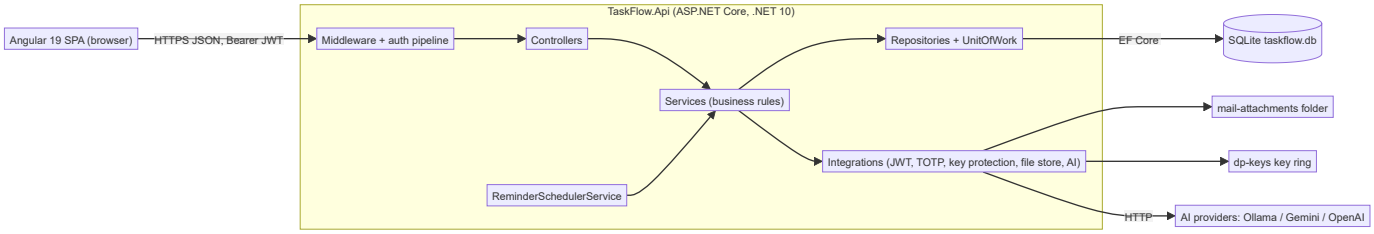
2.3 Key design decisions

#	Decision	Rationale	Status vs v1.0
D1	Angular SPA + REST API under `/api/v1`	Clear separation, maintainability	Kept
D2	EF Core code-first migrations on SQLite (21 migrations)	Reproducible schema; applied automatically at startup	Kept
D3	JWT bearer access token + refresh-token cookie; session revalidated on every request (`SessionValidation`)	Stateless API, yet role, permission, deactivation and password changes apply on the next request	Extended
D4	`BackgroundService` scheduler + database dedup as reminder engine	No external broker; restart-safe	Kept
D5	Overdue derived at query time from status + deadline (no `Overdue` status)	Single source of truth, never stale	Kept
D6	Soft delete (`IsDeleted`) for history-bearing entities	Protects historical records (BR-18)	Kept

#	Decision	Rationale	Status vs v1.0
D 7	One API project `TaskFlow.Api`, layer-first with module sub-folders; Controller → Service → Repository, commits via `IUnitOfWork`	Simpler build, same separation; enforced by `ArchitectureTests`	Replaces 4-project layout
D 8	Permission-based authorization from a fixed catalog; system roles read-only, custom roles configurable	Fine-grained access; every checkbox is enforced	Replaces fixed role checks
D 9	Mandatory MFA (`Mfa:Required` = true), enforced server-side by `MfaSetupGateMiddleware`	Security layer that the SPA cannot bypass	New
D 10	Internal Mail instead of SMTP; reminders can be copied into Mail	Works offline, no credentials to store, nothing leaves TaskFlow	Replaces SMTP (BRD Screen 14)
D 11	Pluggable AI providers (Ollama, Gemini, OpenAI) behind `ChatModelGateway`; server-side permission checks and confirm-before-act	Free local option; model never decides access	New
D 12	All local state in one data folder; provider keys and MFA secrets encrypted with the Data Protection key ring (DPAPI on Windows)	Simple backup; secrets not stored in plain text	New

3. System Architecture

3.1 High-level view



3.2 Runtime components

#	Component	Responsibility
1	Angular SPA	All screens (20 BRD screens + added ones); lazy-loaded module routes; guards and interceptors; calls only the API
2	Web API (`TaskFlow.Api`)	Authentication, MFA, authorization, business rules, reporting, CSV export, internal mail, assistant
3	`ReminderSchedulerService`	Hosted `BackgroundService`; runs a reminder pass at startup and every `Reminders:TickSeconds`; disabled by `Reminders:SchedulerEnabled` = false
4	SQLite database	`taskflow.db` in the data folder; all tables (§7)
5	Local file store	`LocalMailFileStore` keeps mail attachments under generated names in `Mail:AttachmentsPath` (`mail-attachments`)

#	Component	Responsibility
6	Data Protection key ring	`dp-keys` (`AiProviders:KeyRingPath`); encrypts AI provider API keys (`AiKeyProtector`) and MFA secrets (`MfaSecretProtector`); DPAPI-protected on Windows, checked at startup by `KeyRingProtectionCheck`
7	AI providers	`OllamaProvider` , `GeminiProvider` , `OpenAiProvider` via typed `HttpClient` s (timeout `AiProviders:TimeoutSeconds` = 60, no redirects, no header/body logging)

3.3 Startup sequence (`Program.cs`)

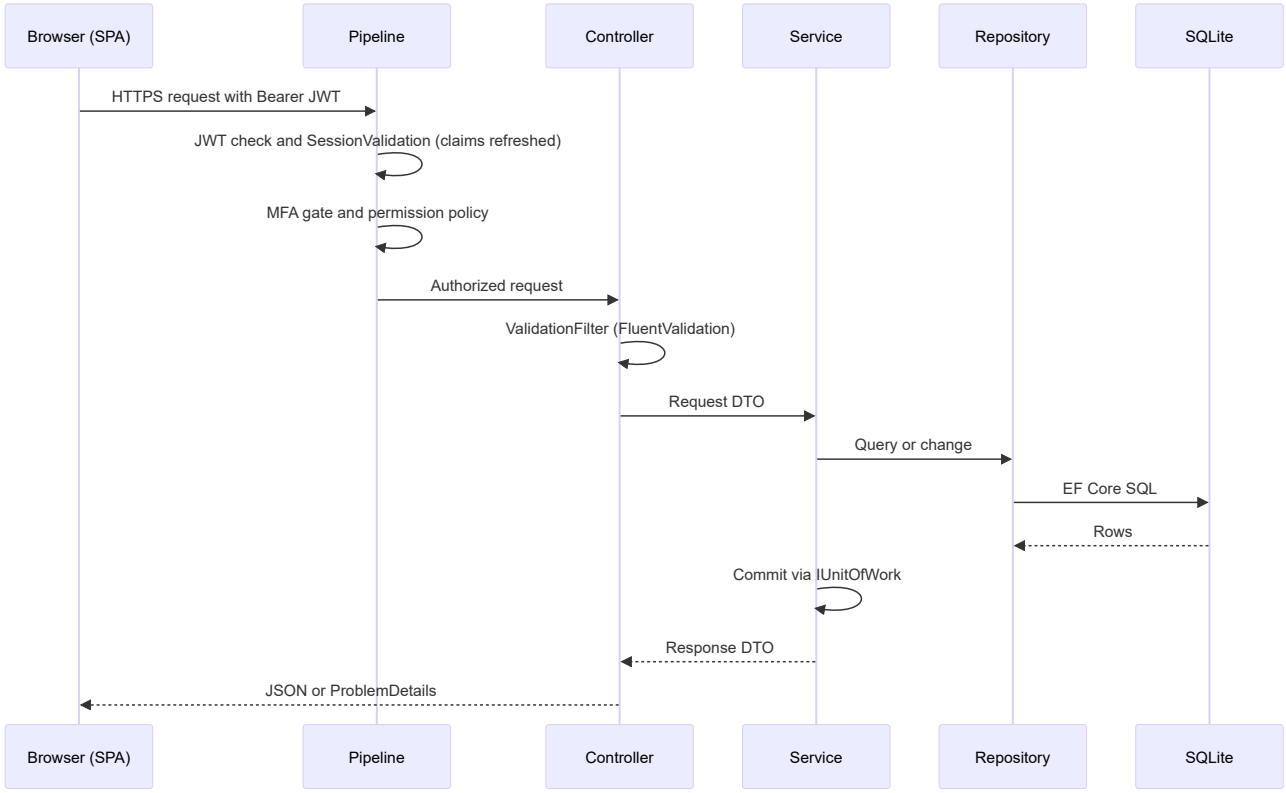
Step	Action
1	Load `appsettings.json` , optional `appsettings.Environment.json` , environment variables
2	Register layers: `AddCrossCutting` , `AddRepositories` , `AddServices` , `AddIntegrations` (`Config/`)
3	Resolve and verify the data folder (`StorageRoot.Resolve` , `StorageRoot.EnsureAvailable`); warn if a legacy `database/taskflow.db` exists
4	On Windows, warn about unprotected keys in the key ring (`KeyRingProtectionCheck`)
5	Apply migrations (`MigrateAsync`), then `SettingsInitializer` , `AccessControlSeeder` , `AiProviderSeeder` , `DatabaseSeeder` (demo users only when `Seed:Enabled`)
6	Development only: OpenAPI document and the diagnostic route
7	Build the request pipeline (§3.4) and start the scheduler

3.4 Request lifecycle

Middleware order (from `Program.cs`): `UseForwardedHeaders` → `RequestLoggingMiddleware` → `UseExceptionHandler` (`GlobalExceptionHandler`) → `UseHttpsRedirection` → `UseCors` (`SpaClient` policy, origins from `Cors:AllowedOrigins`) → `UseAuthentication` (JwtBearer, `SessionValidation` on token validated) → `MfaSetupGateMiddleware` → `UseAuthorization` → controllers (`ValidationFilter` runs first).

Stage	Behaviour
Forwarded headers	Trusts `X-Forwarded-For` only from loopback or `Proxy:KnownProxies` , so the real client address reaches the recovery throttle
Request logging	One entry per request: method, path, status, elapsed ms, user id, trace id; bodies are never logged
JWT validation	Issuer `Jwt:Issuer` , lifetime and signing key checked; `SessionValidation` rejects the token if the user is gone, inactive or the security stamp changed, then replaces role and permission claims with the current ones from the database and adds the MFA-setup-required claim when needed
MFA gate	A signed-in user without an authenticator gets 403 with code `mfa_setup_required` for every API call except `/api/v1/auth/*` , `/api/v1/users/me` and `/api/v1/users/me/mfa/*`
Authorization	Policy `perm:<key>` per catalog permission (`[RequirePermission]` or `[Authorize(Policy = …)]`); `TaskAccess` resource handler for task-scoped access; refusals are audited by `DeniedActionAuditHandler`
Validation	`ValidationFilter` runs the FluentValidation validator of each bound argument; failure = 400 validation ProblemDetails with an `errors` map
Business logic	Controller → Service (rules, scope checks) → Repository → EF Core → SQLite; the service commits through `IUnitOfWork`

Stage	Behaviour
Errors	Services throw `AppException` subclasses; `GlobalExceptionHandler` maps them to ProblemDetails (400/403/404/409/429), unknown errors to a generic 500



3.5 Frontend runtime

`app.config.ts` registers the router, `HttpClient` with `apiBaseUrlInterceptor` and `authInterceptor`, and an app initializer that restores the session (`AuthService.restoreSession`). Authenticated screens are children of `ShellComponent`, guarded by `authGuard` and `mfaSetupGuard` (details in §12).

---

4. Technology Stack & Rationale

4.1 Backend

Layer	Technology	Version	Purpose
Runtime	.NET (`net10.0`)	10	API host
Web framework	ASP.NET Core Web API	10	REST endpoints, ProblemDetails, hosted services
OpenAPI	`Microsoft.AspNetCore.OpenApi`	10.0.8	OpenAPI document (Development only)
ORM	`Microsoft.EntityFrameworkCore` (+ `.Design`)	10.0.12	Data access, code-first migrations
Database provider	`Microsoft.EntityFrameworkCore.Sqlite`	10.0.12	SQLite file database

Layer	Technology	Version	Purpose
Validation	`FluentValidation` + `FluentValidation.DependencyInjectionExtensions`	12.2.0	Request validators, run by `ValidationFilter`
Authentication	`Microsoft.AspNetCore.Authentication.JwtBearer`	10.0.12	Bearer access tokens
Tokens	`System.IdentityModel.Tokens.Jwt`	8.23.0	Token creation (`JwtTokenService`)
SPA integration	`Microsoft.AspNetCore.SpaProxy`	10.0.12	Starting the API launches `npm start` on `http://localhost:4200`
Password hashing	`PasswordHasher<User>` (PBKDF2)	built-in	Password hashes
Secret protection	ASP.NET Core Data Protection (DPAPI on Windows)	built-in	AI provider keys, MFA secrets
MFA	`Totp` , `Base32` (in-house integrations)	n/a	Authenticator codes (TOTP)
Scheduler	`BackgroundService` + `PeriodicTimer`	built-in	Deadline reminders
Logging	`Microsoft.Extensions.Logging` (`Logging:LogLevel`)	built-in	Request log and application log
AI providers	`HttpClient` to Ollama, Gemini, OpenAI	n/a	TFA assistant

4.2 Frontend

Layer	Technology	Version	Purpose
Framework	Angular (standalone components, `@angular/router` , `@angular/forms`)	^19.2.0	SPA
Language	TypeScript	~5.7.2	Typed code
Reactive	RxJS	~7.8.0	HTTP and state streams
Charts	`chart.js` + `ng2-charts`	^4.6.1 / ^8.0.0	Dashboard and analytics charts
Drag and drop	`@angular/cdk`	^19.2.19	Kanban board
QR codes	`qrcode`	^1.5.4	Authenticator setup QR code
Styling	Custom CSS design tokens and `.tf-*` primitives (no CSS framework)	n/a	Layout, themes
Build / CLI	`@angular/cli` , `@angular-devkit/build-angular`	^19.2.27	Build, dev server

4.3 Testing and tooling

Area	Technology	Notes
Backend tests	xUnit: `TaskFlow.UnitTests` (100 files) and `TaskFlow.IntegrationTests` (50 files)	Integration tests use the API's `Program` via `WebApplicationFactory`

Area	Technology	Notes
Architecture rules	`TaskFlow.UnitTests/Architecture/ArchitectureTests.cs`	Layer, folder and naming rules (§5.4)
Frontend tests	Jasmine ~5.6 + Karma ~6.4 (Chrome launcher, coverage)	`npm test` runs `check:structure` then `ng test`
Frontend structure	`npm run check:structure` (`scripts/check-frontend-structure.mjs`)	Folder placement rules (§5.4)

Not used: Serilog, Playwright, AutoMapper, Bootstrap, SMTP client (all listed in v1.0, none installed).

4.4 Why this stack fits the BRD

- EF Core migrations applied at startup satisfy "database schema and initialization scripts" (BRD §9).
- A hosted service satisfies "scheduler resumes after restart" (BRD §5.4) without extra infrastructure.
- Permission policies on the server plus Angular guards enforce role-based access (BRD §5.2).
- Custom CSS tokens cover the responsive layout the BRD expected from Bootstrap, without a framework dependency.

4.5 Mapping from the BRD's recommended stack

BRD component	BRD choice	Implementation
Backend	Python Flask	ASP.NET Core Web API (.NET 10)
ORM	SQLAlchemy	EF Core 10
Database	SQLite	SQLite (`taskflow.db` in `C:\TaskFlow`)
Scheduled reminders	APScheduler	`ReminderSchedulerService` (`BackgroundService`)
Authentication	Flask sessions + hashing	JWT + refresh-token cookie, `PasswordHasher` (PBKDF2), mandatory TOTP MFA
Frontend	HTML/CSS/JS/Bootstrap	Angular 19 + custom CSS tokens
Charts	Chart.js	Chart.js 4 via ng2-charts 8
Email	SMTP	Internal Mail (no SMTP), see §15 and §23
Testing	Pytest	xUnit + Jasmine/Karma

---

5. Solution Structure

5.1 Repository layout

```
TaskFlow Project/
├── README.md
├── TF - Documents/
│   ├── TF - BRD/                approved BRD set: 00-Overall + 16 sub-modules (MD + PDF)
│   ├── TF - FRD/                FRD set, same folder layout
│   ├── TF - HLD/                HLD set, same folder layout
│   ├── TF - LLD/                LLD set, same folder layout
│   ├── TF - TDD/                TDD-TaskFlow.md / .pdf (this document)
```

```
|   ├── Screens Wise Feature Development Plan/   01..20-*.md
|   └── Permission Requirement/   Permissions in Excel/ and Permissions in PDF/ (All,
Administrator, Manager, Employee)
├── openspec/                     OpenSpec changes and specs
├── scripts/                      setup-ai.ps1 / setup-ai.sh (local Ollama), openspec-
update.ps1
├── .claude/skills/               documentation and workflow skills (brd/frd/hld/lld-writer,
module-docs, tdd-sync, …)
└── TaskFlow Proj/
    ├── backend/
    |   ├── TaskFlow.sln
    |   ├── src/TaskFlow.Api/      the only API project
    |   └── tests/
    |       ├── TaskFlow.UnitTests/
    |       └── TaskFlow.IntegrationTests/
    ├── frontend/                 Angular 19 workspace (src/app/...)
    ├── TFWorkflow/               application workflow docs per module (App WF in MD / App WF
in PDF)
    ├── TFTechnicalWorkflow/      technical workflow docs per module (Tech WF in MD / Tech WF
in PDF)
    ├── project.md                developer overview
    └── agent.md                  rules for AI and automation contributors
```

The data folder (`C:\TaskFlow`) is outside the repository; the earlier `database/` folder is no longer used.

5.2 Backend: `TaskFlow.Api` (layer-first, module sub-folders)

Folder	Contains
`Controllers/<Module>/`	HTTP only: routing, binding, status codes (e.g. `Controllers/Auth/PasswordRecoveryController.cs`)
`Services/<Module>/<SubModule>/[<Pair>/]`	`I<Name>Service` + `<Name>Service` , one pair per folder; shared rules in `Rules/` ; assistant help topics in `Knowledge/` (embedded `.md`)
`Repositories/<Module>/<SubModule>/[<Pair>/]`	`I<Name>Repository` + `<Name>Repository` over EF Core; `IBaseRepository` , `BaseRepository` , `IUnitOfWork` , `UnitOfWork` in `Base/`
`Dtos/<Module>/<SubModule>/`	Request/response records and `Validators/`
`Models/Entities/` , `Models/Enums/`	EF Core entities and enums (§7)
`Integrations/<Area>/<Pair>/`	`Ai/` (Providers, Registry, KeyProtection), `Security/` (Tokens, OneTimeCodes, MfaSecrets), `Storage/` (`LocalMailFileStore`)
`Middleware/`	`GlobalExceptionHandler` , `RequestLoggingMiddleware` , `SessionValidation` , `MfaSetupGateMiddleware`
`Filters/`	`ValidationFilter`

Folder	Contains
`Authorization/`	`Policies/` (`RequirePermissionAttribute`, `AuthPolicies`), `TaskAccess/` (`TaskAccessHandler`, `TaskAccessRequirement`), `Assistant/` (`AssistantAuthorizer`), `Auditing/` (`DeniedActionAuditHandler`)
`Config/`	DI registration per layer (`CrossCuttingRegistration`, `RepositoryRegistration`, `ServiceRegistration`, `IntegrationRegistration`) and options classes (`JwtOptions`, `MfaOptions`, `MailOptions`, `ReminderEngineOptions`, `AiProviderOptions`, `ApplicationOptions`, `CorsPolicies`)
`Common/`	`Exceptions/`, `Security/` (`PermissionKeys`, `PermissionCatalog`, `SystemRoles`, `PermissionPolicy`, `AuthClaims`, `EffectivePermissions`, `RefreshTokenCookie`), `Rules/` (e.g. `ReminderCalculator`, `TaskBucketRules`, `EmailPolicy`), `Constants/`, `Models/`, `Utils/`
`Persistence/`	`AppDbContext`, `AppDbContextFactory`, `DatabasePathResolver`, `StorageRoot`, `Migrations/`, `Seeders/`

Modules (same tree in every layer, following the permission catalog): `Auth` (Login, Sessions, Mfa, PasswordRecovery, AttemptLimits), `Dashboard` (Overview, Analytics), `Management` (EmployeeManagement with Allocations, ProjectManagement with Positions, DepartmentManagement), `Monitoring` (TaskManagement with Discussion and Activity, EmployeeWorkload; Board is frontend-only), `Communication` (Mail with Attachments, Notification with Reminders), `Reports`, `Profile`, `Settings` (ApplicationSettings, ReminderSettings, RoleConfiguration, TfaSettings), `Assistant`.

Controllers by module:

Module	Controllers
Auth	`AuthController`, `PasswordRecoveryController`
Dashboard	`DashboardController`, `AnalyticsController`
Management	`EmployeesController`, `DepartmentsController`, `ProjectsController`, `PositionsController`
Monitoring	`TasksController`, `TaskCommentsController`, `TaskActivityController`, `WorkloadController`
Communication	`MailController`, `NotificationsController`
Reports	`ReportController`
Profile	`ProfileController`
Settings	`ApplicationSettingsController`, `ReminderSettingsController`, `RolesController`, `AiProvidersController`
Assistant	`AssistantController`

5.3 Frontend: `frontend/src/app`

```
src/app/
  modules/<module>/<sub-module>/   pages/ components/ services/ models/ utils/
  modules/<module>/<module>.routes.ts   loaded by routes/app.routes.ts with loadChildren
  shared/      auth/ (guards, interceptor, auth service, token store, permissions),
components/,
               directives/, services/, utils/, testing/
  layouts/     ShellComponent, nav-items.ts, theme menu
```

```
  routes/      app.routes.ts (root route table)
  config/      app.config.ts, API base URL and its interceptor
```

Module	Sub-modules / routes
`auth` (pre-sign-in)	`login`, `account-recovery`, `security-setup`
`legal` (pre-sign-in)	`legal/copyright`, `legal/terms`, `legal/privacy`, `legal/disclaimer`, `legal/licenses`
`dashboard`	`admin`, `manager`, `me`; `analytics`
`management`	`employee-management` (`employees`), `project-management` with positions (`projects`, `projects/:id`), `department-management` (`departments`)
`monitoring`	`task-management` with discussion and activity (`tasks`, `tasks/new`, `tasks/:id`, `tasks/:id/discussion`, `tasks/:id/activity`, `tasks/:id/reassign`), `board`, `employee-workload` (`workload`)
`communication`	`mail`, `notification` (`notifications`)
`reports`	`reports`
`profile`	`profile`, `profile/change-password`
`settings`	`settings/reminders`, `settings/application`, `settings/tfa`, `settings/permissions`
`assistant`	TFA panel (components, services, models; no route)

5.4 Structure enforcement

Check	Runs	Enforces
`ArchitectureTests` (xUnit)	`dotnet test`	Controller → Service → Repository only, through interfaces; only `Persistence`, `Repositories` and startup touch `AppDbContext`; repositories never return `IQueryable` or save (services commit via `IUnitOfWork`); controllers take/return no entities, do not validate or catch; namespaces match folders; `*Controller` / `*Service` / `*Repository` naming; one interface per file beside its class; one pair per folder; no empty folders
`npm run check:structure`	`npm test` (first step)	Files in the layer folder for their kind; only `<module>.routes.ts` at module root; API services in the module whose endpoints they call; `loadComponent` targets in `pages/`; code in `shared/` only when used by two or more modules; no `core/` or `features/` folders; no empty folders

Tests mirror the code folders (e.g. `TaskFlow.UnitTests/Auth/Login/…`); cross-module tests sit in `Architecture/`, `Common/`, `Persistence/`, `Integrations/`, shared fixtures in `TestSupport/` (see §20).

---

6. Backend Design

The backend is one ASP.NET Core Web API project, `TaskFlow.Api` (`net10.0`). It is organised layer-first, with module sub-folders in each layer: `Controllers/<Module>`, `Services/<Module>`, `Repositories/<Module>`, `Dtos/<Module>`, plus the shared `Models`, `Integrations`, `Authorization`, `Middleware`, `Filters`, `Config`, `Common` and `Persistence` folders (see §5). Each layer registers its own types through one extension method in `Config/`: `AddCrossCutting`, `AddRepositories`, `AddServices`, `AddIntegrations`.

6.1 Cross-cutting concerns

Concern	Implementation (current code)
Validation	`Filters/ValidationFilter.cs` (a global MVC action filter) runs the FluentValidation validator registered for each bound action argument before the action runs. A failure short-circuits with **400** `ValidationProblemDetails` whose `errors` map names each invalid field. Validators are registered from the assembly (`AddValidatorsFromAssemblyContaining<LoginRequestValidator>`). Services can also throw `RequestValidationException` for field errors found later. Server-side validation is authoritative. Frontend validation is for the user only.
Error responses	`AddProblemDetails()` + `Middleware/GlobalExceptionHandler.cs` (`IExceptionHandler`). Every error is RFC 7807 `application/problem+json` and carries a trace id. Unknown exceptions are logged in full, and the client gets a generic 500 (`"Something went wrong. Please try again."`). See §6.2.
Concurrency	Concurrency tokens on `User.MfaLastUsedStep`, `User.MfaFailedAttempts`, `Task.ActualHours`, `MfaBackupCode.UsedAt`, `PasswordResetGrant.UsedAt`. `UnitOfWork.SaveChangesAsync` turns `DbUpdateConcurrencyException` into `ConcurrencyConflictException` (409), and SQLite `UNIQUE` failures into `UniqueConstraintException` (409).
JSON	`System.Text.Json` with `JsonStringEnumConverter`: enums travel as their names (for example `"InQA"`), matching the SPA's string unions.
Logging	Built-in `Microsoft.Extensions.Logging` (levels from `Logging:LogLevel:*`). `Middleware/RequestLoggingMiddleware.cs` writes one entry per request: method, path, status, elapsed ms, user id and trace id. Request bodies are never logged, because they carry passwords, authenticator codes and mail content. `Authorization/Auditing/DeniedActionAuditHandler.cs` logs every refused authorization as a *denied action* (user id, method, path, policy name only). No third-party logging library is installed.
Authentication	JWT bearer (`Microsoft.AspNetCore.Authentication.JwtBearer`). The issuer is validated (`Jwt:Issuer`), audience validation is off, and the lifetime and signing key are validated. `MapInboundClaims = false`. `OnTokenValidated` = `Middleware/SessionValidation.ValidateAsync` (see §6.6 and §9).
MFA gate	`Middleware/MfaSetupGateMiddleware.cs` runs after authentication. While the token's session has the `mfa_setup_required` claim, every API path except `/api/v1/auth/*`, `/api/v1/users/me/mfa/*` and `/api/v1/users/me` gets **403** with problem `code = mfa_setup_required`.
Authorization	One policy per catalog permission (`perm:<key>`). Resource policy `TaskAccess` is enforced by `TaskAccessHandler`. See §6.6.
CORS	Policy `CorsPolicies.SpaClient`: origins from `Cors:AllowedOrigins` (supplied per environment in `appsettings.Environment.json`), any header, any method, credentials allowed (needed for the refresh-token cookie).

Concern	Implementation (current code)
Forwarded headers	`UseForwardedHeaders` with `X-Forwarded-For` only. Only loopback proxies plus the addresses listed in `Proxy:KnownProxies` are trusted, so a caller cannot spoof the client IP that the password-recovery throttle uses.
API versioning	URL segment: every controller route starts with `/api/v1/`. There is no versioning package.
Time	`TimeProvider.System` is registered as a singleton. `IAppClock` (`AppClock`) resolves the application timezone (see §7.5).
OpenAPI	`Microsoft.AspNetCore.OpenApi`. `MapOpenApi()` runs only in Development. Development also maps a minimal-API fault probe, `/api/v1/diagnostics/throw`. It is not one of the 114 controller actions.
Configuration	`appsettings.json` → optional `appsettings.Environment.json` (reload on change) → environment variables. Options classes are in `Config/` (`JwtOptions`, `MfaOptions`, `MailOptions`, `ReminderEngineOptions`, `ApplicationOptions`, `AiProviderOptions`), plus `StorageOptions` in `Persistence/StorageRoot.cs`. See §18.

Pipeline order (`Program.cs`): `UseForwardedHeaders` → `RequestLoggingMiddleware` → `UseExceptionHandler` → `UseHttpsRedirection` → `UseCors(SpaClient)` → `UseAuthentication` → `MfaSetupGateMiddleware` → `UseAuthorization` → `MapControllers`. Before the pipeline starts, the application does the following in order: resolves and checks the data folder (`StorageRoot.EnsureAvailable`), warns about a legacy database, checks the DPAPI key ring on Windows (`KeyRingProtectionCheck`), applies migrations (`Database.MigrateAsync`), then runs `SettingsInitializer`, `AccessControlSeeder`, `AiProviderSeeder` and `DatabaseSeeder` (demo sign-ins only when `Seed:Enabled`, see §18).

6.2 Exception types and status codes

All expected failures derive from `Common/Exceptions/AppException` (abstract `StatusCode`, optional `Title`, optional `RetryAfter`). `GlobalExceptionHandler` maps them as follows:

Exception (namespace `TaskFlow.Api.Common.Exceptions` unless noted)	HTTP	Notes
`RequestValidationException`	400	`ValidationProblemDetails` with a field → messages map
`MailValidationException` (: `RequestValidationException`)	400	Mail compose/draft field errors
`BusinessRuleException`	400	A business rule is broken
`AnalyticsFilterException` (: `BusinessRuleException`)	400	Invalid dashboard filter or date range
`AssistantRequestException`	400	Invalid TFA request
`AssistantRateLimitedException` (: `AssistantRequestException`)	429	TFA per-minute limit, with `Retry-After`
`ForbiddenException` / `AnalyticsForbiddenException`	403	Caller may not see or change the resource

Exception (namespace `TaskFlow.Api.Common.Exceptions` unless noted)	HTTP	Notes
`NotFoundException` / `AnalyticsEntityNotFoundException`	404	Resource does not exist (or is outside the caller's mailbox or scope)
`ConflictException`	409	Resource is not in a state that allows the change
`ConcurrencyConflictException` (: `ConflictException`)	409	Lost save race. Title "The record was changed at the same time."
`UniqueConstraintException` (: `ConflictException`)	409	"That value is already in use."
`AssistantActionConflictException` (: `ConflictException`)	409	Pending action already resolved or expired
`RateLimitedException`	429	Title "Too many requests", `Retry-After` header in seconds
`RoleException` (`RoleFailure`)	404 / 400 / 403 / 409	`NotFound` → 404, `Invalid` → 400, `ReadOnly` (system role) → 403, others → 409
`DbUpdateConcurrencyException` (EF Core)	409	Handled directly as a retryable conflict
any other exception	500	Logged with method, path and trace id. The body has a generic detail only.

Authentication and authorization refusals keep the framework's own **401** / **403** responses. The MFA gate's 403 carries `code = mfa_setup_required`.

6.3 Service catalogue (by module)

Paths are relative to `Services/`. Each service has an `I<Name>` interface next to it unless it is a static rule or record type.

Module	Folder	Class	Responsibility
Auth	`Auth/Login`	`AuthService`	Login by email or username, MFA verify step, issuing access and refresh tokens, refresh and logout (see §9)
Auth	`Auth/Mfa`	`MfaService`	TOTP setup, enable and check, 10 backup codes, failed-attempt lock. Counters are concurrency tokens.
Auth	`Auth/PasswordRecovery`	`PasswordRecoveryService`	Recovery through an authenticator code → single-use reset grant (`TokenLifetime` 10 min) → password reset
Auth	`Auth/Sessions`	`SessionService`	Loads the per-request session (active flag, security stamp, role, effective permissions, MFA-setup flag) for `SessionValidation`
Auth	`Auth/AttemptLimits`	`AttemptLimiter`	In-memory sliding-window attempt limiter (capped table, idle keys swept)
Dashboard	`Dashboard/Overview`	`DashboardService`	Admin, Manager and Employee overview counts with day bounds in the application timezone

Module	Folder	Class	Responsibility
Dashboard	`Dashboard/Analytics/{Admin,Manager,Employee}`	`AdminAnalyticsService`, `ManagerAnalyticsService`, `EmployeeAnalyticsService`, `AnalyticsBuckets`	Analytics charts and KPIs per scope
Dashboard	`Dashboard/Analytics/Scopes`	`AdminDashboardScope`, `ManagerDashboardScope`, `EmployeeDashboardScope`, `AnalyticsFilterResolver`, `ResolvedAnalyticsFilter`, `ScopedTasks`	Organization / Team / Own task sets and filter resolution (default range, limits, timezone)
Management	`Management/EmployeeManagement/Employees`	`EmployeeService`	Employee CRUD, status, role assignment, MFA reset. Email uniqueness counts deleted users too.
Management	`Management/EmployeeManagement/Allocations`	`UserAllocationService`	Project allocations (percent, position, project-manager flag) and department memberships
Management	`Management/ProjectManagement/Projects`	`ProjectService`	Project CRUD, members, employees, project tasks
Management	`Management/ProjectManagement/Positions`	`PositionService`	List and create positions
Management	`Management/DepartmentManagement`	`DepartmentService`	Department CRUD and members
Monitoring	`Monitoring/TaskManagement/Tasks`	`TaskService`	Task list, board, details, create, edit, delete, status, reassign, recipients, hours. Enforces the visibility scope (§6.6) and the transition table.
Monitoring	`Monitoring/TaskManagement/Rules`	`TaskStatusTransitions`, `TaskAccessScope`	Pure transition rules (Appendix A) and the resource access rule
Monitoring	`Monitoring/TaskManagement/Access`	`TaskAccessService`	Backs `TaskAccessHandler`: live task + view-all / assignee / creator / recipient
Monitoring	`Monitoring/TaskManagement/Discussion`	`TaskCommentService`	Comments. Only the author may edit or soft-delete. Every change is logged and notifies other members.
Monitoring	`Monitoring/TaskManagement/Activity`	`TaskActivityService`	Append-only activity history. An administrator may correct or remove a worked-hours entry.
Monitoring	`Monitoring/EmployeeWorkload`	`WorkloadService`	Per-employee workload aggregation (Screen 16)

Module	Folder	Class	Responsibility
Communication	`Communication/Mail/Messages`	`MailService`	Internal mail: folders, send, drafts, reply/forward compose, flags, trash. Every query starts from the caller's own mailbox items (see §15).
Communication	`Communication/Mail/Attachments`	`MailAttachmentService`, `MailAttachmentDownload`	Validate, store and stream attachments. The content type comes from an extension allowlist.
Communication	`Communication/Notification/Notifications`	`NotificationService`	Read and mark read a user's own notifications (Screen 12)
Communication	`Communication/Notification/Publishing`	`NotificationPublisher`	Writes in-app notifications. Skips them while `Notification.InAppEnabled` is off.
Communication	`Communication/Notification/Reminders`	`ReminderService`, `ReminderSchedulerService`	Reminder pass (history rows, notifications, optional mail copy) and the hosted `BackgroundService` that runs it at startup and every `Reminders:TickSeconds` (see §11)
Reports	`Reports`	`ReportService`, `ReportCsv`	Filtered report aggregation and RFC 4180 CSV export with formula-injection guard
Profile	`Profile`	`ProfileService`	Own profile read and update, password change
Settings	`Settings/ApplicationSettings/General`	`ApplicationSettingsService`	Typed view of the `Application.*` / `Notification.*` setting rows (Screen 20)
Settings	`Settings/ApplicationSettings/Clock`	`AppClock`	Application timezone and local "today"
Settings	`Settings/ReminderSettings`	`ReminderSettingsService`	Typed view of the `Reminder.*` setting rows (Screen 13)
Settings	`Settings/RoleConfiguration`	`RoleService`	Custom role CRUD and permission grants. System roles and the catalog are read-only.
Settings	`Settings/TfaSettings/ProviderConfig`	`AiProviderConfigService`	AI provider configuration, models and protected API key
Settings	`Settings/TfaSettings/ConnectionTest`	`AiProviderConnectionTester`	Provider connection test
Assistant	`Assistant/Chat`	`AssistantService`, `AssistantPrompt`, `AssistantTexts`, `ProviderFailureMessages`	TFA conversation engine: length and secret checks → rate limit → provider with a tool loop (see §16)
Assistant	`Assistant/Actions`	`AssistantActionService`	Pending actions. They run only on the user's own confirm request.

Module	Folder	Class	Responsibility
Assistant	`Assistant/Toolbox`, `Assistant/Tools`	`AssistantToolbox`, `ToolOutcome`, `AssistantReadTools`, `AssistantActionTools`	Tools checked against the same permission policies as REST. Reads go through `ITaskService`.
Assistant	`Assistant/ModelGateway`	`ChatModelGateway`	Calls the selected chat-model provider
Assistant	`Assistant/Rules`	`AssistantRules`, `AssistantActionRules`, `SecretScreen`, `TaskPrioritiser`	Fixed limits, confirm-time policy per action kind, password/API-key screen, deterministic task ordering
Assistant	`Assistant/RateLimits`, `Assistant/AuditLog`, `Assistant/Access`, `Assistant/HelpKnowledge`	`AssistantRateLimiter`, `AssistantAuditLog`, `AssistantCaller`, `HelpKnowledgeBase`, `HelpTopicDto`	Per-user one-minute window, action audit, caller identity from token claims, embedded help topics

Controllers (`Controllers/<Module>`): Auth – `AuthController`, `PasswordRecoveryController`. Dashboard – `DashboardController`, `AnalyticsController`. Management – `EmployeesController`, `ProjectsController`, `PositionsController`, `DepartmentsController`. Monitoring – `TasksController`, `TaskCommentsController`, `TaskActivityController`, `WorkloadController`. Communication – `MailController`, `NotificationsController`. Reports – `ReportController`. Profile – `ProfileController`. Settings – `ApplicationSettingsController`, `ReminderSettingsController`, `RolesController`, `AiProvidersController`. Assistant – `AssistantController`. The full endpoint list is in §8.

6.4 Repositories and unit of work

Element	Design
`Repositories/Base/BaseRepository<TEntity>`	Shared `FindAsync`, `ExistsAsync`, `Add`, `Remove`. Module repositories derive from it and expose queries named for what they find. They never return an `IQueryable` to callers.
`Repositories/Base/UnitOfWork` (`IUnitOfWork`)	One `AppDbContext` per request, shared by every repository, so one `SaveChangesAsync` commits everything. Also provides `ExecuteInTransactionAsync` (joins an existing transaction) and `DetachAll`. Maps concurrency and unique-index failures (§6.1).

Element	Design
Module repositories	Auth: `AccountRepository`, `RefreshTokenRepository`, `MfaBackupCodeRepository`, `PasswordResetGrantRepository`. Dashboard: `DashboardRepository`, `AnalyticsRepository`. Management: `EmployeeRepository`, `UserAllocationRepository`, `ProjectRepository`, `PositionRepository`, `DepartmentRepository`. Monitoring: `TaskRepository`, `TaskCommentRepository`, `TaskActivityRepository`, `WorkloadRepository`. Communication: `MailRepository`, `MailAttachmentRepository`, `NotificationRepository`, `ReminderRepository`. Reports: `ReportRepository`. Settings: `ApplicationSettingsRepository`, `RoleRepository`, `PermissionRepository`, `AiProviderConfigRepository`, `AiModelRepository`. Assistant: `AssistantConversationRepository`, `AssistantMessageRepository`, `AssistantPendingActionRepository`, `AssistantAuditRepository`, `AssistantDirectoryRepository`.
Query/criteria records	For example `TaskListCriteria`, `TaskVisibility`, `ReportCriteria`, `AnalyticsTaskSet`, `DashboardDayBounds`, `ReminderCandidate`, `PendingUploads`. These are typed inputs and outputs that keep EF queries inside the repository layer.
Database	`UseSqlite` with the path from `DatabasePathResolver` under the data folder (`StorageRoot`, default `C:\TaskFlow`, override `Storage:Root`). See §7 and §18.

6.5 Integrations

Folder	Classes	Purpose
`Integrations/Security/Tokens`	`JwtTokenService` (`ITokenService`), `AccessToken`, `RefreshTokenValue`	Issue signed JWT access tokens and random refresh tokens (only the hash is stored, see `RefreshToken`)
`Integrations/Security/OneTimeCodes`	`Totp`, `Base32`	RFC 6238 TOTP codes and Base32 secrets for authenticator apps
`Integrations/Security/MfaSecrets`	`MfaSecretProtector` (`IMfaSecretProtector`)	Encrypts the authenticator secret (`User.MfaSecretProtected`)
`Integrations/Ai/Providers`	`OllamaProvider`, `GeminiProvider`, `OpenAiProvider` (`IChatModelProvider`), `AiProviderRuntime`, `ProviderHttp`, `ProviderFailure`, `ChatModelTypes`	Chat-model adapters, with failures classified as `ProviderFailureKind`
`Integrations/Ai/Registry`	`ChatModelProviderRegistry`	Resolves the provider for an `AiProviderKind`
`Integrations/Ai/KeyProtection`	`AiKeyProtector`, `KeyRingProtectionCheck`	ASP.NET Data Protection for AI API keys. The key ring is under `AiProviders:KeyRingPath` (DPAPI on Windows).
`Integrations/Storage`	`LocalMailFileStore` (`IMailFileStore`)	Attachment files under `Mail:AttachmentsPath` in the data folder, named by `MailAttachment.StorageKey`

6.6 Authorization model

Authorization is **permission-based**. Role names are not checked in controllers. The full model (catalog, role defaults, custom roles) is in §9 and Appendix C. This section covers how the backend enforces it.

Level	Mechanism
Session refresh (every request)	`SessionValidation.ValidateAsync` rejects the token when the user is gone or inactive, or when the `sstamp` claim differs from `User.SecurityStamp`. For a valid session it **replaces** the role claim and adds one `permission` claim per *effective* permission of the user's role, read from the database. A role or permission change therefore applies on the next request.
Effective permissions	`Common/Security/EffectivePermissions.Expand` adds (a) the finer keys that replaced a legacy broad key (`dashboard.admin`, `dashboard.manager`, `projects.manage`, `departments.manage`, `employees.manage`, `settings.ai.manage`) and (b) lower levels of a scope selector. Always-on keys (`dashboard.view.own`, `tasks.view.own`, `profile.view`, `profile.edit`, `profile.password.change`, `profile.mfa.manage`) are granted to every signed-in user and cannot be switched off.
Endpoint policies	One policy per catalog key, named `PermissionPolicy.NameFor(key)` = `perm:<key>`, each requiring claim `permission = <key>`. Controllers use `[RequirePermission(PermissionKeys.X)]` (an `AuthorizeAttribute` subclass) or `[Authorize(Policy = PermissionPolicy.Prefix + PermissionKeys.X)]`. Endpoints without a key use plain `[Authorize]`, and `/api/v1/auth/*` login/recovery endpoints are `[AllowAnonymous]`. TFA tools use the same policy names (`AssistantToolbox`).
Resource check	Policy `AuthPolicies.TaskAccess` → `TaskAccessRequirement` → `TaskAccessHandler`. Task-scoped actions in `TasksController`, `TaskCommentsController` and `TaskActivityController` call `IAuthorizationService.AuthorizeAsync(User, taskId, "TaskAccess")`. Access passes for a live task when the caller holds `tasks.view.all`, or is the assignee, the creator or an additional recipient (`TaskAccessScope.CanAccess`, mirrored in `TaskService` as defence in depth).
Data scope	**Own / Team / Organization.** Dashboards: `EmployeeDashboardScope` (Own, `dashboard.view.own`), `ManagerDashboardScope` (Team, `dashboard.view.team`, 403 outside the team), `AdminDashboardScope` (Organization, `dashboard.view.organization`). Tasks: `TaskService.VisibilityForCallerAsync` builds a `TaskVisibility`. Without `tasks.view.all` the caller sees only their own tasks (assigned, created or recipient). With `tasks.view.all` plus team dashboard but not organization dashboard (manager-scoped), the caller sees their team's tasks plus their own. With `tasks.view.all` otherwise, the caller sees all live tasks. Management lists, Reports and Workload are Organization-wide for any holder of the endpoint key.
Database guard	SQLite triggers (§7.4) make the three system roles (`Administrator` id 1, `Manager` id 2, `Employee` id 3, `SystemRoles`) and the `Permissions` catalog read-only, even for code that bypasses `RoleService`.
Audit	`DeniedActionAuditHandler` logs each 403 refusal of a signed-in user and then lets the default handler produce the response.

7. Data Model

The database is a single SQLite file (`taskflow.db` in the data folder, §18), created and upgraded by EF Core 10.0.12 migrations at startup. There are **31 entity types**, one `DbSet` each in `Persistence/AppDbContext.cs`. Table names equal the `DbSet` names. Entity classes are in `Models/Entities`, and enums are in `Models/Enums`.

Storage conventions (EF Core SQLite provider): `int` / `long` / `bool` → INTEGER, `string` → TEXT, `DateTime` → TEXT (UTC by convention, §7.5), `decimal` → TEXT, `Guid` → TEXT. Enums are stored as **INTEGER** (ordinal) unless the table below says *stored as text* (`HasConversion<string>()` with a max length). New enum values are therefore appended last (`TaskStatus.InQA`, `TaskActivityEventType.Deleted/CommentEdited/CommentDeleted`), so stored values keep their meaning.

7.1 Entity-relationship overview

Diagram 1 – identity, access and organisation

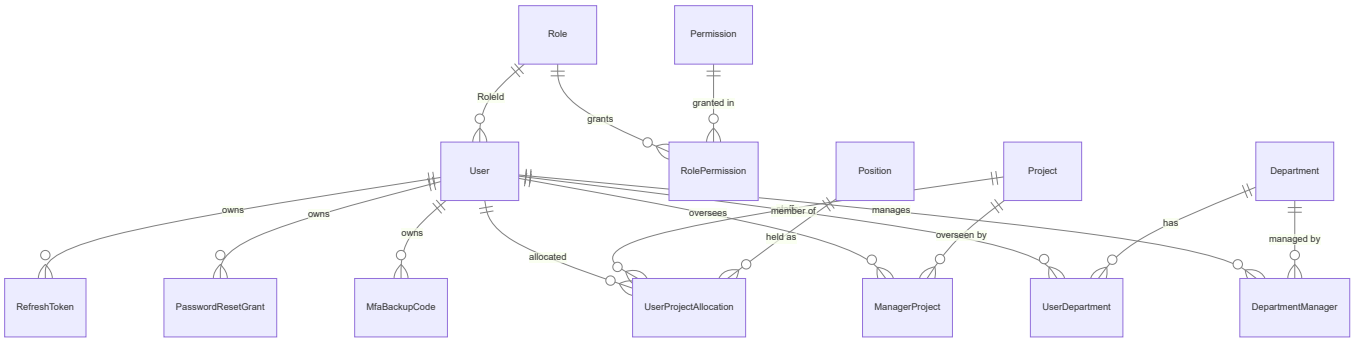
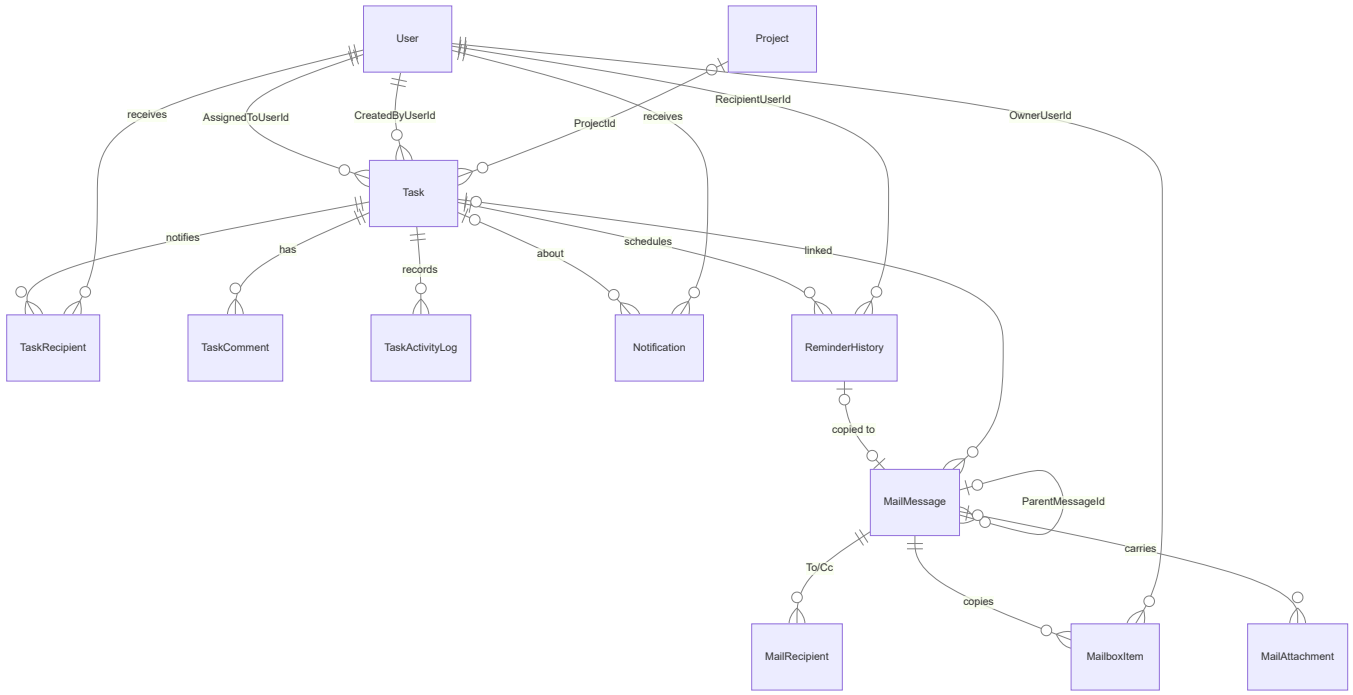
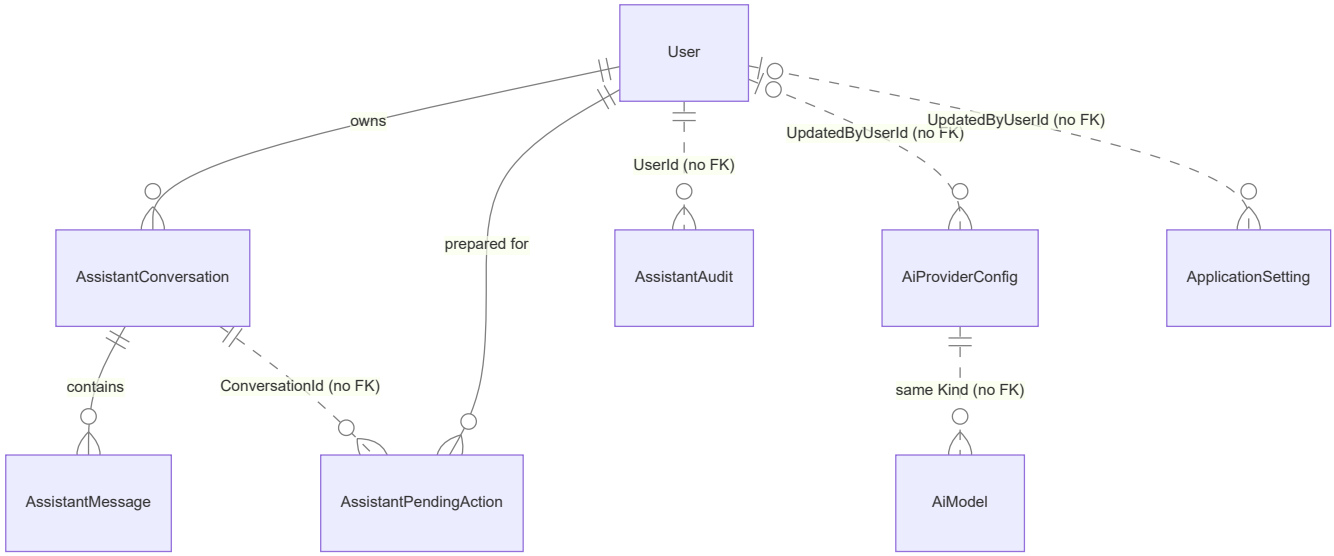


Diagram 2 – tasks, notifications, reminders and mail



`TaskComment.AuthorUserId`, `TaskActivityLog.ActorUserId`, `MailMessage.SenderUserId`, `MailRecipient.UserId` and `MailAttachment.UploadedByUserId` are also FKs to `User`. They are left out of the diagram for readability and listed in §7.2.

Diagram 3 – settings, AI providers and the TFA assistant



Dotted lines are logical links stored as plain columns, with no database foreign key.

7.2 Tables

Legend: **PK** primary key, **FK** foreign key (with delete behaviour), **UQ** unique index, **IX** non-unique index, **CT** concurrency token, *max n* = `HasMaxLength(n)`. Navigation collections are not columns and are not listed.

7.2.1 Identity and access

`User` (table `Users`) – soft-deleted, query filter `!IsDeleted`

Column	Type	Constraints / notes
Id	int	PK
EmployeeCode	string?	max 32
FullName	string	required, max 200
Username	string	required, max 100, **UQ**
Email	string	required, max 256, **UQ** (fixed `@taskflow.com` domain since migration `NormalizeEmailDomain`). Deleted users keep their address reserved.
MobileNumber	string?	max 20, optional E.164 (for example `+919876543210`). Not unique.
MfaSecretProtected	string?	max 512. Encrypted authenticator key, effective only once `MfaEnabledAt` is set.
MfaEnabledAt	DateTime?	when MFA was confirmed. Null = MFA off or setup unconfirmed.
MfaLastUsedStep	long	**CT**. Highest accepted TOTP time step (no code reuse).
MfaFailedAttempts	int	**CT**
MfaLockedUntil	DateTime?	MFA lock after repeated failures
PasswordHash	string	required
SecurityStamp	string	required, max 64. Changes when sign-in credentials change, which invalidates older tokens.
RoleId	int	FK → `Roles.Id`, **Restrict**

Column	Type	Constraints / notes
IsActive	bool	default `true`. Inactive users cannot sign in.
CreatedAt / UpdatedAt	DateTime	UTC
LastLoginAt	DateTime?	UTC
IsDeleted	bool	soft-delete flag

`RefreshToken` (table `RefreshTokens`) – query filter `!User.IsDeleted`

Column	Type	Constraints / notes
Id	int	PK
UserId	int	FK → `Users.Id`, **Cascade**
TokenHash	string	required, **IX** (only the hash is stored)
ExpiresAt / CreatedAt	DateTime	UTC
RevokedAt	DateTime?	set on logout or rotation

`PasswordResetGrant` (table `PasswordResetGrants`)

Column	Type	Constraints / notes
Id	int	PK
UserId	int	FK → `Users.Id`, **Cascade**. **IX** (UserId, UsedAt).
TokenHash	string	required, max 64, **IX**
CreatedAt / ExpiresAt	DateTime	UTC. Lifetime 10 minutes (`PasswordRecoveryService.TokenLifetime`).
UsedAt	DateTime?	**CT**. Set when spent, or when another reset made it pointless.

`MfaBackupCode` (table `MfaBackupCodes`)

Column	Type	Constraints / notes
Id	int	PK
UserId	int	FK → `Users.Id`, **Cascade**. **IX** (UserId, CodeHash).
CodeHash	string	required, max 64 (only a hash is stored)
CreatedAt	DateTime	UTC
UsedAt	DateTime?	**CT**. Single use.

`Role` (table `Roles`)

Column	Type	Constraints / notes
Id	int	PK. System roles have fixed ids 1 Administrator, 2 Manager, 3 Employee.
Name	string	required, max 100, collation `NOCASE`, **UQ** (case-insensitive)
IsSystem	bool	true for the three system roles. Read-only by trigger.
CreatedAt	DateTime	UTC

`Permission` (table `Permissions`) – read-only catalog

Column	Type	Constraints / notes
Id	int	PK
Key	string	required, max 100, **UQ** (for example `tasks.create`)
Name	string	required, max 200
Description	string	required, max 500
Group	string	required, max 100. Area shown on the Role Configuration screen.
IsAssignable	bool	default `true`. False for keys that cannot be granted from the Role Configuration screen (see Appendix C).

Rows are inserted from `PermissionCatalog` by `AccessControlSeeder`. The triggers `TR_Permissions_NoUpdate` and `TR_Permissions_NoDelete` reject updates and deletes.

`RolePermission` (table `RolePermissions`)

Column	Type	Constraints / notes
RoleId	int	**PK (RoleId, PermissionId)**, FK → `Roles.Id`, **Cascade**
PermissionId	int	FK → `Permissions.Id`, **Restrict**

Updates and deletes of rows that belong to a system role are rejected by triggers.

7.2.2 Organisation (projects, departments, positions)

`Project` (table `Projects`) – soft-deleted, query filter `!IsDeleted`

Column	Type	Constraints / notes
Id	int	PK
Name	string	required, max 200, **UQ filtered** `"IsDeleted" = 0` (a deleted project keeps its name)
Description	string?	max 500
CreatedAt	DateTime	UTC
IsDeleted	bool	soft-delete flag

`Department` (table `Departments`) – soft-deleted, query filter `!IsDeleted`

Column	Type	Constraints / notes
Id	int	PK
Name	string	required, max 200, **UQ filtered** `"IsDeleted" = 0`
Description	string?	max 500
CreatedAt	DateTime	UTC
IsDeleted	bool	soft-delete flag

`Position` (table `Positions`)

Column	Type	Constraints / notes
Id	int	PK
Name	string	required, max 100, collation `NOCASE`, **UQ** (case-insensitive)

Seed data (`HasData`): 1 Member, 2 Manager, 3 QA, 4 Developer, 5 DevOps, 6 System Admin. A position is a label only and grants no permissions.

`UserProjectAllocation` (table `UserProjectAllocations`) – query filter `!User.IsDeleted && !Project.IsDeleted`

Column	Type	Constraints / notes
UserId	int	**PK (UserId, ProjectId)**, FK → `Users.Id`, **Cascade**
ProjectId	int	FK → `Projects.Id`, **Cascade**, **IX**
AllocationPercent	int	check constraint `CK_UserProjectAllocations_AllocationPercent`: `BETWEEN 1 AND 100`. A user's total across projects is at most 100 (service rule).
IsProjectManager	bool	marks the project's manager(s), kept in step with `ManagerProject`
PositionId	int	FK → `Positions.Id`, **Restrict**, **IX**

`ManagerProject` (table `ManagerProjects`)

Column	Type	Constraints / notes
ManagerUserId	int	**PK (ManagerUserId, ProjectId)**, FK → `Users.Id`, **Cascade**
ProjectId	int	FK → `Projects.Id`, **Cascade**, **IX**

This is recorded data only. It does not narrow what the manager can see.

`DepartmentManager` (table `DepartmentManagers`)

Column	Type	Constraints / notes
ManagerUserId	int	**PK (ManagerUserId, DepartmentId)**, FK → `Users.Id`, **Cascade**
DepartmentId	int	FK → `Departments.Id`, **Cascade**, **IX**

`UserDepartment` (table `UserDepartments`) – query filter `!User.IsDeleted && !Department.IsDeleted`

Column	Type	Constraints / notes
UserId	int	**PK (UserId, DepartmentId)**, FK → `Users.Id`, **Cascade**
DepartmentId	int	FK → `Departments.Id`, **Cascade**, **IX**

A user may belong to several departments.

7.2.3 Tasks

`Task` (table `Tasks`) – soft-deleted, **no global query filter** (repositories filter `!IsDeleted` explicitly)

Column	Type	Constraints / notes
Id	int	PK

Column	Type	Constraints / notes
TaskCode	string	required, max 32, **UQ**
Title	string	required, max 200
Description	string?	max 4000
AssignedToUserId	int	FK → `Users.Id`, **Restrict**, **IX**
CreatedByUserId	int	FK → `Users.Id`, **Restrict**
ProjectId	int?	FK → `Projects.Id`, **Restrict** (optional project)
Priority	`TaskPriority`	INTEGER
StartDate	DateTime	UTC
Deadline	DateTime	UTC, **IX**
EstimatedHours	decimal	
Status	`TaskStatus`	INTEGER (InQA = 5)
FirstReminderDays	int	default 2
SecondReminderDays	int	default 1
OverdueReminderFrequency	`ReminderFrequency`	INTEGER, default `Daily`
CreatedAt / UpdatedAt	DateTime	UTC
CompletedAt / CancelledAt	DateTime?	UTC
ActualHours	decimal?	**CT**. Actual worked hours. Changes are recorded in the activity log.
IsDeleted	bool	soft-delete flag

Overdue and display stage are **never stored** (§7.5).

`TaskRecipient` (table `TaskRecipients`)

Column	Type	Constraints / notes
Id	int	PK
TaskId	int	FK → `Tasks.Id`, **Cascade**. **UQ (TaskId, UserId)**.
UserId	int	FK → `Users.Id`, **Cascade**

`TaskComment` (table `TaskComments`) – soft-deleted (explicit filter in `TaskCommentRepository`)

Column	Type	Constraints / notes
Id	int	PK
TaskId	int	FK → `Tasks.Id`, **Cascade**. **IX (TaskId, CreatedAt)**.
AuthorUserId	int	FK → `Users.Id`, **Restrict**
Body	string	required
CreatedAt	DateTime	UTC
EditedAt	DateTime?	set when the author edits
IsDeleted	bool	database default `false`. Set by the author's delete.

`TaskActivityLog` (table `TaskActivityLogs`) – append-only

Column	Type	Constraints / notes
Id	int	PK
TaskId	int	FK → `Tasks.Id`, **Cascade**. **IX (TaskId, CreatedAt)**.
EventType	`TaskActivityEventType`	**stored as text**, max 32
ActorUserId	int	FK → `Users.Id`, **Restrict**
OldValue / NewValue	string?	for example ISO-8601 deadlines or hours
Message	string	required
CreatedAt	DateTime	UTC

Only worked-hours entries may be corrected or removed, and only by an administrator, through `/api/v1/tasks/{id}/activity/{entryId}/hours`.

7.2.4 Notifications and reminders

`Notification` (table `Notifications`)

Column	Type	Constraints / notes
Id	int	PK
UserId	int	FK → `Users.Id`, **Cascade**. **IX (UserId, IsRead)**.
TaskId	int?	FK → `Tasks.Id`, **Cascade**
Type	`NotificationType`	**stored as text**, max 32
Message	string	required
IsRead	bool	
CreatedAt	DateTime	UTC

`ReminderHistory` (table `ReminderHistory`)

Column	Type	Constraints / notes
Id	int	PK
TaskId	int	FK → `Tasks.Id`, **Restrict**
RecipientUserId	int	FK → `Users.Id`, **Restrict**
ReminderType	`ReminderType`	**stored as text**, max 32
ScheduledFor	DateTime	UTC instant of local midnight (application timezone) of the reminder's day
Status	`ReminderStatus`	**stored as text**, max 32. The engine writes `Sent`.
ProcessedAt	DateTime?	UTC
DeliveryAttempts	int	
—	—	**UQ (TaskId, RecipientUserId, ReminderType, ScheduledFor)**: duplicate prevention (BRD §3.5, see §11)

7.2.5 Internal mail

`MailMessage` (table `MailMessages`)

Column	Type	Constraints / notes
Id	int	PK
ConversationId	Guid	**IX**. Shared by a mail and all its replies and forwards.
ParentMessageId	int?	FK → `MailMessages.Id` (self), **Restrict**
SenderUserId	int?	FK → `Users.Id`, **Restrict**. Null for the TaskFlow system sender.
IsSystem	bool	true for system mails (reminder copies)
Subject	string	required, max 200
Body	string	required, max 20000
TaskId	int?	FK → `Tasks.Id`, **Restrict**
ReminderHistoryId	int?	FK → `ReminderHistory.Id`, **Restrict**. **UQ filtered** `"ReminderHistoryId" IS NOT NULL` (one mail per reminder event).
DraftTo / DraftCc	string?	max 4000 each. Raw addresses of a draft, resolved when sent.
CreatedAt / UpdatedAt	DateTime	UTC
SentAt	DateTime?	null = the author's draft

`MailRecipient` (table `MailRecipients`)

Column	Type	Constraints / notes
Id	int	PK
MessageId	int	FK → `MailMessages.Id`, **Cascade**. **UQ (MessageId, UserId)**.
UserId	int	FK → `Users.Id`, **Restrict**
Kind	`MailRecipientKind`	**stored as text**, max 8

`MailboxItem` (table `MailboxItems`) – one user's copy of a mail

Column	Type	Constraints / notes
Id	int	PK
MessageId	int	FK → `MailMessages.Id`, **Cascade**. **UQ (MessageId, OwnerUserId, Folder)**.
OwnerUserId	int	FK → `Users.Id`, **Restrict**. **IX (OwnerUserId, Folder, PurgedAt, TrashedAt, SortAt)**.
Folder	`MailFolder`	**stored as text**, max 16
IsRead / IsStarred / IsImportant	bool	personal flags. Starred and Important are views, not folders.
TrashedAt	DateTime?	in Trash view when set
PurgedAt	DateTime?	"delete forever": the row is kept and hidden
SortAt	DateTime	sent time, or last draft save

`MailAttachment` (table `MailAttachments`)

Column	Type	Constraints / notes
Id	int	PK
MessageId	int?	FK → `MailMessages.Id`, **Cascade**, **IX**. Null = pending upload visible only to the uploader.
UploadedByUserId	int	FK → `Users.Id`, **Restrict**
FileName	string	required, max 255
ContentType	string	required, max 128 (from the extension allowlist)
SizeBytes	long	
StorageKey	Guid	name of the stored file under `mail-attachments`. Forwarded copies share it.
CreatedAt	DateTime	UTC

7.2.6 Settings, AI providers and assistant

`ApplicationSetting` (table `ApplicationSettings`) – non-secret key/value settings

Column	Type	Constraints / notes
Id	int	PK
Key	string	required, max 128, **UQ**
Value	string	required, max 1024. Invariant-culture string.
Category	string	required, max 64 (`Application`, `Reminder`, `Notification`)
UpdatedByUserId	int?	no FK
UpdatedAt	DateTime	UTC

Rows inserted by `SettingsInitializer` when missing (`Common/Constants/SettingKeys.cs`). Existing values are never overwritten.

Key	Category	Default
`Application.Name`	Application	`TaskFlow`
`Application.Timezone`	Application	config `Application:Timezone` (`Asia/Kolkata`), or `UTC` when empty
`Application.DefaultPriority`	Application	`Medium`
`Reminder.Enabled`	Reminder	`true`
`Reminder.DeadlineDayEnabled`	Reminder	`true`
`Reminder.DefaultFirstDays`	Reminder	`2`
`Reminder.DefaultSecondDays`	Reminder	`1`
`Reminder.DefaultOverdueFrequency`	Reminder	`Daily`
`Notification.MailEnabled`	Notification	`true`
`Notification.InAppEnabled`	Notification	`true`

`AiProviderConfig` (table `AiProviderConfigs`)

Column	Type	Constraints / notes
Id	int	PK
Kind	`AiProviderKind`	**stored as text**, max 16, **UQ** (one row per provider)
IsEnabled	bool	seeded: Ollama on, Gemini and OpenAI off
ModelName	string	required, max 100
BaseUrl	string?	max 300. Used by Ollama only.
EncryptedApiKey	string?	protected with ASP.NET Data Protection. Never returned to a client.
KeyLast4	string?	max 4, for the "key set" display
UpdatedByUserId	int?	no FK
UpdatedAt	DateTime	UTC

`AiModel` (table `AiModels`)

Column	Type	Constraints / notes
Id	int	PK
Kind	`AiProviderKind`	**stored as text**, max 16. **UQ (Kind, Name)**.
Name	string	required, max 100
IsBuiltIn	bool	true = shipped in code (seeded by `AiProviderSeeder`), false = added by an administrator

`AssistantConversation` (table `AssistantConversations`)

Column	Type	Constraints / notes
Id	int	PK
UserId	int	FK → `Users.Id`, **Cascade**, **IX**. The user's latest conversation is the active one.
CreatedAt / UpdatedAt	DateTime	UTC

`AssistantMessage` (table `AssistantMessages`)

Column	Type	Constraints / notes
Id	int	PK
ConversationId	int	FK → `AssistantConversations.Id`, **Cascade**, **IX**
Role	`AssistantMessageRole`	**stored as text**, max 16
Content	string	required (provider-neutral form)
MetaJson	string?	render metadata (kind, links, ranked tasks, offered pending action)
ProviderKind	`AiProviderKind?`	**stored as text**, max 16. Provider that produced an assistant message.
CreatedAt	DateTime	UTC

`AssistantPendingAction` (table `AssistantPendingActions`)

Column	Type	Constraints / notes
Id	Guid	PK (the `{id}` in `/api/v1/assistant/actions/{id}`)
UserId	int	FK → `Users.Id`, **Cascade**, **IX**

Column	Type	Constraints / notes
ConversationId	int	no FK
Kind	`AssistantActionKind`	**stored as text**, max 32
PayloadJson	string	required, default `{}`. The exact details shown to the user, executed on confirm.
Status	`AssistantActionStatus`	**stored as text**, max 16
CreatedAt / ExpiresAt	DateTime	UTC
ResolvedAt	DateTime?	UTC

`AssistantAudit` (table `AssistantAudits`)

Column	Type	Constraints / notes
Id	int	PK
UserId	int	no FK. **IX (UserId, At)**.
Kind	`AssistantActionKind`	**stored as text**, max 32
Outcome	string	required, max 32 (prepared, confirmed, cancelled, expired, refused, …)
At	DateTime	UTC

Audit rows hold no message bodies, passwords or keys.

7.3 Enumerations (`Models/Enums`)

Enum	Values (stored ordinal in brackets where stored as INTEGER)	Used by
`TaskStatus`	ToDo [0], InProgress [1], Blocked [2], Completed [3], Cancelled [4], InQA [5]	`Task.Status`. InQA was appended last. Completed and Cancelled are terminal (Appendix A).
`TaskPriority`	Low [0], Medium [1], High [2], Critical [3]	`Task.Priority`
`ReminderFrequency`	Daily [0], Weekly [1]	`Task.OverdueReminderFrequency`
`ReminderType`	FirstReminder, SecondReminder, DueToday, Overdue	`ReminderHistory.ReminderType` (text)
`ReminderStatus`	Pending, Sent, Failed, Skipped	`ReminderHistory.Status` (text). The engine writes only Sent.
`NotificationType`	TaskAssigned, Reassigned, DeadlineReminder, Overdue, System, TaskComment	`Notification.Type` (text)
`TaskActivityEventType`	Created, Assigned, Reassigned, StatusChanged, DeadlineChanged, PriorityChanged, Completed, CommentAdded, Administrative, HoursLogged, TimeLogged, Deleted, CommentEdited, CommentDeleted	`TaskActivityLog.EventType` (text)
`MailFolder`	Inbox, Sent, Drafts	`MailboxItem.Folder` (text). Starred, Important and Trash are views.

Enum	Values (stored ordinal in brackets where stored as INTEGER)	Used by
`MailRecipientKind`	To, Cc	`MailRecipient.Kind` (text)
`AiProviderKind`	Ollama, Gemini, OpenAI	`AiProviderConfig.Kind`, `AiModel.Kind`, `AssistantMessage.ProviderKind` (text)
`AssistantMessageRole`	User, Assistant, Tool	`AssistantMessage.Role` (text)
`AssistantActionKind`	SendMail, UpdateOwnProfile, ChangeUserRole	`AssistantPendingAction.Kind`, `AssistantAudit.Kind` (text)
`AssistantActionStatus`	Pending, Confirmed, Cancelled, Expired, Refused, Failed	`AssistantPendingAction.Status` (text)
`ProviderFailureKind`	RateLimited, InvalidKey, Timeout, Unreachable, ModelUnavailable, Unexpected	Not persisted. Classifies AI provider errors (`ProviderFailure`, `ProviderFailureMessages`).

Derived, never-stored classifications in `Common/Rules`: `TaskStage` (Upcoming, DueToday, Overdue, Completed), `DeadlineBucket` (Overdue, DueToday, Next7Days, Later), `TaskBucket` (Pending, InProgress, Completed, Overdue). See §7.5.

7.4 Indexes, constraints and triggers

Kind	Definition
Unique	`Users(Username)`, `Users(Email)`, `Roles(Name)` NOCASE, `Permissions(Key)`, `Positions(Name)` NOCASE, `Projects(Name)` where `IsDeleted = 0`, `Departments(Name)` where `IsDeleted = 0`, `Tasks(TaskCode)`, `TaskRecipients(TaskId, UserId)`, `ReminderHistory(TaskId, RecipientUserId, ReminderType, ScheduledFor)`, `ApplicationSettings(Key)`, `MailMessages(ReminderHistoryId)` where not null, `MailRecipients(MessageId, UserId)`, `MailboxItems(MessageId, OwnerUserId, Folder)`, `AiProviderConfigs(Kind)`, `AiModels(Kind, Name)`
Composite primary keys	`RolePermissions(RoleId, PermissionId)`, `UserProjectAllocations(UserId, ProjectId)`, `ManagerProjects(ManagerUserId, ProjectId)`, `DepartmentManagers(ManagerUserId, DepartmentId)`, `UserDepartments(UserId, DepartmentId)`
Query indexes	`Tasks(Deadline)`, `Tasks(AssignedToUserId)`, `TaskComments(TaskId, CreatedAt)`, `TaskActivityLogs(TaskId, CreatedAt)`, `Notifications(UserId, IsRead)`, `RefreshTokens(TokenHash)`, `PasswordResetGrants(TokenHash)`, `PasswordResetGrants(UserId, UsedAt)`, `MfaBackupCodes(UserId, CodeHash)`, `MailMessages(ConversationId)`, `MailboxItems(OwnerUserId, Folder, PurgedAt, TrashedAt, SortAt)`, `MailAttachments(MessageId)`, `AssistantConversations(UserId)`, `AssistantMessages(ConversationId)`, `AssistantPendingActions(UserId)`, `AssistantAudits(UserId, At)`, plus the FK indexes on the link tables

Kind	Definition
Check constraint	`CK_UserProjectAllocations_AllocationPercent`: `"AllocationPercent" BETWEEN 1 AND 100`
Delete behaviour	**Restrict** on history-bearing references: `Users.RoleId`, `Tasks.AssignedToUserId/CreatedByUserId/ProjectId`, `TaskComments.AuthorUserId`, `TaskActivityLogs.ActorUserId`, `ReminderHistory.TaskId/RecipientUserId`, every `MailMessages` FK, `MailRecipients.UserId`, `MailboxItems.OwnerUserId`, `MailAttachments.UploadedByUserId`, `RolePermissions.PermissionId`, `UserProjectAllocations.PositionId`. **Cascade** on owned children: tokens, grants, backup codes, link tables, `TaskRecipients`, `TaskComments`, `TaskActivityLogs`, `Notifications`, `MailRecipients`, `MailboxItems`, `MailAttachments`, assistant conversations, messages and pending actions. In practice parents are soft-deleted (§7.6), so cascades seldom fire (BR-18).
Triggers (migration `AddRolesAndPermissions`)	`TR_Roles_SystemNoUpdate`, `TR_Roles_SystemNoDelete` (rows with `IsSystem = 1`), `TR_Permissions_NoUpdate`, `TR_Permissions_NoDelete` (whole catalog), `TR_RolePermissions_SystemNoUpdate`, `TR_RolePermissions_SystemNoDelete` (grants of a system role). Each raises `ABORT`. Inserts stay allowed, so the seeder can add new keys and grant them to the system roles.

7.5 Date, time, timezone and overdue derivation

Rule	Implementation
Storage	All timestamps are written as **UTC** (`TimeProvider` / `IAppClock.UtcNow`). Request dates with `Kind` Local are converted to UTC. `Unspecified` is treated as UTC (for example `ReportService`, `TaskActivityController`).
Application timezone	`AppClock.GetTimeZoneAsync` reads the `Application.Timezone` setting row, then config `Application:Timezone` (`Asia/Kolkata`), then **UTC** if the id is unknown (logged, never fatal).
Local day	`ReminderCalculator.LocalDate(utc, tz)` and `ReminderCalculator.LocalMidnightUtc(day, tz)`. DST-safe: when midnight falls in a gap, the earliest valid local time is used. When midnight occurs twice, the earlier instant is used.
Overdue (instant rule, BR-9)	`Deadline < nowUtc AND Status NOT IN (Completed, Cancelled)` and not deleted. Used by `DeadlineBucketRules` (`DeadlineBucket.Overdue`), `TaskBucketRules.BucketOf` (dashboard buckets, where Cancelled is not counted), the board card `IsOverdue` flag (`TaskService.ToCard`) and `ReportRepository`.
Display stage (day rule)	`TaskStageCalculator.Calculate`: Completed/Cancelled → `Completed`. Otherwise the deadline's local day is compared with today in the application timezone: earlier → `Overdue`, same day → `DueToday`, later → `Upcoming`. This matches the reminder engine's day logic.
Deadline buckets	`DeadlineBucketRules.BoundsFor(nowUtc, tz)`: DueToday = `[now, local midnight tomorrow)`, Next7Days = `[tomorrow, tomorrow + 7 days)`, Later = after that.
Reminder slot	`ReminderHistory.ScheduledFor` = UTC instant of local midnight of the reminder's day. Only reminders whose day is *today* are generated (no backfill, see §11 and Appendix B).
Never stored	Overdue state, stage and buckets are computed at read time, so they cannot go stale.

> Note (known gap): the two overdue rules can disagree on the deadline day itself. A task whose deadline instant has passed earlier today is *Overdue* by the instant rule (board flag, dashboard buckets, reports), but its stage is

> *DueToday* by the day rule (task list and details).

7.6 Soft delete and query filters

Entity	Soft delete	How it is filtered
`User`	`IsDeleted`	Global query filter `!IsDeleted`. `EmployeeRepository` uses `IgnoreQueryFilters` where deleted users still count (for example the email reservation).
`Project`, `Department`	`IsDeleted`	Global query filter `!IsDeleted`. Unique name indexes cover live rows only.
`UserProjectAllocation`, `UserDepartment`	–	Global filter hides rows whose user, project or department is soft-deleted
`RefreshToken`	–	Global filter `!User.IsDeleted`
`Task`	`IsDeleted`	**No global filter.** Every repository query adds `!task.IsDeleted` (`TaskRepository`, `DashboardRepository`, `WorkloadRepository`, `ReminderRepository`, `ReportRepository`, `ProjectRepository`, `DeadlineBucketRules`, `TaskBucketRules`). Notifications that point to a deleted task are returned without the task link.
`TaskComment`	`IsDeleted` (+ `EditedAt`)	No global filter. `TaskCommentRepository` filters `!IsDeleted`.
`MailboxItem`	`TrashedAt` / `PurgedAt`	Per-owner state. Trash is a view. "Delete forever" sets `PurgedAt` and keeps the row.

Deactivation (`User.IsActive = false`) is separate from deletion. It blocks sign-in and invalidates live sessions on the next request (`SessionValidation`). Records referenced by history (users, projects, departments, tasks) are soft-deleted to keep audit and activity data intact (BR-18).

7.7 Migrations

Migrations are in `Persistence/Migrations` and are applied automatically at startup (`Database.MigrateAsync`). `AppDbContextFactory` supports design-time tooling.

#	Migration	Main change
1	`20261011171611_InitialAuth`	`Users`, `RefreshTokens`
2	`20261012104823_AddDepartmentsAndUserDepartment`	`Departments` and `Users.DepartmentId` (later reshaped, see 15 and 18)
3	`20261012124843_AddTasks`	`Tasks`, `TaskRecipients`
4	`20261012173144_AddUserSecurityStamp`	`Users.SecurityStamp`

#	Migration	Main change
5	`20261013075057_AddTaskTracking`	`TaskComments`, `TaskActivityLogs`, `Notifications`
6	`20261013104629_AddTaskReminderSettings`	`Tasks.FirstReminderDays`, `SecondReminderDays`, `OverdueReminderFrequency`. Removes the duplicate `AssignedToId` / `CreatedById` columns in favour of `AssignedToUserId` / `CreatedByUserId`.
7	`20261017062735_AddReminderEngine`	`ApplicationSettings`, `ReminderHistory` with the dedup index
8	`20261017074817_AddEmailDelivery`	`EmailDeliveryLogs` (SMTP, superseded)
9	`20261017090431_AddInternalMail`	Drops `EmailDeliveryLogs` and the SMTP settings. Adds `MailMessages`, `MailRecipients`, `MailboxItems`, `MailAttachments`.
10	`20261018183210_AddAssistant`	`AiProviderConfigs`, `AssistantConversations`, `AssistantMessages`, `AssistantPendingActions`, `AssistantAudits`
11	`20261019133820_AddRolesAndPermissions`	`Roles`, `Permissions`, `RolePermissions`. Replaces the `Users.Role` column with `Users.RoleId`. Adds the read-only triggers.
12	`20261019163241_AddAiModels`	`AiModels`
13	`20261019173718_NormalizeEmailDomain`	Data migration: every user is moved to the fixed `@taskflow.com` domain (local parts tidied and de-duplicated, deleted users included). Idempotent.
14	`20261019174259_AddManagerAssignments`	`ManagerDepartments` link table and `Users.ManagerId`
15	`20261020173455_RenameDepartmentsToProjectsAndAddDepartments`	Renames `Departments` → `Projects` and `ManagerDepartments` → `ManagerProjects`, and renames the `DepartmentId` columns to `ProjectId`. Adds new `Departments` and `DepartmentManagers`. Refreshes the `departments.*` permission wording around the trigger.
16	`20261021070943_FixDepartmentFollowUps`	Grants `projects.*` keys to roles that held the old `departments.*` keys, so nobody loses project access. Adds indexes.
17	`20261023101938_AddMfaAndAccountRecovery`	`MfaBackupCodes`, `PasswordResetGrants`. Adds `Users.MfaSecretProtected`, `MfaEnabledAt`, `MfaLastUsedStep`, `MfaFailedAttempts`, `MfaLockedUntil`, `MobileNumber`.
18	`20261023160825_UserProjectAllocationsAndDepartments`	`Positions` (seeded), `UserProjectAllocations`, `UserDepartments`. Each old single project becomes a 100 % `Member` allocation and each old department a membership. Drops `Users.ProjectId`, `DepartmentId` and `ManagerId`.
19	`20261023173211_AddProjectManagerFlag`	`UserProjectAllocations.IsProjectManager`
20	`20261024074305_AddTaskActualHours`	`Tasks.ActualHours`
21	`20261024122725_AddTaskCommentEditDelete`	`TaskComments.EditedAt`, `TaskComments.IsDeleted`

8. API Design

The API is one ASP.NET Core project (`TaskFlow.Api`) with 21 controllers in module sub-folders of `Controllers/` . It exposes **114 HTTP actions over 83 distinct routes**, all under `/api/v1` . Every action is listed below exactly once, grouped by module in the order of the module docs. The Legal module (module 10) has no API endpoints. Its pages are client routes only (see §12).

8.1 Conventions

Topic	Convention (as implemented)
Base path / versioning	Literal route prefix `api/v1/...` on every controller (`[Route("api/v1/...")]`). No version negotiation package.
Format	JSON in and out. ASP.NET Core web defaults give `camelCase` property names. Enums travel as their **names** (`JsonStringEnumConverter` in `Config/CrossCuttingRegistration.cs`), e.g. `"priority": "High"` , `"status": "InQA"` .
Authentication	`Authorization: Bearer <access-token>` (JWT, issuer `Jwt:Issuer`). On every request `SessionValidation.ValidateAsync` re-checks the session and refreshes the permission claims from the database. The refresh token is an HttpOnly, Secure, `SameSite=Strict` cookie named `taskflow_refresh` , scoped to the path `/api/v1/auth` (`Common/Security/RefreshTokenCookie.cs`).
MFA gate	`MfaSetupGateMiddleware` answers every `/api` call with 403 and the code `mfa_setup_required` until the signed-in user has set up an authenticator. The only exceptions are `/api/v1/auth/*` , `/api/v1/users/me` and `/api/v1/users/me/mfa/*` (see §9).
Permission policies	One policy per catalog key, named `perm:` + key (`PermissionPolicy.Prefix` = `"perm:"`). The policy requires the `permission` claim. Two attribute forms are used: `[RequirePermission(PermissionKeys.X)]` and `[Authorize(Policy = PermissionPolicy.Prefix + PermissionKeys.X)]` . Both resolve to the same policy. The tables below show the resolved key string. When several keys are listed with **+**, all of them are required: class-level and action-level attributes add up.
Resource policy	`TaskAccess` (`Authorization/TaskAccess/TaskAccessHandler.cs`). The caller passes if they are the assignee, the creator or an additional recipient of the task, or hold `tasks.view.all` . Controllers call it imperatively with the task id and return 403 on failure.
In-action checks	Some endpoints check claims or the role in code after the policy passes, for example `employees.role.assign` , `roles.administrator.assign` , `roles.modify.access` , `roles.view.all` , or role = `Administrator` . These checks are listed in the Permission column as "in code".
Caller identity	The user id always comes from the token's `NameIdentifier` claim. "Own" endpoints (`/users/me` , mail, notifications, assistant, employee dashboard) never take a user id from the request.
Validation	`ValidationFilter` runs the FluentValidation validator registered for each bound argument before the action runs. A failure returns **400** `ValidationProblemDetails` with an `errors` map keyed by field.
Errors	`application/problem+json` throughout. `GlobalExceptionHandler` maps `BusinessRuleException` / `RequestValidationException` → 400, `ForbiddenException` → 403, `NotFoundException` → 404, `ConflictException` and `DbUpdateConcurrencyException` → 409, `RateLimitedException` → 429 (with `Retry-After` when a delay is given). Anything else → 500 with a generic detail and a trace id. Policy failures return 401/403, and `DeniedActionAuditHandler` records them in the audit trail.
Status codes	Reads return 200. Creates that use `CreatedAtAction` return **201** (departments, users, projects, tasks, roles). Commands with no body return **204** (logout, delete, mark-read, password reset/change, etc.). A missing or out-of-scope resource returns 404.

Topic	Convention (as implemented)
Paging	Paged lists return `PagedResult<T>` = `{ items, total, page, pageSize }`. The paged lists are the task list (`page`, `pageSize`, default 20), notifications (default 20) and mail folders (default 25, max 50).
CORS	Policy `SpaClient`. Origins come from `Cors:AllowedOrigins`, with any header, any method and credentials allowed (needed for the refresh cookie).
OpenAPI	`AddOpenApi()`. The document is mapped (`MapOpenApi()`) only in the Development environment. A development-only diagnostics endpoint is also mapped in `Program.cs`. It is not a controller action and is not counted here.
Route notation	Route constraints are left out below (`{id:int}` is written `{id}`, `{id:guid}` is written `{id}`).

8.2 Authentication

`AuthController` (`/api/v1/auth`) and `PasswordRecoveryController` (`/api/v1/auth/password-recovery`), in `Controllers/Auth/`.

Method	Route	Permission / policy	Purpose
POST	`/api/v1/auth/login`	Anonymous	Checks email-or-username and password. With MFA on it returns `{ mfaRequired: true, challengeToken }` and starts no session. Otherwise it returns the access token and user and sets the refresh cookie. 401 on failure.
POST	`/api/v1/auth/mfa/verify`	Anonymous	Second sign-in step. Takes the challenge token plus an authenticator or backup code and issues the session, like a normal sign-in. 400 for a wrong code, 429 when locked out.
POST	`/api/v1/auth/refresh`	Anonymous (refresh cookie)	Rotates the refresh cookie and returns a new access token. On failure it clears the cookie and returns 401.
POST	`/api/v1/auth/logout`	Anonymous (refresh cookie)	Revokes the refresh token, clears the cookie and returns 204.
GET	`/api/v1/auth/me`	Authenticated	Current user: id, name, username, email, role, employee code, effective permission keys, `mfaSetupRequired`.
POST	`/api/v1/auth/password-recovery/authenticator/verify`	Anonymous	Recovery step 1. Takes the account identifier and a current authenticator code and returns a single-use reset token. Every failure gets one generic 400 message, and there is a 429 throttle per client address.
POST	`/api/v1/auth/password-recovery/reset`	Anonymous	Recovery step 2. Sets the new password with the reset token and returns 204. An invalid or expired token returns 400.

Example requests

```
// POST /api/v1/auth/login            (LoginRequest)
{ "emailOrUsername": "admin@taskflow.com", "password": "<secret>" }

// POST /api/v1/auth/mfa/verify       (MfaVerifyRequest)
{ "challengeToken": "<challenge-token>", "code": "123456" }

// POST /api/v1/auth/password-recovery/authenticator/verify   (AuthenticatorRecoveryRequest)
```

```
{ "identifier": "jdoe", "code": "123456" }

// POST /api/v1/auth/password-recovery/reset                    (RecoverPasswordRequest)
{ "resetToken": "<reset-token>", "newPassword": "<secret>", "confirmPassword": "<secret>" }
```

8.3 Dashboard

`DashboardController` and `AnalyticsController` (`/api/v1/dashboard`), in `Controllers/Dashboard/`. `dashboard.view.own` is always on, so the employee endpoints only need a signed-in user (plus `dashboard.analytics.view` for analytics).

Method	Route	Permission / policy	Purpose
GET	`/api/v1/dashboard/admin`	`dashboard.view.organization`	Organization-wide KPI cards (`AdminDashboardDto`).
GET	`/api/v1/dashboard/manager`	`dashboard.view.team`	KPI cards for the caller's team (`ManagerDashboardDto`).
GET	`/api/v1/dashboard/employee`	Authenticated	KPI cards for the caller's own tasks (`EmployeeDashboardDto`).
GET	`/api/v1/dashboard/admin/analytics`	`dashboard.analytics.view` + `dashboard.view.organization`	Organization analytics for an `AnalyticsFilter` (query: `from`, `to`, `basis` = `Created` / `Deadline` / `Completed`, `departmentId`, `projectId`, `employeeId`, `managerId`, `allEmployees`).
GET	`/api/v1/dashboard/admin/analytics/filters`	`dashboard.filters.use` + `dashboard.view.organization`	Filter options (departments, projects, employees, managers) for the admin analytics.
GET	`/api/v1/dashboard/manager/analytics`	`dashboard.analytics.view` + `dashboard.view.team`	Team analytics for an `AnalyticsFilter`, limited to the calling manager's team.
GET	`/api/v1/dashboard/manager/analytics/filters`	`dashboard.filters.use` + `dashboard.view.team`	Filter options within the manager's team.
GET	`/api/v1/dashboard/employee/analytics`	`dashboard.analytics.view`	Analytics of the caller's own tasks. The only input is `period`, and any other query key is ignored.

8.4 Management

`DepartmentsController`, `EmployeesController` (`/api/v1/users`), `PositionsController` and `ProjectsController`, in `Controllers/Management/`.

Departments

Method	Route	Permission / policy	Purpose
GET	`/api/v1/departments`	`departments.view`	All departments with member counts (`DepartmentWithCountsDto`).
GET	`/api/v1/departments/{id}`	`departments.view`	One department. 404 if it does not exist.
GET	`/api/v1/departments/{id}/members`	`departments.view`	The department's managers and employees (`DepartmentMembersDto`).
POST	`/api/v1/departments`	`departments.create`	Creates a department and returns 201.
PUT	`/api/v1/departments/{id}`	`departments.edit`	Renames the department or changes its description.
PUT	`/api/v1/departments/{id}/members`	`departments.members.manage`	Replaces the department's managers and employees.
DELETE	`/api/v1/departments/{id}`	`departments.delete`	Deletes the department and returns 204 (the service applies the safe-delete rules, see §10).

Employees

Method	Route	Permission / policy	Purpose
GET	`/api/v1/users`	`employees.view`	Employee list. Query: `search`, `projectId`, `role`, `isActive`, `departmentId`.
GET	`/api/v1/users/roles`	Authenticated; in code: `employees.role.assign` or `roles.manage`	Role options for the employee form and role filter. The Administrator role is listed only for holders of `roles.administrator.assign`.
GET	`/api/v1/users/{id}`	Authenticated; in code: self, or `employees.edit`	One employee (`UserDto`) with departments, managers and project allocations.
POST	`/api/v1/users`	`employees.create`; in code: `employees.role.assign` for any role other than Employee, `roles.administrator.assign` for Administrator	Creates an employee and returns 201.
PUT	`/api/v1/users/{id}`	`employees.edit`; in code: `employees.role.assign` to change the role, and also `roles.administrator.assign` when the change moves anyone into or out of Administrator	Updates the employee. The `password` field is ignored.
PATCH	`/api/v1/users/{id}/status`	`employees.status.change`	Activates or deactivates the account.
DELETE	`/api/v1/users/{id}`	`employees.delete`	Deletes the employee (the caller's id goes to the service) and returns 204.

Method	Route	Permission / policy	Purpose
POST	`/api/v1/users/{id}/mfa/reset`	`employees.mfa.reset`	Administrator reset of a user's authenticator MFA. Ends that user's sessions and returns 204.

Positions and projects

Method	Route	Permission / policy	Purpose
GET	`/api/v1/positions`	`projects.view`	All project positions (QA, Developer, ...). A position is a label and grants no access.
POST	`/api/v1/positions`	`positions.create`	Get-or-create a position by name (case-insensitive match). Returns 200 with the position.
GET	`/api/v1/projects`	`projects.view`	All projects with employee count and manager names (`ProjectWithCountDto`).
GET	`/api/v1/projects/{id}`	`projects.view`	One project.
GET	`/api/v1/projects/{id}/employees`	`projects.view`	Employees on the project (`ProjectEmployeeDto`).
GET	`/api/v1/projects/{id}/members`	`projects.view`	The project team with each member's position, percentage and manager flag (`ProjectMemberDto`).
GET	`/api/v1/projects/{id}/tasks`	`projects.view`	The project's tasks with each assignee's position. `ProjectTaskFilter` query (`status` , `priority` , `deadlineFrom` , `deadlineTo` , `assignedToUserId` , `positionId`), filters combined with AND.
PUT	`/api/v1/projects/{id}/members`	`projects.members.manage`	Replaces the team, all or nothing. Nothing is saved if any member would go over 100 % in total.
POST	`/api/v1/projects`	`projects.create`	Creates a project and returns 201.
PUT	`/api/v1/projects/{id}`	`projects.edit`	Updates the name and description.
DELETE	`/api/v1/projects/{id}`	`projects.delete`	Deletes the project and returns 204.

Example requests

```
// POST /api/v1/departments           (CreateDepartmentRequest; PUT /{id} uses
UpdateDepartmentRequest, same shape)
{ "name": "Engineering", "description": "Product development" }

// PUT /api/v1/departments/{id}/members   (UpdateDepartmentMembersRequest)
{ "managerIds": [2], "employeeIds": [5, 6, 9] }
```

```
// POST /api/v1/users                (CreateUserRequest; ProjectAllocationInput = projectId,
positionId, percent 1-100)
{
  "employeeCode": "EMP-014",
  "fullName": "Jane Doe",
  "username": "jdoe",
  "email": "jane.doe@example.com",
  "password": "<secret>",
  "roleId": 3,
  "isActive": true,
  "projectAllocations": [ { "projectId": 1, "positionId": 2, "percent": 60 } ],
  "departmentIds": [1]
}

// PATCH /api/v1/users/{id}/status    (UserStatusRequest)
{ "isActive": false }

// POST /api/v1/positions             (CreatePositionRequest)
{ "name": "QA" }

// POST /api/v1/projects              (CreateProjectRequest; PUT /{id} uses
UpdateProjectRequest, same shape)
{ "name": "Website Revamp", "description": "Public site redesign" }

// PUT /api/v1/projects/{id}/members  (ReplaceProjectMembersRequest of ProjectMemberInput)
{ "members": [ { "userId": 5, "positionId": 2, "percent": 50, "isManager": false },
               { "userId": 2, "positionId": 1, "percent": 20, "isManager": true } ] }
```

`UpdateUserRequest` has the same fields as `CreateUserRequest` without `password` and `isActive` (a legacy `password` field is accepted and ignored). The `roleId` values of the system roles are 1 = Administrator, 2 = Manager and 3 = Employee (`SystemRoles`).

8.5 Monitoring

`TasksController` , `TaskCommentsController` and `TaskActivityController` (all `/api/v1/tasks` , class-level `[Authorize]`), plus `WorkloadController` , in `Controllers/Monitoring/` . Task scope (Own, Team or Organization) is also enforced in `TaskService` (see §9 and §10).

Tasks, Kanban board and workload

Method	Route	Permission / policy	Purpose
GET	`/api/v1/tasks`	Authenticated (scope applied by the service)	Paged task list (`TaskListFilter` : `assignedToUserId` , `status` , `priority` , `deadlineFrom` , `deadlineTo` , `search` , `page` , `pageSize` , `bucket` , `deadlineBucket` , `departmentId` , `projectId` , `employeeId` , `managerId` , `from` , `to` , `basis`). Returns `PagedResult<TaskListItemDto>` .
POST	`/api/v1/tasks`	`tasks.create`	Creates and assigns a task (project mandatory). Omitted reminder fields take the reminder defaults. Returns 201 with `TaskDetailsDto` .

Method	Route	Permission / policy	Purpose
GET	`/api/v1/tasks/assignees`	Authenticated (scope)	Distinct assignees of the tasks the caller may access, for the employee filter.
GET	`/api/v1/tasks/board`	`board.view`	Flat list of accessible tasks (`TaskCardDto`). The Kanban board groups it by status on the client.
GET	`/api/v1/tasks/{id}`	Authenticated + `TaskAccess`	Task details (`TaskDetailsDto`). 404 when outside the manager's team.
PUT	`/api/v1/tasks/{id}`	`tasks.update`	Edits title, description, priority, dates, estimated hours and, optionally, actual hours.
POST	`/api/v1/tasks/{id}/hours`	Authenticated; in service: assignee, or `tasks.update` plus scope	Adds worked hours (more than 0 and at most 24 per entry) to the running total. 409 on a concurrent change.
DELETE	`/api/v1/tasks/{id}`	`tasks.delete`	Soft-deletes the task and returns 204.
PATCH	`/api/v1/tasks/{id}/status`	`tasks.status.change` + `TaskAccess`	Changes status according to the transition table (incl. `InQA` , see Appendix A). Returns `TaskCardDto` .
POST	`/api/v1/tasks/{id}/reassign`	`tasks.reassign`	Reassigns the task to a new assignee with a reason.
GET	`/api/v1/tasks/{id}/recipients`	Authenticated + `TaskAccess`	The task's additional recipients (`TaskRecipientDto`).
PUT	`/api/v1/tasks/{id}/recipients`	`tasks.recipients.manage`	Replaces the additional recipients and returns 204.
GET	`/api/v1/workload`	`workload.view`	Task load per employee (`EmployeeWorkloadDto`). Optional `projectId` .

Comments and activity history

Method	Route	Permission / policy	Purpose
GET	`/api/v1/tasks/{id}/comments`	Authenticated + `TaskAccess`	The task's discussion (`TaskCommentDto`).

Method	Route	Permission / policy	Purpose
POST	`/api/v1/tasks/{id}/comments`	`tasks.comment.create` + `TaskAccess`	Adds a comment.
PUT	`/api/v1/tasks/{id}/comments/{commentId}`	`tasks.comment.edit` + `TaskAccess`	Edits a comment (the service checks ownership and sets `EditedAt`).
DELETE	`/api/v1/tasks/{id}/comments/{commentId}`	`tasks.comment.delete` + `TaskAccess`	Deletes a comment and returns 204.
GET	`/api/v1/tasks/{id}/activity`	`tasks.activity.view` + `TaskAccess`	Activity history. Query: `eventType` (`TaskActivityEventType`), `actorUserId`, `from`, `to` (a date-only `to` covers the whole day), `search`.
PUT	`/api/v1/tasks/{id}/activity/{entryId}/hours`	Authenticated; in code: role = `Administrator`	Corrects the hours of one "worked hours logged" entry and returns 204.
DELETE	`/api/v1/tasks/{id}/activity/{entryId}/hours`	Authenticated; in code: role = `Administrator`	Removes one "worked hours logged" entry, rebuilds the worked total and returns 204.

Example requests

```
// POST /api/v1/tasks                 (CreateTaskRequest)
{
  "title": "Prepare Monthly Report",
  "description": "Compile October figures",
  "assignedToUserId": 12,
  "priority": "High",
  "startDate": "2026-10-31T04:30:00Z",
  "deadline": "2026-11-03T12:30:00Z",
  "estimatedHours": 6,
  "additionalRecipientUserIds": [7],
  "firstReminderDays": 2,
  "secondReminderDays": 1,
  "overdueReminderFrequency": "Daily",
  "projectId": 3
}

// PUT /api/v1/tasks/{id}             (UpdateTaskRequest)
{ "title": "Prepare Monthly Report", "description": "Compile October figures", "priority":
"Critical",
  "startDate": "2026-10-31T04:30:00Z", "deadline": "2026-11-04T12:30:00Z", "estimatedHours": 8,
"actualHours": null }

// PATCH /api/v1/tasks/{id}/status    (UpdateTaskStatusRequest)
{ "status": "InQA" }
```

```
// POST /api/v1/tasks/{id}/reassign   (ReassignTaskRequest)
{ "newAssigneeUserId": 9, "reason": "Original assignee on leave" }

// POST /api/v1/tasks/{id}/hours      (LogHoursRequest)       PUT .../activity/{entryId}/hours
(UpdateHoursEntryRequest)
{ "hours": 2.5 }

// PUT /api/v1/tasks/{id}/recipients  (SetRecipientsRequest)
{ "recipientUserIds": [7, 11] }

// POST /api/v1/tasks/{id}/comments   (AddCommentRequest; PUT .../comments/{commentId} uses the
same DTO)
{ "body": "Draft uploaded, please review." }
```

8.6 Communication

`MailController` (`/api/v1/mail`) and `NotificationsController` (`/api/v1/notifications`), both with class-level `[Authorize]` , in `Controllers/Communication/` . Every endpoint works on the caller's own mailbox and notifications only. A mail or attachment the caller cannot see returns 404.

Mail

Method	Route	Permission / policy	Purpose
GET	`/api/v1/mail`	`mail.read`	One page of a folder. Query: `folder` (`MailView` : `Inbox` , `Starred` , `Important` , `Sent` , `Drafts` , `Trash`), `search` , `page` , `pageSize` .
GET	`/api/v1/mail/summary`	`mail.read`	Per-folder summary (`MailSummaryDto`).
GET	`/api/v1/mail/unread-count`	`mail.read`	Unread Inbox count for the menu badge.
GET	`/api/v1/mail/items/{itemId}`	`mail.read`	Opens a mail with its conversation and marks it read.
PATCH	`/api/v1/mail/items/{itemId}`	`mail.organize`	Sets the personal flags read, starred and important (null leaves a flag unchanged). Returns 204.
POST	`/api/v1/mail/items/{itemId}/trash`	`mail.organize`	Moves the mail to Trash and returns 204.
POST	`/api/v1/mail/items/{itemId}/restore`	`mail.organize`	Restores the mail from Trash and returns 204.
DELETE	`/api/v1/mail/items/{itemId}`	`mail.organize`	Deletes the mail permanently from Trash, or discards a draft. Returns 204.
GET	`/api/v1/mail/items/{itemId}/compose`	`mail.compose`	Prefilled recipients, subject and quoted text. Query `mode` = `Reply` , `ReplyAll` or `Forward` .
POST	`/api/v1/mail/send`	`mail.compose`	Sends a new mail, a reply or forward, or a saved draft to TaskFlow users.
POST	`/api/v1/mail/drafts`	`mail.compose`	Creates a draft (recipients may still be incomplete).

Method	Route	Permission / policy	Purpose
PUT	`/api/v1/mail/drafts/{messageId}`	`mail.compose`	Updates a draft.
GET	`/api/v1/mail/contacts`	`mail.compose`	Active users matching `search` (name or email), for the recipient picker.
POST	`/api/v1/mail/attachments`	`mail.attachments`	Uploads one file (`multipart/form-data`, field `file`, at most 10 MB per file). The request size limit is 10 MB plus 1 MB of multipart overhead. Returns `MailAttachmentDto`.
GET	`/api/v1/mail/attachments/{attachmentId}`	`mail.attachments`	Downloads the attachment as a file, with `X-Content-Type-Options: nosniff`. Never rendered inline.
DELETE	`/api/v1/mail/attachments/{attachmentId}`	`mail.attachments`	Removes an upload that is not yet on a mail or draft. Returns 204.

Notifications

Method	Route	Permission / policy	Purpose
GET	`/api/v1/notifications`	`notifications.view`	A page of the caller's notifications, newest first. `NotificationFilter` query: `type` (`NotificationType`), `isRead`, `page`, `pageSize`.
GET	`/api/v1/notifications/unread-count`	`notifications.view`	Unread count for the header badge (`{ count }`).
POST	`/api/v1/notifications/{id}/read`	`notifications.manage`	Marks one notification read and returns 204. 404 if the caller does not own it.
POST	`/api/v1/notifications/read-all`	`notifications.manage`	Marks all of the caller's notifications read and returns 204.

Example requests

```
// POST /api/v1/mail/send             (SendMailRequest; draftId sends a saved draft,
parentItemId + mode for reply/forward)
{
  "to": ["jane.doe@example.com"],
  "cc": [],
  "subject": "Monthly report",
  "body": "Please find the report attached.",
  "attachmentIds": [41],
  "draftId": null,
  "parentItemId": null,
  "mode": null
}

// POST /api/v1/mail/drafts           (SaveDraftRequest; PUT /drafts/{messageId} uses the same
DTO)
{ "to": ["jane.doe@example.com"], "cc": null, "subject": "Re: Monthly report", "body":
"Draft…",
```

```
  "attachmentIds": [], "parentItemId": 120, "mode": "Reply" }

// PATCH /api/v1/mail/items/{itemId}  (UpdateMailFlagsRequest)
{ "isRead": null, "isStarred": true, "isImportant": null }
```

Mail limits (`MailRules`): subject 200 characters, body 20,000, at most 50 recipients, at most 5 attachments of 10 MB each (25 MB in total per mail), at most 20 pending uploads (100 MB). See §15.

8.7 Reports

`ReportController` (`/api/v1/reports`, class-level `reports.view`), in `Controllers/Reports/`.

Method	Route	Permission / policy	Purpose
GET	`/api/v1/reports`	`reports.view`	Totals for the filtered scope (`ReportSummaryDto`: total, pending, completed, overdue). `ReportFilter` query: `deadlineFrom`, `deadlineTo`, `assignedToUserId`, `projectId`, `status`, `priority`.
GET	`/api/v1/reports/export`	`reports.view` + `reports.export`	The filtered task rows as a CSV file download (UTF-8, `ReportCsv.ContentType` / `ReportCsv.FileName`).

8.8 Profile

`ProfileController` (`/api/v1/users/me`, class-level `[Authorize]`), in `Controllers/Profile/`. The profile keys (`profile.view`, `profile.edit`, `profile.password.change`, `profile.mfa.manage`) are always on, so these endpoints need no policy beyond sign-in. The user id always comes from the token. `/api/v1/users/me` and `/api/v1/users/me/mfa/*` stay reachable while MFA setup is pending.

Method	Route	Permission / policy	Purpose
GET	`/api/v1/users/me`	Authenticated (self)	The caller's own account (`UserDto`).
PUT	`/api/v1/users/me`	Authenticated (self)	Updates the caller's full name, email and mobile number. Role, project, username, employee code and status are not in the contract.
POST	`/api/v1/users/me/password`	Authenticated (self)	Changes the password after checking an authenticator code (the current password is not asked for). Ends every session of the user and returns 204.
POST	`/api/v1/users/me/mfa/setup`	Authenticated (self)	Starts authenticator setup after checking the current password. Returns the key and the provisioning (QR) URI once.
POST	`/api/v1/users/me/mfa/enable`	Authenticated (self)	Confirms setup with the first code, turns MFA on and returns the 10 backup codes once.
POST	`/api/v1/users/me/mfa/check`	Authenticated (self)	Checks a code without spending it. Returns 204 on success, 400 for a wrong code and 429 when locked out (shares the sign-in lockout).
POST	`/api/v1/users/me/mfa/backup-codes`	Authenticated (self)	Regenerates the backup codes (the old set stops working). Needs the current password and a valid code.

Example requests

```
// PUT /api/v1/users/me               (UpdateOwnProfileRequest)
{ "fullName": "Jane Doe", "email": "jane.doe@example.com", "mobileNumber": "+919876543210" }
```

```
// POST /api/v1/users/me/password     (ChangePasswordRequest)
{ "newPassword": "<secret>", "mfaCode": "123456" }

// POST /api/v1/users/me/mfa/setup    (MfaSetupRequest)
{ "currentPassword": "<secret>" }

// POST /api/v1/users/me/mfa/enable   (MfaEnableRequest; /mfa/check uses the same DTO)
{ "code": "123456" }

// POST /api/v1/users/me/mfa/backup-codes   (MfaPasswordCodeRequest)
{ "currentPassword": "<secret>", "code": "123456" }
```

8.9 Settings

`ApplicationSettingsController`, `ReminderSettingsController`, `RolesController` and `AiProvidersController`, in `Controllers/Settings/`. `AiProvidersController` is routed under `/api/v1/assistant/providers` but belongs to Settings (TFA Settings screen).

Application and reminder settings

Method	Route	Permission / policy	Purpose
GET	`/api/v1/settings/application`	Authenticated	Non-secret application settings (name, timezone, default priority, in-app notifications on or off), readable by every signed-in user.
PUT	`/api/v1/settings/application`	`settings.application.manage`	Saves the application settings. Every field is required.
GET	`/api/v1/settings/reminders`	`settings.reminders.manage`	Global reminder configuration, including the application timezone (read-only here).
PUT	`/api/v1/settings/reminders`	`settings.reminders.manage`	Saves the reminder configuration. Every field is required, and there is no timezone field.

Role configuration (`RolesController`, route `/api/v1`, class-level `roles.manage`)

Method	Route	Permission / policy	Purpose
GET	`/api/v1/roles`	`roles.manage`; in code: `roles.view.all` to see every role, otherwise only the caller's own role	Roles with user counts and permission keys (`RoleDto`).
GET	`/api/v1/roles/{id}`	`roles.manage`	One role with its permission keys.
POST	`/api/v1/roles`	`roles.manage`; in code: `roles.modify.access`	Creates a custom role and returns 201.
PUT	`/api/v1/roles/{id}`	`roles.manage`; in code: `roles.modify.access`	Renames a custom role or changes its permission keys. System roles are read-only.
DELETE	`/api/v1/roles/{id}`	`roles.manage`; in code: `roles.modify.access`	Deletes a custom role and returns 204.

Method	Route	Permission / policy	Purpose
GET	`/api/v1/permissions`	`roles.manage`	The fixed permission catalog (`PermissionDto`: key, name, module, screen, section, action, scope group/rank, assignable, always granted).
POST	`/api/v1/permissions`	`roles.manage`	Always 403 "The permission list is fixed and cannot be changed."
PUT	`/api/v1/permissions/{id}`	`roles.manage`	Always 403 (fixed catalog).
DELETE	`/api/v1/permissions/{id}`	`roles.manage`	Always 403 (fixed catalog).

TFA Settings: AI providers (`AiProvidersController`, class-level `[Authorize]`)

Method	Route	Permission / policy	Purpose
GET	`/api/v1/assistant/providers/config`	`settings.ai.view`	Every provider (`AiProviderKind`: `Ollama`, `Gemini`, `OpenAI`) with status, model lists, `keySet` and `keyLast4`. The API key is never returned.
PUT	`/api/v1/assistant/providers/config`	`settings.ai.edit`	Saves provider settings. An empty `apiKey` keeps the stored key, and `clearKey` removes it.
DELETE	`/api/v1/assistant/providers/{kind}/models`	`settings.ai.model.remove`	Removes an administrator-added model (query `name`, required). Built-in models and the saved model stay.
POST	`/api/v1/assistant/providers/{kind}/test`	`settings.ai.test`	Sends one tiny request to the provider. Returns `{ ok, message, failureKind }`, where `message` is fixed text and never the provider's own error.

Example requests

```
// PUT /api/v1/settings/application   (UpdateApplicationSettingsRequest)
{ "name": "TaskFlow", "timezone": "Asia/Kolkata", "defaultPriority": "Medium",
"inAppNotificationsEnabled": true }

// PUT /api/v1/settings/reminders     (UpdateReminderSettingsRequest)
{ "enabled": true, "deadlineDayEnabled": true, "defaultFirstDays": 2, "defaultSecondDays": 1,
  "defaultOverdueFrequency": "Daily", "mailEnabled": true }

// POST /api/v1/roles                 (SaveRoleRequest; PUT /roles/{id} uses the same DTO)
{ "name": "Team Lead", "permissionKeys": ["tasks.view.all", "tasks.create", "tasks.update",
"board.view", "reports.view"] }

// PUT /api/v1/assistant/providers/config   (UpdateAiProvidersRequest of
UpdateAiProviderRequest)
{ "providers": [
    { "kind": "Ollama", "isEnabled": true, "modelName": "llama3.2:3b", "baseUrl":
```

```
"http://localhost:11434", "apiKey": null, "clearKey": null },
    { "kind": "OpenAI", "isEnabled": true, "modelName": "gpt-4o-mini", "baseUrl": null,
"apiKey": "<secret>", "clearKey": false } ] }
```

The example values are the built-in defaults from `Common/Constants/AiProviderCatalog.cs`: `llama3.2:3b` and base URL `http://localhost:11434` for Ollama, `gemini-2.0-flash` for Gemini, `gpt-4o-mini` for OpenAI. `AiProviderSeeder` enables only Ollama on a new installation.

8.10 Assistant

`AssistantController` (`/api/v1/assistant`, class-level `[Authorize]`), in `Controllers/Assistant/`. Actions run only through the confirm endpoint. The model cannot call it (see §16).

Method	Route	Permission / policy	Purpose
GET	`/api/v1/assistant/conversation`	`assistant.chat`	The caller's current conversation with its action cards (`AssistantConversationDto`).
POST	`/api/v1/assistant/conversation/messages`	`assistant.chat`	Sends a message to the chosen provider. A provider failure returns 200 with a `failure` object. A message that cannot be handled returns 400, and sending too fast returns 429 (`Retry-After`).
POST	`/api/v1/assistant/conversation/new`	`assistant.chat`	Starts a new conversation (earlier messages are no longer used as context) and returns 204.
PUT	`/api/v1/assistant/actions/{id}`	`assistant.actions.confirm`	Edits a prepared mail action's recipients, subject and body before it is confirmed.
POST	`/api/v1/assistant/actions/{id}/confirm`	`assistant.actions.confirm`	Runs a pending action (`SendMail`, `UpdateOwnProfile`, `ChangeUserRole`) with the caller's own permissions. Returns `AssistantActionResultDto`.
POST	`/api/v1/assistant/actions/{id}/cancel`	`assistant.actions.confirm`	Cancels a pending action.
GET	`/api/v1/assistant/providers`	`assistant.chat`	Configured providers for the chat model selector, with no key details (`AiProviderOptionDto`).

Example requests

```
// POST /api/v1/assistant/conversation/messages   (SendAssistantMessageRequest)
{ "text": "Which of my tasks should I do first?", "provider": "Ollama" }

// PUT /api/v1/assistant/actions/{id}             (EditMailActionRequest)
{ "to": ["jane.doe@example.com"], "subject": "Status of TSK-000042", "body": "Hi Jane, could
you share an update?" }
```

8.11 Endpoint count check

Section	Controllers	Actions
8.2 Authentication	`AuthController` (5), `PasswordRecoveryController` (2)	7
8.3 Dashboard	`DashboardController` (3), `AnalyticsController` (5)	8
8.4 Management	`DepartmentsController` (7), `EmployeesController` (8), `PositionsController` (2), `ProjectsController` (9)	26
8.5 Monitoring	`TasksController` (12), `WorkloadController` (1), `TaskCommentsController` (4), `TaskActivityController` (3)	20
8.6 Communication	`MailController` (16), `NotificationsController` (4)	20
8.7 Reports	`ReportController` (2)	2
8.8 Profile	`ProfileController` (7)	7
8.9 Settings	`ApplicationSettingsController` (2), `ReminderSettingsController` (2), `RolesController` (9), `AiProvidersController` (4)	17
8.10 Assistant	`AssistantController` (7)	7
Total	21 controllers	**114**

The three `/api/v1/permissions` write rows are one C# method (`CatalogIsFixed`) carrying three HTTP attributes. They count as three HTTP actions, matching the fact sheet.

---

9. Authentication & Authorization

9.1 Components

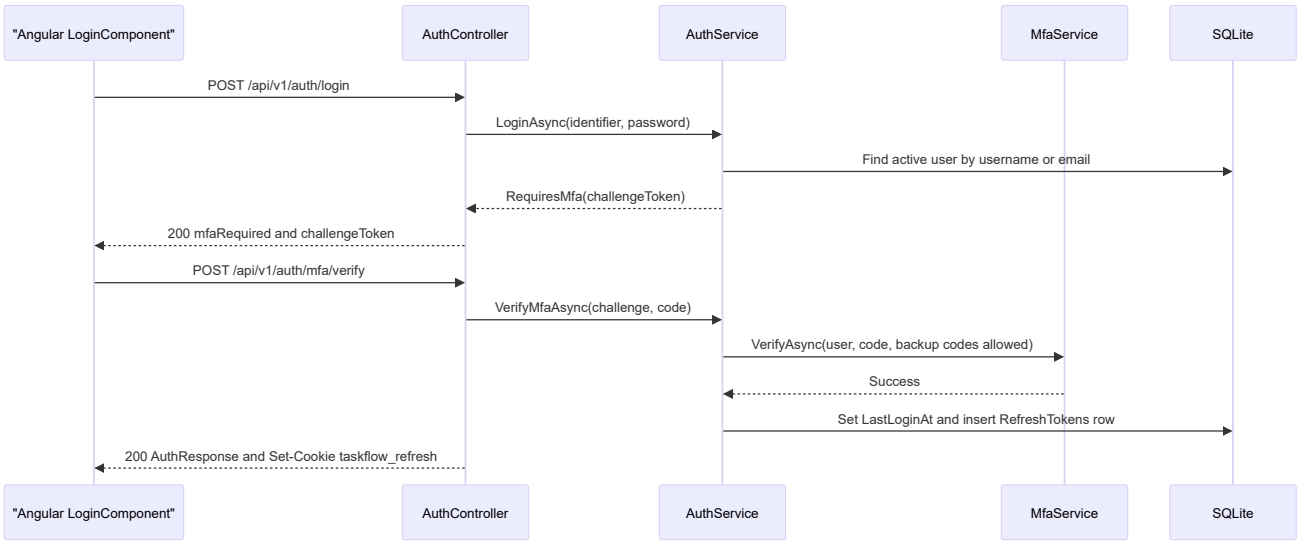
Concern	Implementation (backend `TaskFlow.Api`)	Frontend (`shared/auth`, `modules/auth`)
Password sign-in	`AuthController` → `AuthService.LoginAsync`; ASP.NET Core Identity `PasswordHasher<User>` (PBKDF2)	`LoginComponent`, `AuthService.login`
Second step (MFA)	`AuthService.VerifyMfaAsync` → `MfaService.VerifyAsync`; challenge token from `MfaSecretProtector.CreateChallenge`	`LoginComponent` (challenge step), `AuthService.verifyMfa`
MFA enrolment	`ProfileController` (`/api/v1/users/me/mfa/*`) → `ProfileService` → `MfaService`; `Totp`, `Base32` (in-house RFC 6238)	`SecuritySetupComponent` (`/security-setup`), `securitySetupGuard`, `mfaSetupGuard`
Mandatory-MFA gate	`MfaSetupGateMiddleware` (`Mfa:Required`)	`authInterceptor` handles `403` with `code` = `mfa_setup_required`

Concern	Implementation (backend `TaskFlow.Api`)	Frontend (`shared/auth`, `modules/auth`)
Tokens	`JwtTokenService` (access JWT + refresh token), `RefreshTokenCookie`, `RefreshTokenRepository`	`TokenStore` (in memory), `authInterceptor` (silent refresh), `restoreSession()`
Session validation	`Middleware/SessionValidation.cs` on `JwtBearerEvents.OnTokenValidated` → `SessionService`	Shell session check (`GET /api/v1/auth/me`)
Password recovery	`PasswordRecoveryController` → `PasswordRecoveryService`, `AttemptLimiter`, `PasswordResetGrant`	`AccountRecoveryComponent` (`/account-recovery`)
Authorization	`perm:{key}` policies from `PermissionCatalog`, `RequirePermissionAttribute`, `TaskAccessHandler`, `EffectivePermissions`, `DeniedActionAuditHandler`	`permissionGuard`, permission checks on menus/buttons (see §12)
Roles	`RolesController` → `RoleService`; `AccessControlSeeder`; `SystemRoles`	Role Configuration screen (see §12)

Pipeline order in `Program.cs`: `UseAuthentication` → `MfaSetupGateMiddleware` → `UseAuthorization` (CORS and request logging run earlier, see §3).

9.2 Login flow (Screen 1)

1. The client posts `{ emailOrUsername, password }` to `` `/api/v1/auth/login` `` (`[AllowAnonymous]`, `LoginRequestValidator`).
2. `AccountRepository.FindByLoginForUpdateAsync` matches `Username` or `Email` (emails are stored lower-case by `EmailPolicy`); soft-deleted users are excluded by the global query filter.
3. Missing user, `IsActive = false` (BR-17) or a wrong password → a bare `401` with no body (no user enumeration). `SuccessRehashNeeded` re-hashes and stores the password.
4. **MFA on** (`MfaEnabledAt` set): the response is `{ mfaRequired: true, challengeToken }`. The challenge is a Data-Protection-signed token bound to the user id and `SecurityStamp`, valid for `Mfa:ChallengeMinutes` (5). No session and no `LastLoginAt` update yet.
5. The client posts `{ challengeToken, code }` to `` `/api/v1/auth/mfa/verify` ``. The challenge must be valid, unexpired and carry the current stamp; the code may be a TOTP or an unused backup code. Results: success → session; `LockedOut` → `429`; anything else → `400`.
6. **Session issued** (step 5, or step 4 when MFA is off – possible only while `Mfa:Required` is false or before enrolment): `LastLoginAt` set, access token returned in the body (`AuthResponse`), refresh token set in the `taskflow_refresh` cookie.
7. The `UserDto` in the response carries role, effective permission keys and `mfaSetupRequired` (= `Mfa:Required` and MFA not on). The SPA routes to the `returnUrl` (internal paths only) or to `dashboardRouteFor(role, permissions)`; while `mfaSetupRequired` is true the shell guard sends the user to `/security-setup`.



9.3 Mandatory MFA (TOTP + backup codes)

Item	Rule (from `MfaService`, `MfaOptions`, `MfaSetupGateMiddleware`)
Enforcement	`Mfa:Required` = `true` (default). `SessionValidation` adds the `mfa_setup_required` claim while the user has no MFA; `MfaSetupGateMiddleware` then returns `403` ProblemDetails (`detail` "Set up two-step verification to continue.", `code` = `mfa_setup_required`) for every `/api` call except `/api/v1/auth/*`, `/api/v1/users/me/mfa/*` and exactly `/api/v1/users/me`. Break-glass: `Mfa__Required=false`.
Start setup	`` `/api/v1/users/me/mfa/setup` `` (POST) with the current password. Password attempts are budgeted per user (§9.6). Generates a 20-byte secret, stores it encrypted (`MfaSecretProtector`, ASP.NET Core Data Protection, key ring in `dp-keys`) in `User.MfaSecretProtected`, returns the Base32 key and an `otpauth://` URI (issuer `Mfa:Issuer`) once. Refused when MFA is already on.
Enable	`` `/api/v1/users/me/mfa/enable` `` (POST) with the first TOTP code (backup codes not allowed). Sets `MfaEnabledAt` and returns `Mfa:BackupCodeCount` (10) backup codes once, shown as `XXXXX-XXXXX`.
Backup codes	Stored only as SHA-256 hashes in `MfaBackupCode`; single use (`UsedAt`). Regenerated with `` `/api/v1/users/me/mfa/backup-codes` `` (POST, password + code); old codes are deleted.
Code check	`` `/api/v1/users/me/mfa/check` `` (POST) checks a TOTP without spending it (used by Profile).
TOTP validation	6 digits, window current step ±1, constant-time compare; the step must be greater than `MfaLastUsedStep` (replay protection). `MfaLastUsedStep`, `MfaFailedAttempts` and `MfaBackupCode.UsedAt` are concurrency tokens, so a code cannot be spent twice in parallel.
Lockout	`Mfa:MaxFailedAttempts` (5) consecutive wrong codes set `MfaLockedUntil = now + Mfa:LockoutMinutes` (5). The counter is shared by sign-in, code check, backup-code regeneration and password change.
Administrator reset	`` `/api/v1/users/{id}/mfa/reset` `` (POST, `employees.mfa.reset`): clears the key, flags and backup codes, rotates `SecurityStamp`, revokes all refresh tokens and publishes a `System` notification. The user must enrol again at next sign-in.

The self-service endpoints take the user id from the token, never from the request. Their catalog keys (`profile.mfa.manage` and the other `profile.*` keys) are Always On and are not checked by attribute (§9.8).

9.4 Tokens and sessions

Item	Value / behaviour
Access token	JWT, HMAC-SHA256 with `Jwt:Key` (`<secret>`), issuer `Jwt:Issuer`, no audience; lifetime `Jwt:AccessTokenMinutes` (code default 30). Claims: `sub`, `ClaimTypes.NameIdentifier`, `ClaimTypes.Name`, `ClaimTypes.Role`, `sstamp`, `jti`. Kept **in memory only** on the client (`TokenStore`).
Refresh token	64 random bytes, lifetime `Jwt:RefreshTokenDays` (code default 7); stored only as a SHA-256 hash in `RefreshToken`. Cookie `taskflow_refresh`: HttpOnly, Secure, SameSite=Strict, Path `/api/v1/auth`.
Refresh	`` `/api/v1/auth/refresh` `` (POST, `[AllowAnonymous]`, cookie): revokes the presented token and issues a new pair (rotation). Unknown, revoked, expired token or inactive user → `401` and the cookie is cleared.
Logout	`` `/api/v1/auth/logout` `` (POST, `[AllowAnonymous]`): revokes the cookie's token (no-op if unknown) and always deletes the cookie → `204`.
Current user	`` `/api/v1/auth/me` `` (GET, `[Authorize]`): user, role, effective permissions, `mfaSetupRequired`. The shell calls it every 15 s and on `visibilitychange` to sync permissions.
Session validation	On every authenticated request `SessionValidation` loads `SessionUser` (`AccountRepository.GetSessionAsync`, no tracking). The request fails with "The session is no longer valid. Sign in again." when the user is missing, inactive or the `sstamp` claim differs from `User.SecurityStamp`. It then **replaces** the role claim and the `permission` claims from the database, so role and permission edits apply on the next call.
Client renewal	`authInterceptor` adds the bearer token, refreshes once on `401` (shared in-flight call) and retries; a failed refresh clears the session and opens `/login?reason=session-ended`. `restoreSession()` runs at start-up (`provideAppInitializer`).

Events that end all of a user's sessions (new `SecurityStamp` and/or revoked refresh tokens):

Event	Service	Stamp rotated	Refresh tokens revoked
Password change (Profile)	`ProfileService.ChangePasswordAsync`	Yes	Yes
Password recovery reset	`PasswordRecoveryService.ResetPasswordAsync`	Yes	Yes
Administrator MFA reset	`MfaService.ResetAsync`	Yes	Yes
Username or role changed by an administrator	`EmployeeService.UpdateAsync`	Yes	Yes
Deactivation / soft delete of the employee	`EmployeeService.UpdateStatusAsync`, `DeleteAsync`	Yes	Yes
Permissions of a custom role changed	`RoleService.UpdateAsync` (all holders of the role)	Yes	Yes

9.5 Password recovery (authenticator only)

Self-service recovery uses the authenticator app; no email, SMS or link is sent.

1. `` `/api/v1/auth/password-recovery/authenticator/verify` `` (POST, `[AllowAnonymous]`) with `{ identifier, code }`. `AttemptLimiter` refuses blocked caller addresses before any database work, then reserves an attempt for the account key **before** the code is checked. The code must be a current TOTP; backup codes are refused, and a wrong code does **not** increase the account's MFA failure counter (`countFailure: false`), so a stranger cannot lock the owner out. An existing MFA lock still refuses.

2. Success returns a single-use reset token: 32 random bytes (Base64Url), stored as a SHA-256 hash in `PasswordResetGrant`, valid 10 minutes.
3. `` `/api/v1/auth/password-recovery/reset` `` (POST) with `{ resetToken, newPassword, confirmPassword }` (6–128 characters, must match). Sets the new hash, rotates `SecurityStamp`, revokes all refresh tokens, marks this and every other unused grant used (`UsedAt` is a concurrency token), and publishes a `System` notification ("Your password was reset with your authenticator app. …").
4. Every failure returns one message (anti-enumeration): `400` "The details are incorrect, or the code has expired. …", `429` "Too many attempts. Try again in a few minutes.", or `400` "This reset link is no longer valid. …" for a bad reset token.

Users without MFA (possible only when `Mfa:Required` is false) cannot use self-service recovery; an administrator handles them.

9.6 Attempt limits and lockouts

Limiter	Scope	Limit	Used by
`AttemptLimiter` (singleton, in memory)	Per account key (`user:{id}` or `name:{IDENTIFIER}`)	5 per sliding 5-minute window	Password recovery verify
`AttemptLimiter`	Per caller address (IPv6 pooled by /64)	20 per window	Password recovery verify
`AttemptLimiter`	Per user password key `pwd:{id}` (no address)	5 per window	MFA setup and backup-code regeneration (password re-entry)
MFA lockout (`User.MfaFailedAttempts`, `MfaLockedUntil`)	Per account, in the database	5 wrong codes → 5-minute lock	Sign-in second step, code check, backup-code regeneration, password change

The limiter tracks at most 10 000 keys (oldest evicted). It is in memory: a restart resets it and it is not shared between instances (single-instance deployment, see §2).

9.7 Password and account rules (Screens 5, 19)

Rule	Where
Passwords are hashed with `PasswordHasher<User>` (PBKDF2); never stored or returned in plain text.	`AuthService`, `EmployeeService.CreateAsync`, `ProfileService`, `PasswordRecoveryService`
Password length 6–128 characters.	`ChangePasswordRequestValidator`, `RecoverPasswordRequestValidator`
Changing one's own password (`` `/api/v1/users/me/password` ``, POST) requires MFA to be on and a current authenticator or backup code (`MfaService.RequireCodeAsync`); it ends all sessions and the client returns to `/login?reason=credentials-changed`.	`ProfileService.ChangePasswordAsync`
An administrator sets a password only when creating an employee; `UpdateUserRequest.Password` is ignored on update.	`EmployeeService`
Inactive or deleted users cannot sign in, refresh or keep a session (BR-17).	`AuthService`, `SessionValidation`

Rule	Where
Duplicate email (local part, deleted users included) or username is rejected.	`EmployeeService`, `ProfileService` (§10.11, §10.14)
Users cannot change their own role, status, username, employee code or allocations from Profile.	`ProfileService.UpdateAsync`

9.8 Permission model

Catalog. `Common/Security/PermissionCatalog.cs` is the single source of truth: 64 `PermissionDefinition` entries placed in a Module → Screen → Section → Action tree, plus 6 legacy keys handled by `EffectivePermissions` – 70 keys in `PermissionKeys.cs` in total. Each definition carries the role defaults and these flags:

Flag	Meaning	Keys
`AlwaysGranted` (+ `IsAssignable: false`)	Every signed-in user has the feature; not stored, not shown as a tick box, not emitted as a claim. Endpoints serving it use `[Authorize]` only.	`dashboard.view.own`, `tasks.view.own`, `profile.view`, `profile.edit`, `profile.password.change`, `profile.mfa.manage`
`IsAssignable: false` (stored)	Only the Administrator role may hold it.	`roles.administrator.assign`
`Requires`	The key needs another key (usually the screen's View). `RoleService` refuses a role that breaks it: ""X" also needs "Y"."	e.g. `employees.create` → `employees.view`, `reports.export` → `reports.view`, `board.filters.use` → `board.view`
`ScopeGroup` / `ScopeRank`	Levels of one selector; a higher level includes the lower ones (§9.10).	`dashboard.view.*` (own 1, team 2, organization 3), `tasks.view.*` (own 1, all 2)
`GrantedToManager`, `GrantedToEmployee`	Default grant to that system role, applied once when the permission is first seeded.	see summary below
`GrantToAllExistingRoles`, `GrantFrom`	Migration aids at first seeding: every existing role, or every holder of an older key, receives the new key so nobody loses access.	e.g. mail and assistant keys; `workload.filters.use` from `workload.view`

Policies. `CrossCuttingRegistration` registers one policy `perm:{key}` per catalog entry (`PermissionPolicy.Prefix` = `perm:`), each `RequireClaim("permission", key)`. Controllers use `[RequirePermission(PermissionKeys.X)]` or `[Authorize(Policy = PermissionPolicy.Prefix + PermissionKeys.X)]` (§8 lists the key per endpoint). The `permission` claims are re-read from `RolePermission` on every request (§9.4), so a change to a role applies to its users' very next call. The assistant authorizes its tools against the same policy names (see §16).

Effective permissions. `EffectivePermissions.Expand(heldKeys)` = held keys ∪ the finer children of any legacy key ∪ the lower levels of any scope selector (always-on levels excluded). The same expansion feeds the claims, `` `/api/v1/auth/me` `` and the services' own re-checks (`TaskService.HasPermissionAsync`).

Seeding. `AccessControlSeeder` runs on every start (independent of `Seed:Enabled`): it ensures the three system roles, inserts catalog entries missing from `Permission`, grants **every** stored permission to the Administrator role (including ones added in later releases) and applies Manager/Employee defaults only to newly inserted keys, so later edits are never overwritten.

Role defaults summary (full list per key in Appendix C):

Module (catalog `Group`)	Keys	Administrator	Manager (default granted)	Employee (default granted)	Custom role
Dashboard	5	All	3 (`dashboard.view.team`, `dashboard.analytics.view`, `dashboard.filters.use`) + own (Always On)	2 (`dashboard.analytics.view`, `dashboard.filters.use`) + own (Always On)	Configurable
TaskFlow Monitoring	17	All	15 (all except `tasks.delete`; `tasks.view.own` Always On)	7 (comment create/edit/delete, `tasks.activity.view`, `board.view`, `board.filters.use`, `tasks.status.change`) + `tasks.view.own` (Always On)	Configurable
TaskFlow Management	18	All	3 (`employees.view`, `projects.view`, `departments.view`)	0	Configurable
Communication Management	6	All	6 (`mail.*`, `notifications.*`)	6	Configurable
Report	2	All	2 (`reports.view`, `reports.export`)	0	Configurable
Settings	10	All	1 (`settings.reminders.manage`)	0	Configurable, except `roles.administrator.assign` (Administrator only)
My Profile	4	Always On	Always On	Always On	Always On
TFA assistant	2	All	2 (`assistant.chat`, `assistant.actions.confirm`)	2	Configurable
Total	**64**	58 stored + 6 Always On	32 granted	17 granted	–

Legacy keys. Older broad keys stay valid for roles that still hold them; `EffectivePermissions.LegacyChildren` maps them to their finer replacements. They are not in the catalog, not shown and not assignable; saving a role in Role Configuration stores the finer keys instead (`RoleService.ResolvePermissionsAsync`).

Legacy key	Expands to
`dashboard.admin`	`dashboard.view.organization`
`dashboard.manager`	`dashboard.view.team`
`projects.manage`	`projects.create`, `projects.edit`, `projects.delete`, `projects.members.manage`, `positions.create`

Legacy key	Expands to
`departments.manage`	`departments.create`, `departments.edit`, `departments.delete`, `departments.members.manage`
`employees.manage`	`employees.create`, `employees.edit`, `employees.status.change`, `employees.mfa.reset`
`settings.ai.manage`	`settings.ai.view`, `settings.ai.edit`, `settings.ai.test`, `settings.ai.model.remove`

9.9 Roles (system and custom)

Rule	Implementation
System roles **Administrator** (id 1), **Manager** (id 2), **Employee** (id 3) (`SystemRoles`) are created by the migration and by `AccessControlSeeder`, have `Role.IsSystem = true` and are read-only.	`RoleService.LoadCustomRoleAsync` refuses update/delete ("… is a system role and is read-only …"); SQLite triggers `TR_Roles_SystemNoUpdate`, `TR_Roles_SystemNoDelete`, `TR_RolePermissions_SystemNoUpdate`, `TR_RolePermissions_SystemNoDelete` back this up.
Custom roles are created, renamed, re-permissioned and deleted in Role Configuration (`` `/api/v1/roles` ``, `` `/api/v1/roles/{id}` ``).	`RolesController` (`roles.manage`) → `RoleService`
Name required, at most 100 characters, unique case-insensitively (unique index decides concurrent saves).	`RoleService.ValidateName`, `EnsureNameFreeAsync`
Unknown keys, broken `Requires`, or non-assignable keys are refused; always-on keys are silently dropped (never stored).	`RoleService.ResolvePermissionsAsync`
A custom role still held by users cannot be deleted ("n users still hold this role. Move them to another role first.").	`RoleService.DeleteAsync`
Changing a custom role's permissions ends the sessions of all its holders.	`RoleService.EndSessionsOfRoleHoldersAsync`
Without `roles.view.all` the role list shows only the caller's own role; without `roles.modify.access` create/update/delete return `403` (view mode).	`RolesController`
The permission catalog is fixed: `POST` `` `/api/v1/permissions` ``, `PUT` / `DELETE` `` `/api/v1/permissions/{id}` `` always return `403` "The permission list is fixed and cannot be changed."; triggers `TR_Permissions_NoUpdate`, `TR_Permissions_NoDelete` forbid changes in the database.	`RolesController.CatalogIsFixed`
Choosing any role other than Employee when creating a user, or changing a user's role, needs `employees.role.assign`; moving anyone into or out of Administrator also needs `roles.administrator.assign`. `` `/api/v1/users/roles` `` lists the Administrator role only to callers who may hand it out.	`EmployeesController.Create`, `Update`, `GetRoleOptions`
At least one active Administrator must remain (cannot be moved to another role, deactivated or deleted).	`EmployeeService.EnsureAnotherActiveAdministratorAsync`

9.10 Data scopes and resource checks

Scope	Dashboard (`dashboard.view.*`)	Tasks (`tasks.view.*`)
Own (Always On)	`` `/api/v1/dashboard/employee` `` – `EmployeeDashboardScope`	Tasks the user is assigned to, created, or is a recipient of
Team	`` `/api/v1/dashboard/manager` `` (`dashboard.view.team`) – `ManagerDashboardScope`	A caller holding `dashboard.view.team` but not `dashboard.view.organization` is *manager-scoped*: with `tasks.view.all` they see their team's tasks plus their own
Organization	`` `/api/v1/dashboard/admin` `` (`dashboard.view.organization`) – `AdminDashboardScope`	`tasks.view.all` without manager scoping – every task

Team = the active users (other than the manager) in a department the manager manages (`DepartmentManager`) or on a project where the manager is flagged `IsProjectManager` (`UserProjectAllocation`).

`TaskAccess` policy (`AuthPolicies.TaskAccess`, `TaskAccessHandler` → `TaskAccessService`): the resource is the task id; access is granted to the assignee, the creator, any `TaskRecipient`, or a holder of `tasks.view.all`. A missing or deleted task fails the check. Controllers call it through `IAuthorizationService.AuthorizeAsync` and return `403` on failure for: task details, status change, recipients read, all comment endpoints and the activity history. `TaskService` repeats the rule (`TaskAccessScope`, defence in depth) and additionally applies the manager team check (`ITaskRepository.IsTeamTaskAsync`) for details, update, status, hours and reassign; out-of-scope mutations return `403` "You do not have access to this task."

Other in-code checks

Check	Rule
`` `/api/v1/users/{id}` `` (GET)	Own record, or a holder of `employees.edit`
Worked-hours entry correction (`` `/api/v1/tasks/{id}/activity/{entryId}/hours` ``, PUT/DELETE)	Role name must be `Administrator` (not a permission key)
Log hours (`` `/api/v1/tasks/{id}/hours` ``, POST)	Assignee, or a holder of `tasks.update`
Edit/delete a comment	Author only, on top of `tasks.comment.edit` / `tasks.comment.delete` and `TaskAccess`
Notifications	Owner only (`UserId` match in `NotificationService`)

> ⚠️ To be confirmed: `TaskAccessHandler` does not apply the manager team restriction, so comment and activity endpoints are reachable by a manager-scoped `tasks.view.all` holder for tasks outside their team; `DELETE /api/v1/tasks/{id}` and `PUT /api/v1/tasks/{id}/recipients` have no task-scope check (see §22).

9.11 Denied-action audit

`DeniedActionAuditHandler` (`IAuthorizationMiddlewareResultHandler`) logs every refused authorization of an authenticated caller as a warning – "Denied action: user {UserId} attempted {Method} {Path}; authorization policy {Policy} refused." – with identifiers only (no body, query string or credential). The response keeps its normal `401` / `403` shape.

---

10. Business Logic

All rules below are enforced server-side; FluentValidation validators run first through `ValidationFilter` (400 validation ProblemDetails), service exceptions are mapped by `GlobalExceptionHandler` (see §6).

10.1 Task creation (Screen 8, BRD §3.1)

`/api/v1/tasks` (POST, `tasks.create`) → `TaskService.CreateAsync`.

Validation	Message
`Title` required, ≤ 200; `Description` ≤ 4000	FluentValidation defaults
Assignee required, live and active	"An assigned employee is required." / "The assigned employee does not exist or is not active."
Project required and active	"A project is required." / "The selected project does not exist."
`Deadline` after `StartDate` (FR-10)	"Deadline must be after the start date."
`EstimatedHours` > 0 (FR-11)	"Estimated hours must be a positive number."
`Priority` ∈ `TaskPriority` (Low, Medium, High, Critical)	FluentValidation default
At most 50 additional recipients, each live	"A task can have at most 50 additional recipients." / "One or more recipients do not exist."
`FirstReminderDays`, `SecondReminderDays` 0–30 and ordered (`ReminderIntervals.AreOrdered`: first = 0 or second ≤ first)	"The second reminder cannot come before the first reminder."

Steps (one transaction, `IUnitOfWork.ExecuteInTransactionAsync`):

1. Null reminder fields take the defaults from Reminder Settings (`Reminder.DefaultFirstDays`, `Reminder.DefaultSecondDays`, `Reminder.DefaultOverdueFrequency`, see §11.3).
2. Insert the task with `Status = ToDo` (there is no initial-status field) and `CreatedByUserId` = caller; then set `TaskCode` = `TSK-` + id padded to 6 digits (e.g. `TSK-000042`).
3. Add one `TaskRecipient` per user in {assignee} ∪ additional recipients (BR-5, BR-6).
4. Append `TaskActivityLog` entries `Created` and `Assigned`.
5. Publish `TaskAssigned` to the assignee ("A task was assigned to you: {title}"), inside the transaction.

The reminder engine picks the task up on its next pass (§11); nothing is pre-scheduled.

10.2 Status transitions (Screens 9, 10)

`/api/v1/tasks/{id}/status` (PATCH, `tasks.status.change` + `TaskAccess`) → `TaskService.ChangeStatusAsync`, rules in `TaskStatusTransitions.Can` (diagram in Appendix A):

From	Allowed to
`ToDo`	`InProgress`, `Blocked`, `Cancelled`
`InProgress`	`InQA`, `Blocked`, `Completed`, `Cancelled`
`InQA`	`InProgress`, `Completed`, `Blocked`, `Cancelled`
`Blocked`	`InProgress`, `Cancelled`
`Completed`, `Cancelled`	none (terminal)

Repeating the current status is always allowed. Any other move → `400` "Cannot move a task from {from} to {to}." On success: `Status`, `UpdatedAt`, `CompletedAt` (Completed) or `CancelledAt` (Cancelled) are set; `StatusChanged` is

logged, plus `Completed` on completion. Completed and Cancelled tasks receive no further reminders (BR-10, §11.7). No notification is published for a status change.

10.3 Editing (Screen 9)

`` `/api/v1/tasks/{id}` `` (PUT, `tasks.update` + task scope) → `TaskService.UpdateAsync` overwrites `Title`, `Description`, `Priority`, `StartDate`, `Deadline`, `EstimatedHours` (same validations as create) and optionally `ActualHours`. Changed fields write `DeadlineChanged`, `PriorityChanged` and `HoursLogged` entries (old/new values). A changed deadline is picked up by the reminder engine on its next pass (§11.7). `UpdateTaskRequest` has no reminder fields, so a task's reminder intervals cannot be changed after creation.

10.4 Worked hours

Action	Rule
Log hours (`` `/api/v1/tasks/{id}/hours` ``, POST)	0 < hours ≤ 24 per entry; assignee or `tasks.update` holder; `ActualHours += hours`; `HoursLogged` entry with old/new totals. `ActualHours` is a concurrency token → `409` "The task's worked hours changed while you were saving. Please try again."
Correct / delete an entry (`` `/api/v1/tasks/{id}/activity/{entryId}/hours` ``, PUT/DELETE)	Administrator role only. Entries that did not add hours can only be deleted. All `HoursLogged` entries are replayed from the first baseline (clamped at 0), `ActualHours` is set to the new total and an `Administrative` entry records the correction.
Side effects	`NotifyHoursChangedAsync` (best effort): `System` notification and an internal mail "Worked hours changed: {title}" to the assignee and recipients.

10.5 Reassignment (Screen 15)

`` `/api/v1/tasks/{id}/reassign` `` (POST, `tasks.reassign` + task scope) → `TaskService.ReassignAsync`:

1. Reason required, ≤ 500 characters (FR-2, FR-3): "A reassignment reason is required."
2. New assignee must be live and active: "The target employee does not exist or is not active."
3. Set `AssignedToUserId`; log `Reassigned` "Reassigned from {old} to {new}. Reason: {reason}" with actor and time (FR-3, FR-4). Comments and history are preserved (FR-6, BR-11).
4. Add the new assignee as a recipient if missing (BR-5, FR-8); the previous assignee **stays** a recipient.
5. Best effort: `Reassigned` notification to new assignee, old assignee, recipients and actor (FR-7), and an internal mail "Task reassigned: {title}".

10.6 Recipients

`` `/api/v1/tasks/{id}/recipients` `` (GET, `TaskAccess`) lists recipients. PUT (`tasks.recipients.manage`) sets the list: requested ids ∪ {assignee}; the assignee can never be removed. Logs `Assigned` "Task recipients updated". Validator: list required, ≤ 50, ids > 0.

10.7 Comments (Screen 11)

`TaskCommentService`; all endpoints also pass `TaskAccess`.

Action	Endpoint and key	Rules
List	`` `/api/v1/tasks/{id}/comments` `` (GET)	Live comments, oldest first, with author name
Add	same route (POST), `tasks.comment.create`	`Body` required, ≤ 4000 (FR-4); task must be live; logs `CommentAdded` with an excerpt (FR-7)

Action	Endpoint and key	Rules
Edit	`` `/api/v1/tasks/{id}/comments/{commentId}` `` (PUT), `tasks.comment.edit`	Author only ("Only the author can edit this comment."); sets `EditedAt`; logs `CommentEdited` with old/new excerpts
Delete	same route (DELETE), `tasks.comment.delete`	Author only; soft delete (`IsDeleted`); logs `CommentDeleted` with the old excerpt

Every add/edit/delete notifies the recipients, assignee and actor (live users) with a `TaskComment` notification and, when `Notification.MailEnabled` is on (or unreadable), a system mail "Comment {verb}: {title}". Earlier text is preserved in the activity history (FR-5).

10.8 Activity history (Screen 18)

`TaskActivityService.AppendAsync` writes `TaskActivityLog` rows (actor, event, old/new value, message, UTC time); the caller owns the transaction. `` `/api/v1/tasks/{id}/activity` `` (GET, `tasks.activity.view` + `TaskAccess`) reads them with filters. History cannot be edited by users; the only exception is the Administrator worked-hours correction, which is itself logged as `Administrative`.

Action	`TaskActivityEventType`
Create	`Created`, `Assigned`
Edit	`DeadlineChanged`, `PriorityChanged`, `HoursLogged`
Log hours / correct hours	`HoursLogged` / `Administrative`
Status change	`StatusChanged`, `Completed`
Reassign / set recipients	`Reassigned` / `Assigned`
Comments	`CommentAdded`, `CommentEdited`, `CommentDeleted`
Delete	`Deleted`

`TimeLogged` exists in the enum but is not written by the current services.

10.9 Display stage and overdue derivation

`TaskStageCalculator.Calculate(status, deadline, now, appTimeZone)`: Completed or Cancelled → `Completed`; otherwise the deadline's calendar day is compared with today in the application time zone → `Overdue`, `DueToday` or `Upcoming` (Screen 9 FR-8). The task list sets `IsOverdue` from the stage; the board marks a card overdue when `Deadline` < UTC now and the task is open. The time zone is `Application.Timezone` (setting) → `Application:Timezone` (config, `Asia/Kolkata`) → UTC (`AppClock`).

10.10 Delete and referential safety (BR-18)

Entity	Rule
`Task`	`` `/api/v1/tasks/{id}` `` (DELETE, `tasks.delete`): soft delete (`IsDeleted`), `Deleted` entry logged; comments, history and reminder history are kept.
`User`	Soft delete (`IsDeleted`, `IsActive = false`). Refused for one's own account, for the last active Administrator, and while the user has unfinished assigned tasks ("… still has n unfinished task(s). Reassign …"). Manager links are cleared and sessions ended.
`Department`	Refused while it has members ("Department cannot be deleted because it still has members. Remove them first."); otherwise manager links removed and soft-deleted.

Entity	Rule
`Project`	Allocations and `ManagerProject` links are removed, then soft-deleted; tasks of the project are not changed.

Soft-deleted rows are hidden by global query filters (see §7).

10.11 Employees (Screen 5)

`EmployeeService`:

- Email is trimmed and lower-cased (`EmailPolicy`); it is unique by **local part**, and deleted users keep their address reserved ("Email already exists."). Username is unique case-insensitively, deleted users included.
- Role must exist; role rules in §9.9. Creating sets a new `SecurityStamp` and hashes the password.
- Project allocations and department memberships can be set on create/update (`UserAllocationService`, §10.13); `null` leaves a list unchanged, an empty list clears it.
- Moving a user out of the Manager role clears their `ManagerProject`, `DepartmentManager` links and `IsProjectManager` flags.
- Status change (`` `/api/v1/users/{id}/status` ``, PATCH, `employees.status.change`): deactivation ends sessions; the last active Administrator cannot be deactivated.

10.12 Departments (Screen 6)

`DepartmentService`: name trimmed and unique ("Department name already exists."). Members (`` `/api/v1/departments/{id}/members` ``, PUT, `departments.members.manage`) are two lists: managers must be active users with the Manager system role ("User {id} is not an active manager."), employees must be active users. The service diffs `DepartmentManager` and `UserDepartment` rows for that department only.

10.13 Projects, positions and allocations

Rule	Implementation
Project name trimmed and unique ("Project name already exists.").	`ProjectService`
Position name required, ≤ 100 characters, spaces collapsed; an existing name (case-insensitive) is reused, otherwise created (`` `/api/v1/positions` ``, POST, `positions.create`).	`PositionService.GetOrCreateAsync`
Each allocation 1–100 %, one row per user per project, position must exist.	`UserAllocationService`
A person's total allocation across all projects ≤ 100 % (`MaxTotalPercent`), checked inside a transaction so two concurrent saves cannot both pass ("{name} would be allocated {total}% in total …").	`UserAllocationService.ReplaceProjectMembersAsync`, `ValidateForUserAsync`
A non-empty project team needs at least one member flagged as project manager; only users with the Manager role can be flagged. `ManagerProject` links are synced to the flagged managers.	`UserAllocationService.ReplaceProjectMembersAsync` (`` `/api/v1/projects/{id}/members` ``, PUT, `projects.members.manage`)
Members must exist and be active.	same

10.14 Profile (Screen 19)

`` `/api/v1/users/me` `` (GET/PUT, `[Authorize]`, Always On keys) → `ProfileService`:

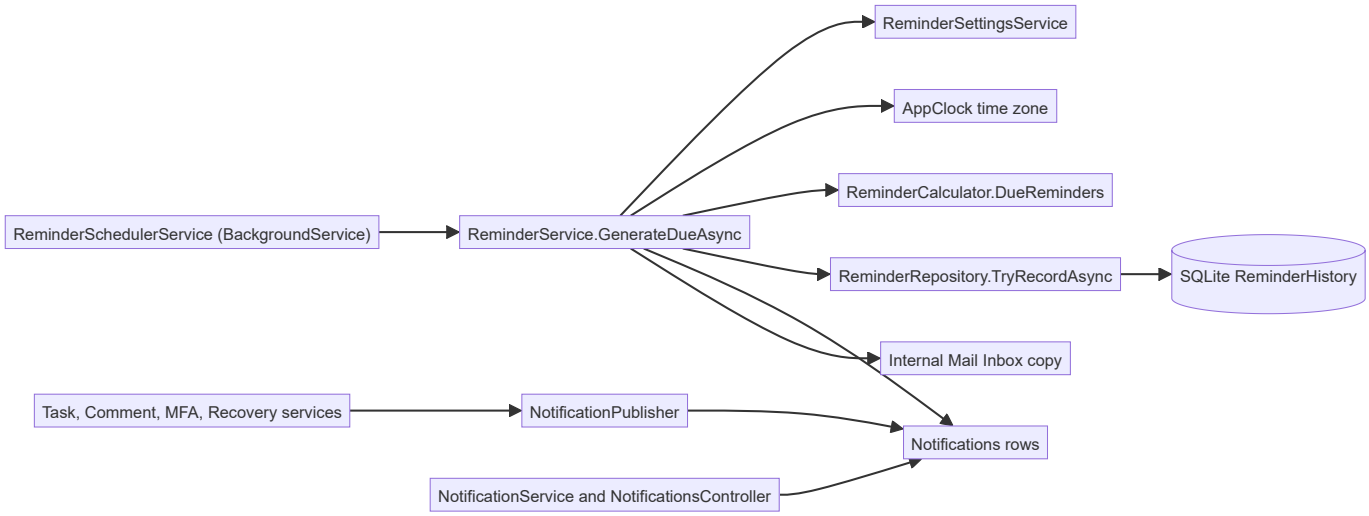
- Editable: `FullName`, `Email`, `MobileNumber` only. Email is normalised and unique by local part; the mobile number is normalised (`MobileNumberPolicy`, E.164) and email + mobile together must be unique.

- Role, project, username, employee code and status are never changed here.
- Password change and MFA management: §9.3, §9.7.

---

11. Notification & Reminder Engine

11.1 Components



Class	Role
`ReminderSchedulerService`	Hosted service; runs reminder passes
`ReminderService`	Selects candidate tasks, computes due reminders, delivers them
`ReminderCalculator`, `ReminderIntervals` (`Common/Rules`)	Pure rules: which reminder is due today; interval ordering
`ReminderRepository`	Candidate query, delivered pre-read, insert-or-ignore into `ReminderHistory`
`NotificationPublisher` (`INotificationPublisher`)	Writes event notifications for other modules
`NotificationService`	Notification Center queries and mark-read
`ReminderSettingsService`	Global reminder settings in `ApplicationSetting` rows

11.2 Scheduler

Item	Value
Type	`ReminderSchedulerService : BackgroundService`, one `PeriodicTimer`
Cadence	One pass at start-up, then every `Reminders:TickSeconds` seconds (default 60, minimum 1)
Switch	`Reminders:SchedulerEnabled` (default `true`; tests switch it off) – when false it logs "Reminder scheduler is disabled by configuration." and exits
Scope	Each pass creates a new DI scope (own `AppDbContext`) and resolves `IReminderService`
Errors	Any exception in a pass is logged ("Reminder pass failed.") and swallowed; the next tick retries
Restart safety	No in-memory state; after a restart the next pass resumes and the unique index prevents duplicates (NFR §5.4)

11.3 Settings and per-task intervals (Screen 13)

Global settings (`/api/v1/settings/reminders` , GET/PUT, `settings.reminders.manage`), stored as `ApplicationSetting` rows:

Setting key	Meaning	Default
`Reminder.Enabled`	Master switch; when false a pass does nothing	`true`
`Reminder.DeadlineDayEnabled`	Send the `DueToday` reminder	`true`
`Reminder.DefaultFirstDays`	Default first reminder, days before the deadline (1–30)	`2`
`Reminder.DefaultSecondDays`	Default second reminder (0–30, 0 = on the deadline day), must be less than the first	`1`
`Reminder.DefaultOverdueFrequency`	`Daily` or `Weekly`	`Daily`
`Notification.MailEnabled`	"Also deliver reminders to Mail" – copy reminders into internal Mail	`true`

All six fields are required on save (`UpdateReminderSettingsRequestValidator`); the time zone is read-only here (owned by Application Settings, Screen 20). Missing or unreadable rows fall back to the defaults in `SettingKeys` . Changes apply without code changes (Screen 13 FR-4).

Each `Task` copies its own `FirstReminderDays` , `SecondReminderDays` (0–30) and `OverdueReminderFrequency` (`ReminderFrequency`) at creation (§10.1); the global values are only defaults for new tasks.

11.4 Reminder calculation (BRD §3.2)

`ReminderCalculator.DueReminders(snapshot, utcNow, timeZone, deadlineDayEnabled)` returns zero or one reminder **for today** (application time zone). `daysUntilDeadline` = deadline's local calendar day − today.

Condition	`ReminderType`
Task `Completed` or `Cancelled`	none (BR-10)
`daysUntilDeadline` = `SecondReminderDays` (> 0)	`SecondReminder`
`daysUntilDeadline` = `FirstReminderDays` (> 0)	`FirstReminder` (equal intervals collapse into one reminder)
`daysUntilDeadline` = 0, now before the deadline instant, and (`Reminder.DeadlineDayEnabled` or a task interval = 0)	`DueToday`
`daysUntilDeadline` < 0 and days overdue divisible by 1 (`Daily`) or 7 (`Weekly`)	`Overdue`

The reminder's slot `ScheduledFor` = UTC instant of local midnight of today (`LocalMidnightUtc` , DST gaps and ambiguous times handled). Only today's slot is ever produced, so a day that has passed is never back-filled (Screen 13 FR-6). Worked example: Appendix B.

BRD event	Implemented as
Task assigned	`TaskAssigned` notification at creation (§10.1), `Reassigned` on reassignment (§10.5)
Two days / one day before deadline	`FirstReminder` / `SecondReminder` with per-task intervals (defaults 2 and 1)
On deadline day	`DueToday`
After deadline	`Overdue` , daily or weekly per task

BRD event	Implemented as
Task completed / cancelled	No reminders (calculator and insert both check the status)
Deadline changed	New deadline used on the next pass (§11.7)

11.5 Candidates and recipients

- Candidates (`ReminderRepository.GetOpenTasksDueByAsync`): not deleted, status not Completed/Cancelled, `Deadline` ≤ now + 32 days (widest interval 30 days plus time-zone slack). There is no lower bound: overdue tasks keep receiving reminders while open.
- Recipients per task: all `TaskRecipient` users (the assignee is always one, BR-5) **plus the task creator**, de-duplicated, restricted to active, non-deleted users.

11.6 Delivery and duplicate prevention (BRD §3.5, BR-14)

1. **Pre-read:** `GetDeliveredAsync` loads today's existing `ReminderHistory` rows so already-delivered reminders are skipped without taking the SQLite write lock.
2. **Per recipient and reminder, one transaction** (`TryDeliverAsync`):
   - `TryRecordAsync` runs `INSERT … SELECT … WHERE EXISTS (task still live, open and with the same Deadline) ON CONFLICT (TaskId, RecipientUserId, ReminderType, ScheduledFor) DO NOTHING`, with `Status = Sent`, `ProcessedAt = now`, `DeliveryAttempts = 0`.
   - Only when a row was inserted: add a `Notification` (`Overdue` for overdue reminders, otherwise `DeadlineReminder`) and, if `Notification.MailEnabled`, a system `MailMessage` (`IsSystem = true`, no sender, `TaskId`, `ReminderHistoryId`) with a `MailRecipient` (To) and an Inbox `MailboxItem`.
   - Save and commit; the change tracker is detached after each delivery.
3. **Guarantees:** the unique index on `ReminderHistory (TaskId, RecipientUserId, ReminderType, ScheduledFor)` makes repeated passes idempotent; the filtered unique index on `MailMessage.ReminderHistoryId` ensures at most one mail per reminder.

Messages (`BuildMessage`, deadline shown in the application time zone):

Type	Notification text	Mail subject prefix
`FirstReminder`, `SecondReminder`	Reminder: "{title}" is due in {n} day(s) ({dd MMM yyyy, HH:mm}).	`Task reminder:`
`DueToday`	Reminder: "{title}" is due today at {HH:mm}.	`Task due today:`
`Overdue`	Overdue: "{title}" was due on {dd MMM yyyy, HH:mm}.	`Overdue task:`

The mail body repeats the text, adds "Open the task: {Mail:AppBaseUrl}/tasks/{id}" when `Mail:AppBaseUrl` is configured, and ends with "This is an automatic reminder from TaskFlow." Subjects are cut to `MailRules.MaxSubjectLength`.

11.7 Recalculation

No future reminders are pre-generated, so nothing has to be cancelled or rebuilt:

- Each pass reads the task's **current** status, deadline and intervals; a deadline change, reassignment, recipient change, completion or cancellation takes effect on the next pass.
- `TryRecordAsync` re-checks that the task is still open and that its deadline equals the one the pass read, so a task changed during a pass gets no stale reminder.

- `ReminderStatus` values `Pending`, `Failed` and `Skipped` exist but are not written by the current engine (every row is `Sent`).

11.8 Channels

Channel	Behaviour
In-app (`Notification` rows)	Always written for reminders; event notifications go through `NotificationPublisher`, which skips writing while `Notification.InAppEnabled` (Application Settings) is false (missing or unreadable = enabled).
Internal Mail (system messages)	Reminder copies when `Notification.MailEnabled` is on; comment mails under the same setting; reassignment and worked-hours mails are always sent (best effort). Mail design: §15.
Email / SMTP (BRD Screen 14)	**Superseded** – no SMTP delivery exists; internal Mail replaces it (see §23).

The SPA picks up new notifications on navigation and through the shell's 15-second refresh; there is no server push.

11.9 Event notifications (`NotificationType`)

`NotificationType`	Written by	Recipients
`TaskAssigned`	`TaskService.CreateAsync`	Assignee
`Reassigned`	`TaskService.ReassignAsync`	New and old assignee, recipients, actor
`DeadlineReminder`	`ReminderService` (`FirstReminder`, `SecondReminder`, `DueToday`)	Recipients + creator
`Overdue`	`ReminderService` (`Overdue`)	Recipients + creator
`TaskComment`	`TaskCommentService` (add, edit, delete)	Recipients, assignee, actor
`System`	`TaskService` (worked hours), `MfaService.ResetAsync`, `PasswordRecoveryService.ResetPasswordAsync`	Assignee and recipients / the affected user

11.10 Notification Center (Screen 12)

Endpoint	Key	Behaviour
`` `/api/v1/notifications` `` (GET)	`notifications.view`	Caller's own notifications, filter by type and read state, paged (page ≥ 1, size 1–100, else 20)
`` `/api/v1/notifications/unread-count` `` (GET)	`notifications.view`	Unread badge count (index `(UserId, IsRead)`)
`` `/api/v1/notifications/{id}/read` `` (POST)	`notifications.manage`	Marks one own notification read; idempotent; another user's id → `404`
`` `/api/v1/notifications/read-all` `` (POST)	`notifications.manage`	Marks all own unread notifications read

Notifications are never deleted; no read time is stored.

11.11 Reminder history (Screen 13 FR-9)

`ReminderHistory` keeps task, recipient, `ReminderType`, `ScheduledFor` slot, `Status`, `ProcessedAt` and `DeliveryAttempts` for every reminder delivered; together with the linked system mail (`MailMessage.ReminderHistoryId`) it is the audit trail for reminders. Rows reference the task with `Restrict` delete; tasks are only soft-deleted.

11.12 Known gaps

Gap	Note
Reminder notifications bypass the in-app switch	`ReminderService` adds `Notification` rows directly, so `Notification.InAppEnabled` = false does not stop reminder notifications.
Recipients include the creator	BR-5 names task recipients only; the code adds the creator (see §23).
No retry/failure tracking	`DeliveryAttempts` stays 0 and `Status` is always `Sent`.
Frontend `TaskComment` type	The SPA notification model lists only five types, so `TaskComment` has no label or filter option (see §12).
No status-change notification	Help text mentions status-change notifications, but none is published.

---

12. Frontend Design

The SPA is an Angular 19.2 standalone application (TypeScript 5.7, no NgModules). Every screen is a lazily loaded standalone component; every module keeps its own route table in `modules/<module>/<module>.routes.ts`, and the root table `routes/app.routes.ts` loads them with `loadChildren`.

12.1 Bootstrap and core infrastructure

Element	File	Responsibility
`appConfig`	`config/app.config.ts`	`provideZoneChangeDetection({ eventCoalescing: true })`, `provideRouter(routes)`, `provideHttpClient(withInterceptors([apiBaseUrlInterceptor, authInterceptor]))`, and `provideAppInitializer` that runs `AuthService.restoreSession()` before the first route is resolved
`apiBaseUrlInterceptor`	`config/api-base-url.interceptor.ts`	When `API_BASE_URL` is set, prefixes every `/api/...` request with it and sends it `withCredentials` (refresh cookie). Empty by default: the SPA calls `/api/*` on its own origin through SpaProxy
`authInterceptor`	`shared/auth/auth.interceptor.ts`	Adds `Authorization: Bearer`; handles 403 `mfa_setup_required` and 401 → refresh → retry (see §12.4)
`AuthService`	`shared/auth/auth.service.ts`	Login, MFA verify, refresh (one shared in-flight refresh through `shareReplay`), logout, password recovery, `restoreSession`, `checkSession` (`GET /api/v1/auth/me`), `currentUser` signal, `hasPermission(key)`, permission re-sync from `/auth/me`
`TokenStore`	`shared/auth/token.store.ts`	Holds the access token in a signal, in memory only (never in `localStorage` / `sessionStorage`)

Element	File	Responsibility
`Permissions`	`shared/auth/permissions.ts`	Permission key constants that mirror `PermissionCatalog.Stored`; a backend test fails when the two lists differ. Also `EmployeeScreenPermissions` and `DepartmentScreenPermissions` (any one action key opens the screen)
`dashboardRouteFor`	`shared/auth/role-routes.ts`	Landing route: Administrator → `/admin`, Manager → `/manager`; any other role → `/admin` if it holds `dashboard.view.organization`, `/manager` if it holds `dashboard.view.team`, else `/me`
Login reasons	`shared/auth/login-reasons.ts`	`reason` query values for the login notice: `session-ended`, `credentials-changed`, `password-reset`

12.2 Shell and navigation

Element	File	Behaviour
`ShellComponent`	`layouts/shell.component.ts`	Authenticated frame (header, side menu, account area, routed content, `ToastHostComponent`, `LegalFooterComponent`, `ThemeMenuComponent`). Mounts `TfaPanelComponent` only when `canUseAssistant()` (user holds `assistant.chat`)
Navigation	`layouts/nav-items.ts` → `buildNavigation(user)`	Menu built from the user's permissions, so a custom role sees exactly the screens it may open (table below)
Session check	`ShellComponent`	`checkSession()` every `SESSION_CHECK_INTERVAL_MS` (15 s) and when the tab becomes visible; a 401 is handled by the interceptor
Badges	`ShellComponent`	Notification unread count (`notifications.view`) and Mail unread Inbox count (`mail.read`) refreshed on navigation and on the same 15 s cadence; a role without the key never calls the endpoint
Brand name	`ApplicationSettingsService.appName`	Loaded from `GET /api/v1/settings/application`; falls back to "TaskFlow"
Layout	`layouts/layout.service.ts`	`WIDE_VIEWPORT_QUERY = (min-width: 992px)`: collapsible rail on wide screens, drawer on narrow screens; collapsed state in `localStorage` `tf.sidebar.collapsed`
Themes	`shared/services/theme.service.ts`	`ThemeName` = `default`, `professional`, `dark`; stored in `localStorage` `tf.theme`

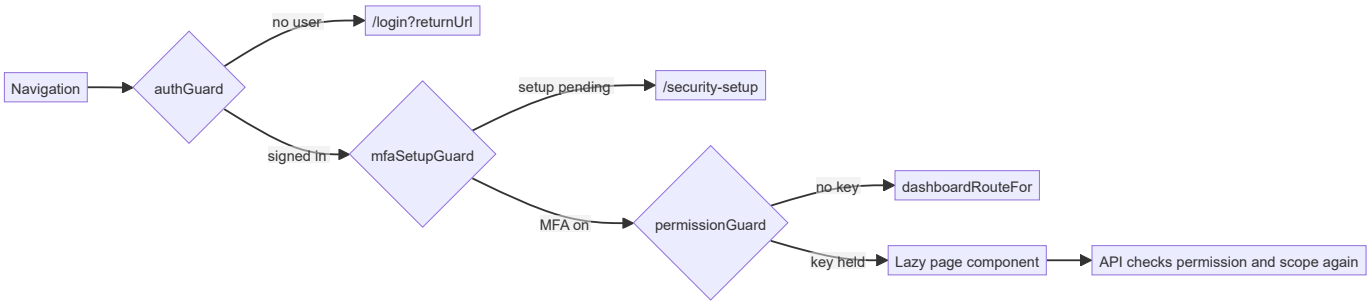
Menu built by `buildNavigation`

Menu entry	Route	Shown when the user holds
Dashboard	`dashboardRouteFor(role, permissions)`	always
TaskFlow Monitoring → Task Management / Employee Workload / Board	`/tasks`, `/workload`, `/board`	`tasks.view.all` (Workload needs `workload.view`, Board needs `board.view`)
Task Management → My Tasks / Employee Workload / Board	`/tasks`, `/workload`, `/board`	no `tasks.view.all` (same sub-entry rules)

Menu entry	Route	Shown when the user holds
TaskFlow Management → Employee Management	`/employees`	any key in `EmployeeScreenPermissions`
TaskFlow Management → Project Management	`/projects`	`projects.view`
TaskFlow Management → Department Management	`/departments`	any key in `DepartmentScreenPermissions`
Communication Management → Mail (with unread badge)	`/mail`	`mail.read`
Communication Management → Notifications	`/notifications`	`notifications.view`
Reports	`/reports`	`reports.view`
Settings → Reminder Settings	`/settings/reminders`	`settings.reminders.manage`
Settings → Application Settings	`/settings/application`	`settings.application.manage`
Settings → TFA Settings	`/settings/tfa`	`settings.ai.view`
Settings → Role Configuration	`/settings/permissions`	`roles.manage`
My Profile	`/profile`	always

12.3 Guards

Guard	File	Rule
`authGuard`	`shared/auth/auth.guard.ts`	Signed-in user passes; otherwise `/login?returnUrl=…` (the login screen uses `returnUrl` only when it is an internal path)
`mfaSetupGuard`	same	On the shell route: a user with `mfaSetupRequired` is sent to `/security-setup`
`securitySetupGuard`	same	On `/security-setup`: only users who still need setup; others go to `dashboardRouteFor(...)`
`permissionGuard`	same	Allows the route when the user holds **at least one** key in `route.data.permissions`; otherwise sends the user to their dashboard. It only hides screens: the API checks every request again



12.4 HTTP error handling in `authInterceptor`

Case	Handling
Request to `/auth/login` or `/auth/refresh`	No bearer token added; errors passed through

Case	Handling
403 with problem `code = mfa_setup_required` (from `MfaSetupGateMiddleware`)	`AuthService.markMfaRequired()` and navigate to `/security-setup`
401 on any other request	`AuthService.refresh()` (cookie `taskflow_refresh`), then the original request is retried once with the new token
Refresh fails	`clearSession()` and navigate to `/login?reason=session-ended`
Other errors	Passed to the caller; screens show the problem `detail` through `shared/utils/api-error.ts` (`apiErrorMessage`)

12.5 Permission checks inside screens

There is no permission directive (`shared/directives/` holds only `EmailLocalPartDirective`). The UI hides and shows elements in three layers:

Layer	Mechanism	Example
Menu	`buildNavigation(user)` adds an entry only when `user.permissions.includes(key)` (§12.2)	Mail entry only with `mail.read`
Route	`permissionGuard` with `data.permissions` (§12.3, §12.8)	`/tasks/new` needs `tasks.create`
Element	`computed()` signals in the component built from `AuthService.hasPermission(key)` / `currentUser().permissions`, used in `@if` blocks	`canEdit = computed(() => this.can(Permissions.DepartmentsEdit))` in `DepartmentListComponent`; `canCompose()` / `canOrganize()` in `MailReaderComponent` and `MailListComponent`; `canEdit`, `canTest`, `mayRemoveModel` in `AiProviderSettingsComponent`; `canEdit` in `TaskDetailsComponent` (combined with "assignee is the current user"); the **Reset MFA** button with `employees.mfa.reset`; the shell's `canUseAssistant()`, `canSeeMail()`, `canSeeNotifications()`

None of these is a security boundary: the API re-checks every request (§9, §14). Role and permission changes reach the SPA through the `/auth/me` permission re-sync on the 15 s session check, and the API applies them on the next request (§9).

12.6 State, UI libraries and shared parts

Concern	Implementation
State	Angular signals for feature and session state (`currentUser`, `unreadCount`, assistant feed); RxJS for HTTP, timers and streams
Charts	Chart.js 4 through ng2-charts 8: `donut-chart`, `line-chart`, `stacked-bar-chart`, `kpi-card`, `progress-list` in `modules/dashboard/components/charts/`; providers in `modules/dashboard/utils/chart-providers.ts`
Drag and drop	Angular CDK drag-drop on the Kanban board (`modules/monitoring/board/pages/kanban.component`)
QR code	`qrcode` package renders the authenticator key on `SecuritySetupComponent`

Concern	Implementation
Shared components	`ConfirmDialogComponent`, `DrawerComponent`, `IconComponent`, `ToastHostComponent` + `ToastService`, `AppBackdropComponent`, `LegalFooterComponent`; directive `EmailLocalPartDirective` (fixed `@taskflow.com` suffix)
Shared utils	`shared/utils/` (`api-error`, `save-blob`, `local-day`, `drill-down`, `analytics-filter`, `reminder-options`, `role-badge`, `mobile-number`, `email-domain`, …)
Browser storage	Non-secret preferences only: `tf.theme`, `tf.sidebar.collapsed`, `taskflow.lastIdentifier` (remembered sign-in name), `tfa.provider`, `tfa.cloudAck`, `tfa.handsfree`
Validation	Client-side validation mirrors, never replaces, the server's FluentValidation; server field errors are mapped back onto form fields

12.7 Folder structure

`src/app/` contains `config/`, `routes/`, `layouts/`, `shared/` (`auth`, `components`, `directives`, `services`, `utils`, `testing`) and `modules/<module>/<sub-module>/{pages,components,services,models,utils}` for the modules `auth`, `dashboard`, `management`, `monitoring`, `communication`, `reports`, `profile`, `settings`, `assistant` and `legal`. The layout is enforced by `npm run check:structure` (`scripts/check-frontend-structure.mjs`), which runs before `ng test` (§5, §20).

12.8 Routing (screen → route → component → guard)

All routes except Login, Account Recovery, Security Setup and Legal are children of the shell route (`authGuard` + `mfaSetupGuard`).

Screen	Route	Component	Guard / required permission
1 Login	`/login`	`LoginComponent`	public
Account Recovery (added)	`/account-recovery`	`AccountRecoveryComponent`	public
Security Setup (added)	`/security-setup`	`SecuritySetupComponent`	`authGuard`, `securitySetupGuard`
2 Admin Dashboard	`/admin`	`AdminDashboardComponent`	`permissionGuard`: `dashboard.view.organization`
3 Manager Dashboard	`/manager`	`ManagerDashboardComponent`	`permissionGuard`: `dashboard.view.team`
4 Employee Dashboard	`/me`	`EmployeeDashboardComponent`	shell only (every signed-in user)
5 Employee Management	`/employees`	`EmployeeListComponent`	`permissionGuard`: any of `EmployeeScreenPermissions`
6 Department Management	`/departments`	`DepartmentListComponent`	`permissionGuard`: any of `DepartmentScreenPermissions`

Screen	Route	Component	Guard / required permission
Project Management (added)	`/projects`, `/projects/:id`	`ProjectListComponent`, `ProjectDetailComponent`	`permissionGuard: projects.view`
7 Task List	`/tasks`	`TaskListComponent`	shell only (API scopes the list)
8 Create Task	`/tasks/new`	`TaskCreateComponent`	`permissionGuard: tasks.create`
9 Task Details	`/tasks/:id`	`TaskDetailsComponent`	shell only (API `TaskAccess` check)
10 Kanban Board	`/board`	`KanbanComponent`	`permissionGuard: board.view`
11 Task Update & Comments	`/tasks/:id/discussion`	`TaskDiscussionComponent`	shell only (API `TaskAccess` + comment keys)
12 Notification Center	`/notifications`	`NotificationCenterComponent`	`permissionGuard: notifications.view`
13 Deadline Reminder Settings	`/settings/reminders`	`ReminderSettingsComponent`	`permissionGuard: settings.reminders.manage`
14 Email Notification Settings	– (superseded)	–	replaced by internal Mail (§15) and the "Also deliver reminders to Mail" toggle on Screen 13
15 Task Reassignment	`/tasks/:id/reassign`	`TaskReassignComponent`	`permissionGuard: tasks.reassign`
16 Employee Workload	`/workload`	`WorkloadComponent`	`permissionGuard: workload.view`
17 Reports	`/reports`	`ReportComponent`	`permissionGuard: reports.view`
18 Task Activity History	`/tasks/:id/activity`	`ActivityComponent`	`permissionGuard: tasks.activity.view`
19 User Profile	`/profile`, `/profile/change-password`	`ProfileComponent`, `ChangePasswordComponent`	shell only
20 Application Settings	`/settings/application`	`ApplicationSettingsComponent`	`permissionGuard: settings.application.manage`
Mail (added)	`/mail`	`MailComponent`	`permissionGuard: mail.read`
TFA Settings (added)	`/settings/tfa`	`TfaSettingsComponent`	`permissionGuard: settings.ai.view`
Role Configuration (added)	`/settings/permissions`	`PermissionsManagementComponent`	`permissionGuard: roles.manage`

Screen	Route	Component	Guard / required permission
TFA Assistant panel (added)	no route	`TfaPanelComponent` (in the shell)	rendered only with `assistant.chat`
Legal pages (added)	`/legal/copyright`, `/legal/terms`, `/legal/privacy`, `/legal/disclaimer`, `/legal/licenses`	`LegalPageComponent` (`data.documentId`)	public
Fallback	`/` → `/login`; `**` → `/login`	–	–

> ⚠️ To be confirmed: the source files of `TaskDiscussionComponent` (`task-management/discussion/pages/`) and `ActivityComponent` (`task-management/activity/pages/`) are referenced by `monitoring.routes.ts` but were not in the reviewed copy of the frontend; their names are taken from the route table.

---

13. Screen-to-Implementation Traceability

Frontend services are under `modules/<module>/.../services/`; backend services under `Services/<Module>/`. All routes are under `/api/v1` (§8). The requirements, functional behaviour and detailed design of each screen are in the sub-module BRD, FRD, HLD and LLD named in §1.6 (the "#" column below is the original BRD screen number used in that table).

13.1 BRD screens

#	Screen	Frontend route / service	Backend controller / services	Key API endpoints
1	Login	`/login`; `AuthService`	`AuthController`; `AuthService`, `MfaService`, `SessionService`	`/api/v1/auth/login`, `/api/v1/auth/mfa/verify`, `/api/v1/auth/refresh`, `/api/v1/auth/logout`, `/api/v1/auth/me`
2	Admin Dashboard	`/admin`; `DashboardAnalyticsService`	`AnalyticsController`; `AdminAnalyticsService`, `AdminDashboardScope`, `AnalyticsFilterResolver`	`/api/v1/dashboard/admin/analytics`, `/api/v1/dashboard/admin/analytics/filters` (`/api/v1/dashboard/admin` via `DashboardService` exists but is not called by the SPA)
3	Manager Dashboard	`/manager`; `ManagerAnalyticsService`	`AnalyticsController`; `ManagerAnalyticsService`, `ManagerDashboardScope`	`/api/v1/dashboard/manager/analytics`, `/api/v1/dashboard/manager/analytics/filters` (`/api/v1/dashboard/manager` not called by the SPA)
4	Employee Dashboard	`/me`; `EmployeeAnalyticsService`	`AnalyticsController`; `EmployeeAnalyticsService`, `EmployeeDashboardScope`	`/api/v1/dashboard/employee/analytics` (`/api/v1/dashboard/employee` not called by the SPA)
5	Employee Management	`/employees`; `UserService`	`EmployeesController`, `PositionsController`; `EmployeeService`, `UserAllocationService`, `PositionService`, `MfaService`	`/api/v1/users`, `/api/v1/users/{id}`, `/api/v1/users/{id}/status`, `/api/v1/users/{id}/mfa/reset`, `/api/v1/users/roles`, `/api/v1/positions`

#	Screen	Frontend route / service	Backend controller / services	Key API endpoints
6	Department Management	`/departments`; `DepartmentService`	`DepartmentsController`; `DepartmentService`	`/api/v1/departments`, `/api/v1/departments/{id}`, `/api/v1/departments/{id}/members`
7	Task List	`/tasks`; `TaskService`	`TasksController`; `TaskService`, `TaskAccessService`	`GET /api/v1/tasks`, `DELETE /api/v1/tasks/{id}`
8	Create Task	`/tasks/new`; `TaskService`	`TasksController`; `TaskService`, `NotificationPublisher`	`POST /api/v1/tasks`, `GET /api/v1/tasks/assignees`
9	Task Details	`/tasks/:id`; `TaskService`	`TasksController`; `TaskService`, `TaskStatusTransitions`, `TaskAccessService`	`GET` / `PUT /api/v1/tasks/{id}`, `PATCH /api/v1/tasks/{id}/status`, `POST /api/v1/tasks/{id}/hours`, `GET` / `PUT /api/v1/tasks/{id}/recipients`
10	Kanban Board	`/board`; `TaskService`	`TasksController`; `TaskService`, `TaskStatusTransitions`	`GET /api/v1/tasks/board`, `PATCH /api/v1/tasks/{id}/status`
11	Task Update & Comments	`/tasks/:id/discussion`; `TaskService`	`TaskCommentsController`; `TaskCommentService`	`GET` / `POST /api/v1/tasks/{id}/comments`, `PUT` / `DELETE /api/v1/tasks/{id}/comments/{commentId}`
12	Notification Center	`/notifications`; `NotificationService`	`NotificationsController`; `NotificationService`	`/api/v1/notifications`, `/api/v1/notifications/unread-count`, `/api/v1/notifications/{id}/read`, `/api/v1/notifications/read-all`
13	Deadline Reminder Settings	`/settings/reminders`; `ReminderSettingsService`	`ReminderSettingsController`; `ReminderSettingsService`, `ReminderService`, `ReminderSchedulerService`	`GET` / `PUT /api/v1/settings/reminders`
14	Email Notification Settings	superseded	`MailService` (reminder mails, §15.6)	no SMTP endpoints; delivery to Mail is the `Notification.MailEnabled` setting on Screen 13 (§23)
15	Task Reassignment	`/tasks/:id/reassign`; `TaskService`	`TasksController`; `TaskService`, `MailService`, `NotificationPublisher`	`POST /api/v1/tasks/{id}/reassign`
16	Employee Workload	`/workload`; `WorkloadService`	`WorkloadController`; `WorkloadService`	`GET /api/v1/workload`
17	Reports	`/reports`; `ReportService`	`ReportController`; `ReportService`, `ReportCsv`	`GET /api/v1/reports`, `GET /api/v1/reports/export`

#	Screen	Frontend route / service	Backend controller / services	Key API endpoints
18	Task Activity History	`/tasks/:id/activity`; `TaskService`	`TaskActivityController`; `TaskActivityService`	`GET /api/v1/tasks/{id}/activity`, `PUT` / `DELETE /api/v1/tasks/{id}/activity/{entryId}/hours`
19	User Profile	`/profile`, `/profile/change-password`; `ProfileService`	`ProfileController`; `ProfileService`, `MfaService`	`GET` / `PUT /api/v1/users/me`, `POST /api/v1/users/me/password`, `POST /api/v1/users/me/mfa/check`
20	Application Settings	`/settings/application`; `ApplicationSettingsService`	`ApplicationSettingsController`; `ApplicationSettingsService`, `AppClock`	`GET` / `PUT /api/v1/settings/application`

13.2 Screens added beyond the BRD

Screen	Frontend route / service	Backend controller / services	Key API endpoints
Account Recovery	`/account-recovery`; `AuthService`	`PasswordRecoveryController`; `PasswordRecoveryService`, `AttemptLimiter`	`/api/v1/auth/password-recovery/authenticator/verify`, `/api/v1/auth/password-recovery/reset`
Security Setup (mandatory MFA)	`/security-setup`; `ProfileService`, `AuthService`	`ProfileController`; `MfaService`, `ProfileService`	`GET` / `PUT /api/v1/users/me`, `/api/v1/users/me/mfa/setup`, `/api/v1/users/me/mfa/enable`, `/api/v1/users/me/mfa/backup-codes`
Project Management	`/projects`, `/projects/:id`; `ProjectService`	`ProjectsController`, `PositionsController`; `ProjectService`, `PositionService`	`/api/v1/projects`, `/api/v1/projects/{id}`, `/api/v1/projects/{id}/employees`, `/api/v1/projects/{id}/members`, `/api/v1/projects/{id}/tasks`, `/api/v1/positions`
Mail	`/mail`; `MailService`	`MailController`; `MailService`, `MailAttachmentService`	16 endpoints under `/api/v1/mail` (§15.2)
Role Configuration	`/settings/permissions`; `RoleService`	`RolesController`; `RoleService`	`/api/v1/roles`, `/api/v1/roles/{id}`, `GET /api/v1/permissions`, `GET /api/v1/users/roles`
TFA Settings	`/settings/tfa`; `AiProviderAdminService`	`AiProvidersController`; `AiProviderConfigService`, `AiProviderConnectionTester`	`/api/v1/assistant/providers/config`, `/api/v1/assistant/providers/{kind}/models`, `/api/v1/assistant/providers/{kind}/test`

Screen	Frontend route / service	Backend controller / services	Key API endpoints
TFA Assistant panel	shell; `AssistantService`, `WakePhraseService`, `SpeakService`	`AssistantController`; `AssistantService`, `AssistantActionService`, `AssistantToolbox`	7 endpoints under `/api/v1/assistant` (§16.2)
Legal pages	`/legal/*`; `LegalPageComponent` (static content in `modules/legal/utils/legal-documents.ts`)	none	none

14. Security Design

14.1 BRD NFR controls (BRD §5.2)

Requirement	Design control in the code
Secure password hashing	`IPasswordHasher<User>` → ASP.NET Core `PasswordHasher<User>` (PBKDF2); only `User.PasswordHash` is stored, never returned or logged. New passwords 6–128 characters
Role-based access control	Permission-based: one policy `perm:{key}` per `PermissionCatalog.All` entry (`RequireClaim(AuthClaims.Permission, key)`), applied with `[Authorize(Policy = …)]` / `[RequirePermission]`; system roles Administrator/Manager/Employee are read-only, custom roles configurable (§9)
Server-side input validation	FluentValidation 12.2 validators on write DTOs, run by `ValidationFilter`; errors returned as validation ProblemDetails (`errors` map)
CSRF protection	API uses bearer tokens in the `Authorization` header; the only cookie (`taskflow_refresh`) is `HttpOnly`, `Secure`, `SameSite=Strict`, `Path=/api/v1/auth`
Secure session management	Access token 30 min (`Jwt:AccessTokenMinutes`), held in memory by `TokenStore`; refresh token 7 days (`Jwt:RefreshTokenDays`), rotated on every refresh and stored as SHA-256 hash in `RefreshToken`; `SessionValidation` rejects tokens of inactive/deleted users or with a stale `SecurityStamp` and reloads role and permission claims from the database on every request
Protection against unauthorized record access	Resource check `TaskAccessHandler` (`AuthPolicies.TaskAccess`: assignee, creator, recipient or `tasks.view.all`); scopes Own/Team/Organization in services and repositories; another user's mail, assistant conversation or pending action returns 404, not 403
Safe database queries	EF Core LINQ (parameterised); `LIKE` searches escape `%`, `_`, `\`
Secure handling of email credentials	Not applicable: SMTP was removed (BRD Screen 14 superseded by internal Mail, §15). AI provider API keys are the only third-party secrets (§14.3)

14.2 Authentication hardening (added)

Control	Implementation
Mandatory MFA	`Mfa:Required = true`. `MfaSetupGateMiddleware` (after `UseAuthentication`, before `UseAuthorization`) answers 403 with `code = mfa_setup_required` for every API call except `/api/v1/auth/*`, `/api/v1/users/me/mfa/*` and exactly `/api/v1/users/me`; the SPA guards only mirror it
TOTP	`Totp` (SHA1, 6 digits, 30 s, ±1 step), each step usable once (`User.MfaLastUsedStep`); secret encrypted by `MfaSecretProtector` (Data Protection purpose `TaskFlow.MfaSecrets`) in `User.MfaSecretProtected`
Backup codes	10 codes (`Mfa:BackupCodeCount`), shown once, stored as SHA-256 hashes in `MfaBackupCode`; single use (`UsedAt`); not accepted for password recovery
Sign-in challenge	After a correct password, a Data Protection token (purpose `TaskFlow.MfaChallenge`, 5 min `Mfa:ChallengeMinutes`) bound to user id and security stamp
Code lockout	5 wrong codes (`Mfa:MaxFailedAttempts`) lock code entry for 5 minutes (`Mfa:LockoutMinutes`) using `User.MfaFailedAttempts` / `MfaLockedUntil`
Anti-enumeration	Login returns a bare 401 for every failure; password recovery returns one message for every failure
Attempt limiting	`AttemptLimiter` (in memory, sliding 5 minutes): 5 per account and 20 per client address for recovery (IPv6 pooled by /64); 5 per account for current-password re-entry (`pwd:{id}`); recovery failures never lock the owner's sign-in
Real client address	`UseForwardedHeaders` trusts only loopback proxies unless `Proxy:KnownProxies` lists others, so `X-Forwarded-For` cannot be spoofed
Password reset	`PasswordResetGrant`: 32 random bytes stored as SHA-256 hash, 10 minutes, single use; reset rotates `SecurityStamp` and revokes all refresh tokens
Administrator MFA reset	`POST /api/v1/users/{id}/mfa/reset` (`employees.mfa.reset`) is the only way MFA is removed; the user is signed out everywhere
Email identity	`EmailPolicy`: every email is on the fixed `@taskflow.com` domain; email is the unique identity

14.3 Secrets and data protection

Secret	Handling
JWT signing key (`Jwt:Key`)	From configuration / environment only (`<secret>`); never in responses or logs
AI provider API keys	Encrypted by `AiKeyProtector` (purpose `TaskFlow.AiProviderKeys`) into `AiProviderConfig.EncryptedApiKey`; only `KeyLast4` is ever shown; decrypted in memory for one provider call
Data Protection key ring	Folder `AiProviders:KeyRingPath` (default `dp-keys`) under the data folder (`C:\TaskFlow`); `ProtectKeysWithDpapi()` on Windows, so it decrypts only for the Windows account running the API; `KeyRingProtectionCheck` logs a startup warning about unprotected keys. It protects AI keys, MFA secrets and MFA challenges

Secret	Handling
Seed passwords	`Seed:Enabled` is on only in Development; `Seed:*Password` values are `<secret>`
Logging	`RequestLoggingMiddleware` logs method, path, status, elapsed time, user id and trace id, never bodies; AI provider `HttpClient`s have all loggers removed; `DeniedActionAuditHandler` logs refused authorizations with identifiers only
Chat content	`SecretScreen` blocks messages that look like passwords or keys before any model call (§16.7)

14.4 Transport, CORS and response hardening

Control	Implementation
HTTPS	`UseHttpsRedirection()`; development API on `https://localhost:7055`, SPA on `http://localhost:4200` through SpaProxy
CORS	Policy `CorsPolicies.SpaClient`: origins from `Cors:AllowedOrigins` only, any header/method, credentials allowed (needed for the refresh cookie)
Pipeline order	`UseForwardedHeaders` → `RequestLoggingMiddleware` → `UseExceptionHandler` → `UseHttpsRedirection` → `UseCors` → `UseAuthentication` → `MfaSetupGateMiddleware` → `UseAuthorization` → controllers
Error responses	`GlobalExceptionHandler` returns `application/problem+json`; unhandled errors give a generic 500 message, no stack trace
Downloads	Mail attachments are served as file downloads with a server-chosen content type and `X-Content-Type-Options: nosniff`; upload extension block-list (§15.5)
XSS	Angular template binding; mail bodies are plain text and `linkify` turns only http(s) URLs into links without HTML injection
SSRF	AI provider clients do not follow redirects (`AllowAutoRedirect = false`); the Ollama address validator refuses link-local and cloud-metadata hosts
Development-only endpoints	OpenAPI and `/api/v1/diagnostics/throw` are mapped only when the environment is Development

> ⚠️ To be confirmed: no HSTS (`UseHsts`), Content-Security-Policy, `X-Frame-Options` or `Referrer-Policy` headers are set in `Program.cs`; whether a reverse proxy adds them in deployment is not documented.

14.5 Known security gaps (from the module reviews)

Gap	Effect
TFA `propose_role_change` checks only `roles.manage`, not `employees.role.assign` / `roles.administrator.assign`	A custom role with `roles.manage` could change roles through TFA that `PUT /api/v1/users/{id}` would refuse (§16.5)
`mail.attachments` and draft Discard (`mail.organize`) are not checked in the Mail UI	Buttons visible to users who then get 403 (server is still correct)
`assistant.actions.confirm` is not checked in the TFA action card	Confirm/Edit/Cancel shown, API returns 403
`GET /api/v1/mail/contacts`	Exposes the active-user directory (name, email) to anyone with `mail.compose`

Gap	Effect
In-memory limiters (`AttemptLimiter`, `AssistantRateLimiter`)	Reset on restart; per instance only (single-instance design, §2)

---

15. Internal Mail Design

Internal mail replaces the BRD's SMTP email (Screen 14). It is a mailbox between TaskFlow users only: an address that is not an active TaskFlow user is rejected and nothing leaves TaskFlow. Migration `20261017090431_AddInternalMail`.

15.1 Components

Layer	Implementation
Frontend	`MailComponent` (folder rail, counts, URL state `?folder=&item=`, narrow-screen panes) with `MailListComponent`, `MailReaderComponent`, `ComposeDialogComponent` (drawer: new / reply / forward / draft, uploads), `RecipientPickerComponent`; `MailService` (all calls + `unreadCount` signal for the menu badge); utils `mail-format.ts`, `linkify.ts`
Controller	`MailController` (`Controllers/Communication/`), class-level `[Authorize]` + `perm:mail.*` per action
Services	`MailService` (`Services/Communication/Mail/Messages/`), `MailAttachmentService` + `MailAttachmentDownload` (`.../Attachments/`)
Repositories	`MailRepository` (+ `MailItemHeader`), `MailAttachmentRepository` (+ `AttachmentFile`, `PendingUploads`)
Rules	`Common/Rules/MailRules.cs` (limits, prefixes, file-name sanitising, block-list, safe content types), `EmailPolicy.cs`
Validators	`SendMailRequestValidator`, `SaveDraftRequestValidator`; `MailValidationException` (400 with field `errors`)
File store	`IMailFileStore` → `LocalMailFileStore` (`Integrations/Storage/`)
Config	`MailOptions`: `Mail:AttachmentsPath` (default `mail-attachments`, relative to `Storage:Root`), `Mail:AppBaseUrl` (task links in reminder mails)

15.2 Endpoints and permissions

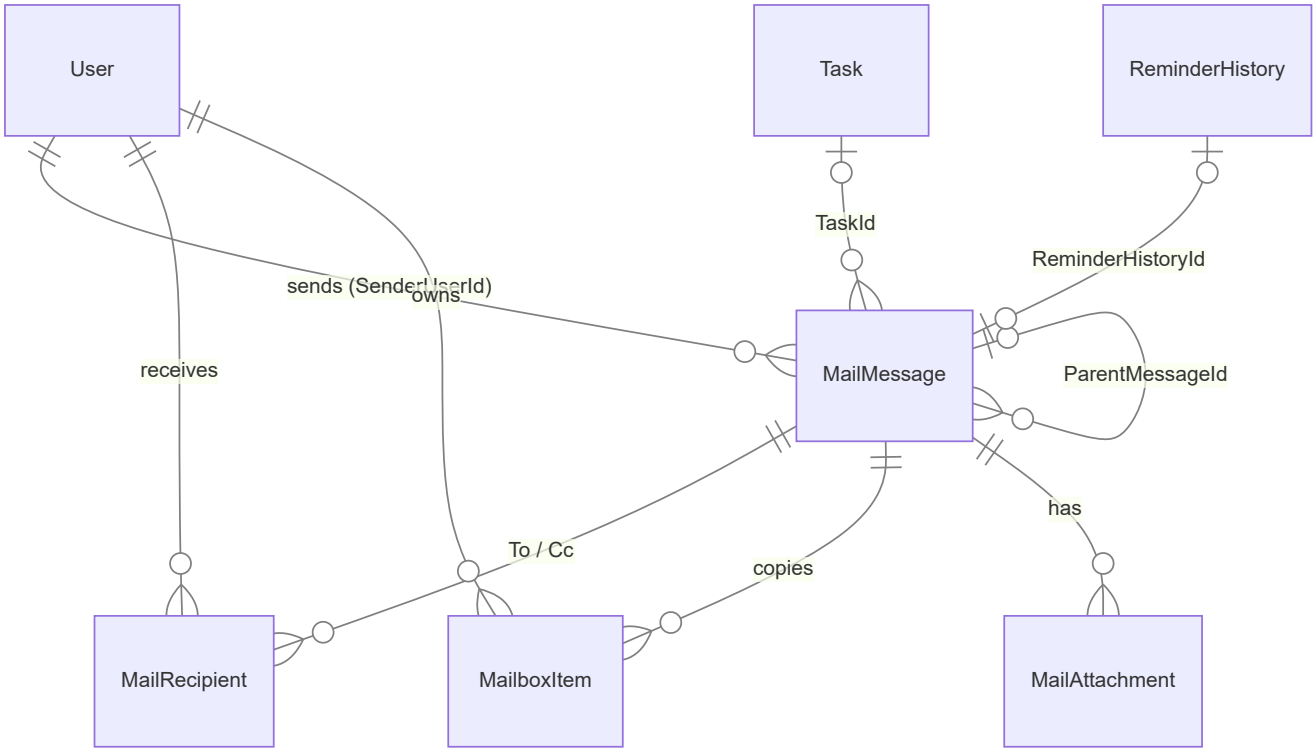
Verb	Route	Permission	Purpose
GET	`/api/v1/mail`	`mail.read`	One page of a view: `folder` (`MailView`), `search`, `page` (default 1), `pageSize` (default 25, max 50)
GET	`/api/v1/mail/summary`	`mail.read`	Rail counts: unread Inbox, Starred, Important; total Drafts
GET	`/api/v1/mail/unread-count`	`mail.read`	Unread Inbox count (menu badge)
GET	`/api/v1/mail/items/{itemId}`	`mail.read`	Open an item with its conversation; marks it read

Verb	Route	Permission	Purpose
PATCH	`/api/v1/mail/items/{itemId}`	`mail.organize`	Set `isRead` / `isStarred` / `isImportant`
POST	`/api/v1/mail/items/{itemId}/trash`	`mail.organize`	Move to Trash (`TrashedAt`)
POST	`/api/v1/mail/items/{itemId}/restore`	`mail.organize`	Restore from Trash
DELETE	`/api/v1/mail/items/{itemId}`	`mail.organize`	Delete forever (trashed item) or discard a draft; 409 otherwise
GET	`/api/v1/mail/items/{itemId}/compose`	`mail.compose`	Prefill for `mode` = `Reply` , `ReplyAll` , `Forward` (`MailComposeMode`)
POST	`/api/v1/mail/send`	`mail.compose`	Send new mail, reply/forward or a saved draft
POST	`/api/v1/mail/drafts`	`mail.compose`	Create a draft
PUT	`/api/v1/mail/drafts/{messageId}`	`mail.compose`	Update a draft (by message id)
GET	`/api/v1/mail/contacts`	`mail.compose`	Up to 10 active users matching name or email
POST	`/api/v1/mail/attachments`	`mail.attachments`	Upload one file (multipart `file` , request limit 11 MB)
GET	`/api/v1/mail/attachments/{attachmentId}`	`mail.attachments`	Download (`nosniff`)
DELETE	`/api/v1/mail/attachments/{attachmentId}`	`mail.attachments`	Remove own upload not yet on a mail

All four `mail.*` keys are granted to Manager and Employee by default and to every existing role when first added (`GrantToAllExistingRoles`), so they can still be switched off per custom role (Appendix C).

15.3 Data model

Each user has a personal copy of a mail (`MailboxItem`), so read state, flags, trash and deletion are personal.



Entity	Key points
`MailMessage`	`ConversationId` (Guid thread key), `ParentMessageId`, `SenderUserId` (null for system mail), `IsSystem`, `Subject` (200), `Body` (20,000, plain text), `TaskId`, `ReminderHistoryId` (unique filtered index: one mail per reminder), `DraftTo` / `DraftCc` (raw draft recipients), `SentAt` (null = draft)
`MailRecipient`	`MessageId`, `UserId`, `Kind` (`MailRecipientKind`: To, Cc); unique `(MessageId, UserId)`
`MailboxItem`	`OwnerUserId`, `Folder` (`MailFolder`: Inbox, Sent, Drafts), `IsRead`, `IsStarred`, `IsImportant`, `TrashedAt`, `PurgedAt`, `SortAt`; unique `(MessageId, OwnerUserId, Folder)`; index `(OwnerUserId, Folder, PurgedAt, TrashedAt, SortAt)`
`MailAttachment`	`MessageId` (null = pending upload), `UploadedByUserId`, `FileName` (255, sanitised), `ContentType` (from the extension allow-list), `SizeBytes`, `StorageKey` (Guid file name, shared by forwarded copies)

15.4 Folders, compose, drafts, reply and forward

Feature	Design
Views (`MailView`)	Inbox, Sent, Drafts = `Folder` and not trashed; Starred / Important = flag, excluding Drafts; Trash = `TrashedAt` set. Every query starts from `MailRepository.Mine(owner)` (`OwnerUserId == caller && PurgedAt == null`); ordered by `SortAt` desc
Search	Escaped `LIKE` on subject, body, `DraftTo`, the "TaskFlow" system sender and sender/recipient name or email (deleted users still match)
Send (`MailService.SendAsync`)	Normalise addresses (trim, lower-case, de-duplicate, drop Cc already in To), 1–50 recipients, all must be active users ("Not an active TaskFlow user: …"); subject or body required; attachments planned and validated before any write; one transaction creates the message, `MailRecipient` rows, a Sent item for the sender (`IsRead = true`) and an Inbox item per recipient, and removes the draft item

Feature	Design
Drafts	`CreateDraftAsync` adds a Drafts item; `UpdateDraftAsync` stores trimmed subject, body, raw `DraftTo` / `DraftCc`, sets `SortAt`; sending a draft by id keeps its saved recipients and attachments
Reply / Reply all / Forward	`GetComposeAsync`: Reply → the sender (or original To when the caller sent it); Reply all → sender + To + Cc minus the caller; Forward → no recipients, original attachments carried; subject prefix `Re: ` / `Fwd: ` (max 200); quoted body ("On {date}, {name} wrote:" or "---------- Forwarded message ----------") in the application timezone, truncated to 20,000. The thread keeps the parent's `ConversationId`
System mail	Cannot be replied to ("Mails from TaskFlow cannot be replied to. Forward it instead."), only forwarded
Trash and delete	Trash sets `TrashedAt`, restore clears it; delete forever only for trashed items or drafts (else 409 `ConflictException`) and is a soft purge (`PurgedAt`)

15.5 Attachments

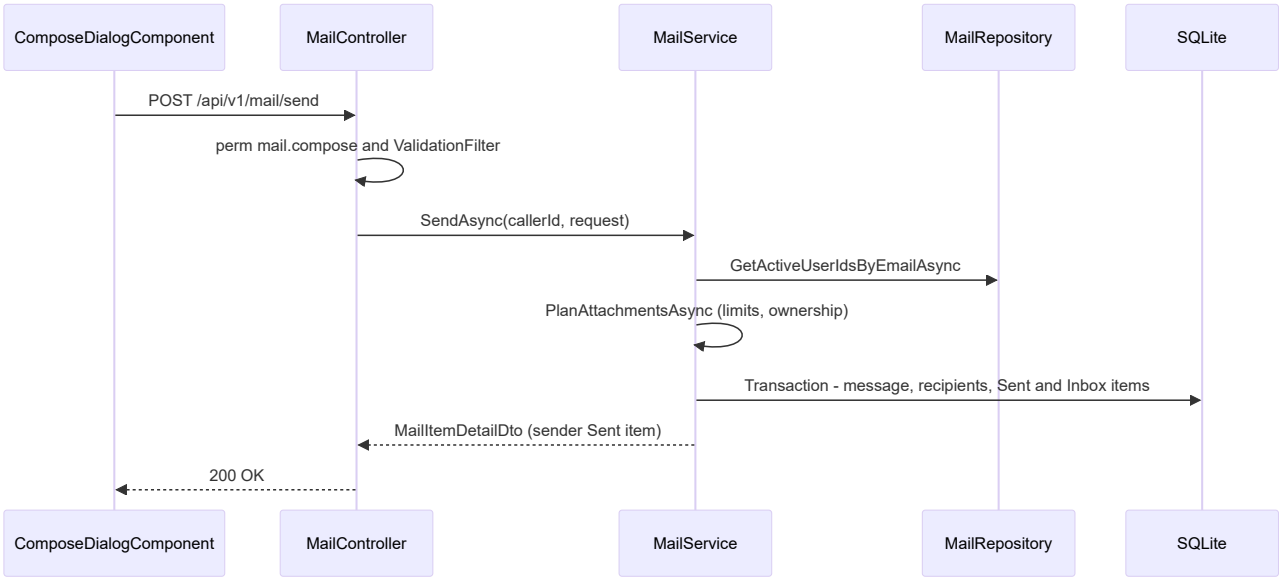
Limit / rule	Value (`MailRules`)
Per file	10 MB (`MaxAttachmentBytes`); empty files refused
Per mail	5 files (`MaxAttachments`), 25 MB total (`MaxTotalAttachmentBytes`), forwarded copies counted
Pending (unattached) uploads per user	20 files (`MaxPendingUploads`), 100 MB (`MaxPendingUploadBytes`)
Blocked extensions	`.exe .dll .bat .cmd .com .msi .ps1 .sh .js .vbs .scr` (`BlockedExtensions`)
File name	`SanitizeFileName`: last path segment, invalid/control characters removed, max 255, fallback "file"
Content type	From the extension allow-list (`SafeTypes`: images, pdf, txt, csv, Office formats, zip), else `application/octet-stream`; never taken from the client
Storage	`LocalMailFileStore` writes each file once to `{Storage:Root}/{Mail:AttachmentsPath}/{StorageKey}` (default `C:\TaskFlow\mail-attachments`); files are never modified; forwarding creates new `MailAttachment` rows with the same `StorageKey`
Download access	The uploader, or a user with a non-purged `MailboxItem` for the message; otherwise 404
Delete upload	Only the caller's own pending upload; removes the row only (file may be shared)

15.6 System mail from other modules

Source	Mail
Deadline reminders (`ReminderService`, §11)	`IsSystem = true`, sender "TaskFlow", linked by `ReminderHistoryId` and `TaskId` (task link under `Mail:AppBaseUrl`); sent when the setting `Notification.MailEnabled` ("Also deliver reminders to Mail" on Reminder Settings) is on. Bell notifications are sent either way
Task comments (`TaskCommentService`)	System mail "Comment {verb}: {title}", same setting
Reassignment (`TaskService`)	Sent through `IMailService.SendAsync` **as the acting user**, subject "Task reassigned: {title}"

Source	Mail
TFA (`DraftMailTool`)	A `SendMail` pending action that the user confirms; then a normal mail from the user's mailbox (§16.6)

15.7 Send sequence



15.8 Privacy and limitations

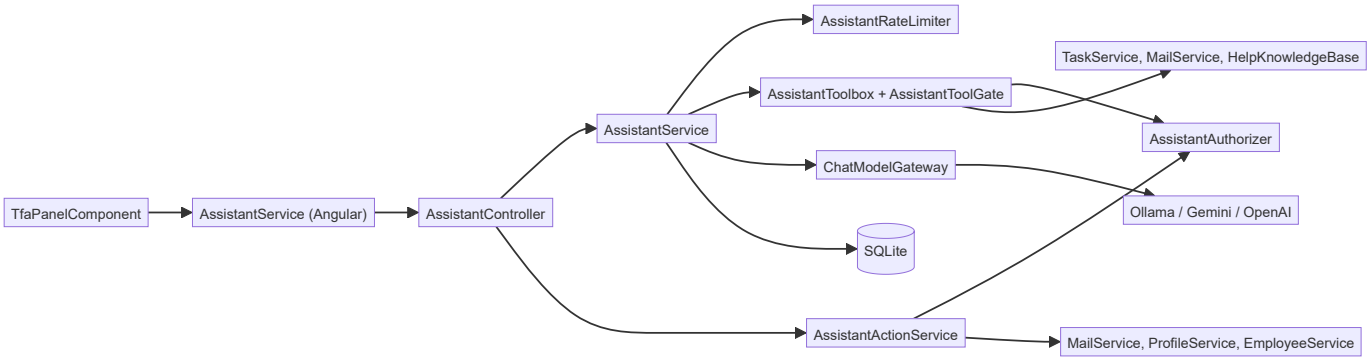
- Data scope is always **Own**: an item, draft or attachment that is not the caller's returns 404, never 403. Administrators have no access to other people's mail. The only directory-wide call is `GET /api/v1/mail/contacts`.
- Delete forever is a soft purge; rows and files are never physically removed and there is no clean-up job for abandoned uploads.
- `mail-attachments` must be backed up together with `taskflow.db` (§19).
- UI gaps: `mail.attachments` and draft Discard (`mail.organize`) are not checked in the UI (§14.5).

---

16. TFA Assistant Design

TFA (TaskFlow Assistant) is a tool-calling chat assistant hosted inside the API. It answers how-to questions from built-in help, reads the user's tasks, ranks priorities, and prepares actions (send mail, update own profile, change a user's role) that run only after the user confirms. Migrations `20261018183210_AddAssistant` and `20261019163241_AddAiModels`.

16.1 Architecture



Part	Classes
Conversation engine	`AssistantService`, `AssistantPrompt`, `AssistantTexts`, `ProviderFailureMessages`, `AssistantCaller` (user id and role from JWT claims only)
Model gateway	`ChatModelGateway` → `ChatModelProviderRegistry` → `OllamaProvider`, `GeminiProvider`, `OpenAiProvider` (`Integrations/Ai`); `AiProviderRuntime`, `ProviderFailure`, `ProviderHttp`
Tools	`AssistantToolbox` + `AssistantToolGate`, `AssistantReadTools`, `AssistantActionTools`, `ToolOutcome`
Actions	`AssistantActionService`, `AssistantActionRules`, `TaskPrioritiser`
Authorization	`IAssistantAuthorizer` → `AssistantAuthorizer` (evaluates the API's own `perm:{key}` policies through `IAuthorizationService`)
Safety	`SecretScreen`, `AssistantRateLimiter`, `AssistantAuditLog`
Help	`HelpKnowledgeBase` (19 embedded markdown topics in `Services/Assistant/Knowledge/`)
Provider configuration	`AiProviderConfigService`, `AiProviderConnectionTester`, `AiKeyProtector` (Settings → TFA Settings)

16.2 Endpoints and permissions

Verb	Route	Permission	Purpose
GET	`/api/v1/assistant/conversation`	`assistant.chat`	Latest conversation, newest 100 messages with action cards
POST	`/api/v1/assistant/conversation/messages`	`assistant.chat`	Send a message; a provider failure returns 200 with a `failure` object
POST	`/api/v1/assistant/conversation/new`	`assistant.chat`	Start a new conversation (204)
GET	`/api/v1/assistant/providers`	`assistant.chat`	Configured providers for the model selector (no key data)
PUT	`/api/v1/assistant/actions/{id}`	`assistant.actions.confirm`	Edit a pending mail (recipients, subject, body)
POST	`/api/v1/assistant/actions/{id}/confirm`	`assistant.actions.confirm`	Execute a prepared action (the only path that runs one)
POST	`/api/v1/assistant/actions/{id}/cancel`	`assistant.actions.confirm`	Cancel a pending action

Verb	Route	Permission	Purpose
GET / PUT	`/api/v1/assistant/providers/config`	`settings.ai.view` / `settings.ai.edit`	TFA Settings: provider configuration
DELETE	`/api/v1/assistant/providers/{kind}/models`	`settings.ai.model.remove`	Remove an added model
POST	`/api/v1/assistant/providers/{kind}/test`	`settings.ai.test`	Connection test through `ChatModelGateway`

`assistant.chat` and `assistant.actions.confirm` are granted to Manager and Employee by default and to every existing role when first added. The `settings.ai.*` keys are Administrator defaults (legacy `settings.ai.manage` expands to the four). Action ids are Guids; another user's conversation or action returns 404.

16.3 Conversation engine (`AssistantService.SendAsync`)

Step	Rule
1 Length	1–2000 characters (`AssistantRules.MaxMessageLength`), else 400
2 Secret screen	`SecretScreen.ContainsSecret` (password phrases, `sk-…`, `AIza…`, `Bearer …`, JWTs, long mixed tokens) → a `Notice` reply; nothing is sent to a model or stored
3 Provider	Requested provider or the first available; 400 when none is configured
4 Rate limit	`AssistantRateLimiter`: 20 requests per user per sliding minute (in memory) → 429 with `Retry-After`
5 History	Latest conversation; newest 12 messages (`HistoryMessages`), at most 6000 characters (`HistoryCharacters`), notices dropped
6 Prompt	`AssistantPrompt.Build(userName, role, today in app timezone, application name)`
7 Tool loop	Up to 4 rounds (`MaxToolRounds`), the last without tools; tool results are marked as data and passed through `SecretScreen.Redact`; a fixed reply (refusal or prepared action) ends the turn with that exact text
8 Persist	Only after the provider answered: user and assistant `AssistantMessage` rows (`MetaJson` = kind, links, ranked tasks, action id; `ProviderKind`); a provider failure stores nothing

Entities: `AssistantConversation` (latest by id is active), `AssistantMessage` (`Role`: `AssistantMessageRole` User, Assistant, Tool). Conversations are kept indefinitely (no retention job).

16.4 AI providers

Provider (`AiProviderKind`)	Endpoint	Locality	Key transport	Built-in default model
Ollama	`{BaseUrl or http://localhost:11434}/api/chat`	Local	none	`llama3.2:3b`
Gemini	`https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent`	Cloud	`x-goog-api-key` header	`gemini-2.0-flash`
OpenAI	`https://api.openai.com/v1/chat/completions`	Cloud	Bearer header	`gpt-4o-mini`

- One `AiProviderConfig` row per provider (`IsEnabled`, `ModelName`, `BaseUrl`, `EncryptedApiKey`, `KeyLast4`); selectable models in `AiModel` (`IsBuiltIn`; built-in names in `AiProviderCatalog`). On a

brand-new installation `AiProviderSeeder` turns Ollama on with `llama3.2:3b`; Gemini and OpenAI start off with no key.

- Failures are classified into `ProviderFailureKind` (RateLimited, InvalidKey, Timeout, Unreachable, ModelUnavailable, Unexpected) and shown as fixed messages with the other available providers as alternatives; TFA never switches model by itself. Raw provider bodies are never shown or logged (provider, kind and HTTP status only).
- `AiProviders:TimeoutSeconds` (60) is the `HttpClient` timeout; redirects are not followed.
- The SPA marks cloud models and shows a one-time notice that the message and related task details go to that provider (`localStorage` `tfa.cloudAck`); the chosen provider is kept in `tfa.provider`.

16.5 Tools

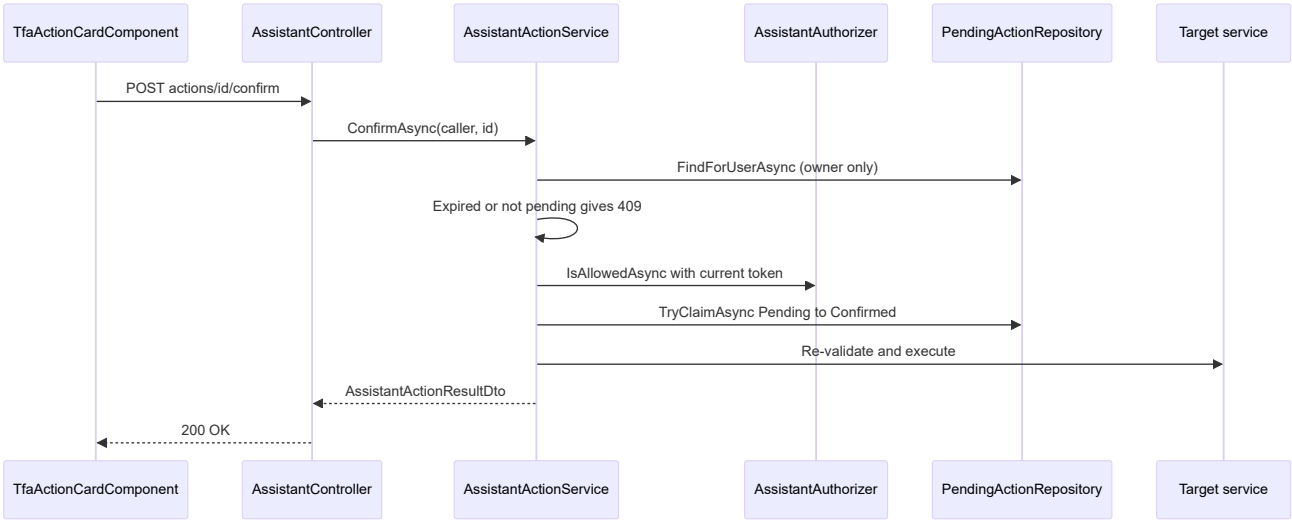
`AssistantToolbox.DefinitionsForAsync` advertises only the tools whose policy the caller passes, and `ExecuteAsync` re-checks the gate on every call. No tool accepts a user id, role or password; the caller always comes from the token.

Tool	Policy	Effect
`get_my_tasks`	signed-in	Open tasks (`upcoming`, `due_today`, `overdue`, `all_open`; scope `mine` or `accessible`; max 20) through `ITaskService`, so Task Management scope applies; links `/tasks/{id}`
`prioritise_tasks`	signed-in	`TaskPrioritiser.Rank`: overdue, then due today, then local due date, priority, deadline, task code (max 10). The model only explains the order
`search_help`	signed-in	`HelpKnowledgeBase.Search(topic, role)`: keyword-scored, role-aware topics with screen links
`find_user`	signed-in	`IMailService.GetContactsAsync` (max 10 active users)
`explain_password_change`	signed-in	Fixed guidance and `/profile` link; passwords never go through chat
`contact_admin_guidance`	signed-in	Fixed guidance text
`draft_mail`	signed-in	Creates a `SendMail` pending action ("my manager" = task creator, else the single Manager of the caller's department)
`propose_profile_update`	signed-in	Creates an `UpdateOwnProfile` pending action (own name and email only)
`propose_role_change`	`roles.manage`	Creates a `ChangeUserRole` pending action; refuses the caller's own role

> ⚠️ Gap: `propose_role_change` and its confirmation check only `roles.manage` and call `IEmployeeService.UpdateAsync` directly, while `PUT /api/v1/users/{id}` also requires `employees.role.assign` and, for Administrator moves, `roles.administrator.assign`. "My manager" resolution only finds users in the system Manager role.

16.6 Confirm-before-act

Action tools only create an `AssistantPendingAction` (`Kind`: `AssistantActionKind` SendMail, UpdateOwnProfile, ChangeUserRole; `Status`: `AssistantActionStatus` Pending, Confirmed, Cancelled, Expired, Refused, Failed; `ExpiresAt` = now + 10 minutes `AssistantActionRules.Lifetime`; `PayloadJson` = the exact payload shown on the card). The Angular `TfaActionCardComponent` shows Confirm / Edit (mail only) / Cancel.



Rule	Behaviour
Ownership	Every lookup filters by `UserId`; unknown or foreign id → 404
Expiry	Evaluated lazily on read/confirm; expired → status `Expired`, 409 "This request has expired. Please ask TFA again."
Permission re-check	`AssistantActionRules.Requirement(kind)` is evaluated again with the caller's current token; refused → status `Refused`, fixed refusal text (HTTP 200)
Run at most once	`TryClaimAsync` is one `ExecuteUpdate` from Pending to Confirmed; a lost race → 409 "This action has already been done."
Execution	`SendMail` → `SendMailRequestValidator` + `IMailService.SendAsync`; `UpdateOwnProfile` → `UpdateOwnProfileRequestValidator` + `IProfileService.UpdateAsync`; `ChangeUserRole` → `IEmployeeService.UpdateAsync` with the new `RoleId`
Failure	Status `Failed` and "I could not do that: {reason}. Nothing was sent/changed."

16.7 Audit, rate limit and secret handling

Control	Design
Audit	`AssistantAuditLog` writes `AssistantAudit` rows (`UserId`, `Kind`, `Outcome`, `At`): `prepared`, `confirmed`, `cancelled`, `expired`, `refused`, `failed`. No message bodies or keys. No audit screen exists
Denied endpoints	403s on TFA endpoints are also logged by `DeniedActionAuditHandler`
Rate limit	`AssistantRateLimiter` (singleton, `RequestsPerMinute = 20`, sliding 1 minute); counted only for messages that pass length, secret and provider checks
Secrets	`SecretScreen` on input and `SecretScreen.Redact` on tool output (`[hidden]`); provider keys encrypted (§14.3)
Model containment	The model can call only advertised tools; action tools only prepare; refusals and confirmations use fixed `AssistantTexts`

16.8 Help knowledge and voice

Feature	Design
Help knowledge	19 embedded `*.md` topics (`EmbeddedResource` in `TaskFlow.Api.csproj`) with front matter `title` , `roles` , `others` , `keywords` , `link` ; keyword-scored search with a minimum score; a user outside `roles` gets the `others` text instead. Reached only through `search_help`
Voice (front end only)	`WakePhraseService` wraps the browser `SpeechRecognition` / `webkitSpeechRecognition` (Chrome, Edge): hands-free wake phrase "Hey TF assistant", 2 s command silence, mic-button dictation; the off choice is stored in `tfa.handsfree` . `SpeakService` reads replies to spoken questions aloud (`speechSynthesis` , max 500 characters). Recognised text goes through the normal `POST /api/v1/assistant/conversation/messages` , so the same permission checks apply

16.9 Frontend

Element	Responsibility
`TfaPanelComponent`	Floating TFA button and panel, cloud notice, feed, failure card, voice wiring; mounted by `ShellComponent` only with `assistant.chat` . No route and no menu entry
`TfaHeaderComponent`	Title, New chat, Close, model selector
`TfaComposerComponent`	Message box, Send, starter suggestions, mic and hands-free toggle
`TfaMessageComponent`	Text, refusal, notice, ranked tasks, screen links, embedded action card
`TfaActionCardComponent`	Confirm / Edit / Cancel, mail edit form, status labels (not gated by `assistant.actions.confirm` , §14.5)
`AssistantService` (Angular)	All `/api/v1/assistant` calls; signals for panel, providers, messages, failure; provider list refreshed every 60 s (`PROVIDER_REFRESH_MS`)

---

17. Non-Functional Design

How the code meets the BRD non-functional requirements (BRD §5). Security controls are in §14.

Area (NFR)	Design approach in the current code
Performance (§5.1)	Paged lists: tasks and notifications default to 20 rows per page, capped at 100 (`TaskService` , `NotificationService`); mail uses `MailRules.DefaultPageSize` / `MailRules.MaxPageSize` . Dashboard and analytics endpoints return aggregated counts from repository queries (`DashboardRepository` , `AnalyticsRepository`). All EF Core calls are async. The reminder pass first reads today's delivered slots (`GetDeliveredAsync`) so already-sent reminders are skipped without taking the SQLite write lock. The report CSV export is capped at 50,000 rows and built in memory (`ReportCsv`).

Area (NFR)	Design approach in the current code
Reliability (§5.4)	Data is persisted in SQLite (`taskflow.db`). `ReminderSchedulerService` keeps no state: it runs a pass at startup and then every `Reminders:TickSeconds`; a failed pass is logged and the next tick retries. Duplicate prevention lives in the database (unique `ReminderHistory` index on TaskId, RecipientUserId, ReminderType, ScheduledFor with `ON CONFLICT DO NOTHING`); each reminder delivery (history row + notification + optional Inbox mail) is one transaction (`IUnitOfWork.ExecuteInTransactionAsync`). Migrations and required seeders run on every start, so a fresh or upgraded install is self-configuring. The data folder is checked for write access before anything is touched (`StorageRoot.EnsureAvailable`).
Usability (§5.3)	Angular 19 standalone components with shared CSS design tokens; menu entries built from the user's permissions (§12); confirmation dialogs for destructive actions and for every TFA action (§16); server messages returned as ProblemDetails.
Maintainability (§5.5)	One API project, layer-first with module sub-folders (§5). Layer and folder rules are enforced on every test run by `ArchitectureTests.cs`; the frontend layout is enforced by `npm run check:structure` (first step of `npm test`). Business rules sit in pure classes (`TaskStatusTransitions`, `ReminderCalculator`, `ReminderIntervals`) that are unit-tested directly. DI registration is grouped per layer in `Config/` (`CrossCuttingRegistration`, `RepositoryRegistration`, `ServiceRegistration`, `IntegrationRegistration`).
Compatibility (§5.6)	Modern desktop browsers; responsive layout. TFA voice input needs the browser Web Speech API (Chrome, Edge); typing always works. Backend runs on Windows, Linux and macOS (.NET 10); DPAPI key-ring protection applies on Windows only.
Backup/Recovery (§5.7)	Single data folder (`C:\TaskFlow`) with a documented backup set (§19).
Observability	`RequestLoggingMiddleware` writes one entry per request (method, path, status, elapsed ms, user id, trace id), never bodies. Refused authorizations are recorded by `DeniedActionAuditHandler`. TFA actions are recorded in `AssistantAudit`. Logging uses the built-in `Microsoft.Extensions.Logging` providers (no Serilog).
Capacity limits	Designed for a single API instance: `AttemptLimiter` (sign-in/recovery throttles) and `AssistantRateLimiter` (20 requests per minute per user) are in memory per process, and the scheduler is not coordinated across instances (duplicates are still blocked by the unique index).

18. Configuration & Deployment

18.1 Configuration sources

Later sources override earlier ones.

Order	Source	Holds
1	`appsettings.json`	Non-secret defaults (§18.2)
2	`appsettings.{Environment}.json` (ASP.NET Core default)	Per-environment overrides, e.g. `Seed:Enabled` in Development (per README)

Order	Source	Holds
3	`appsettings.Environment.json` (optional, `reloadOnChange: true`, added in `Program.cs`)	Per-environment values: CORS origins, mail links, seeding
4	.NET user-secrets (`UserSecretsId` in `TaskFlow.Api.csproj`, Development)	Secrets such as `Jwt:Key`
5	Environment variables (`Section__Key`, e.g. `Mfa__Required=false`, `Storage__Root`)	Deployment overrides and secrets
–	`ApplicationSettings` table	Runtime-editable settings changed from the UI (§18.3)

> ⚠️ To be confirmed: neither `appsettings.Development.json` nor `appsettings.Environment.json` is in the code snapshot, so their actual values cannot be listed.

18.2 Configuration keys and defaults

Key	Default (`appsettings.json` / options class)	Purpose
`Logging:LogLevel:Default`	`Information`	Default log level
`Logging:LogLevel:Microsoft.AspNetCore`	`Warning`	Framework log level
`ConnectionStrings:Default`	`<secret>` (relative `taskflow.db` data source, resolved under the storage root by `DatabasePathResolver`)	SQLite database
`Storage:Root`	`""` → `C:\TaskFlow` on Windows, `TaskFlow` under the common application-data folder elsewhere (`StorageOptions.DefaultRoot`)	Data folder
`Jwt:Issuer`	`TaskFlow`	Token issuer (validated)
`Jwt:Key`	`<secret>` — not in `appsettings.json`; supplied by user-secrets or environment	Symmetric signing key
`Jwt:AccessTokenMinutes`	30 (`JwtOptions`)	Access token lifetime
`Jwt:RefreshTokenDays`	7 (`JwtOptions`)	Refresh token lifetime
`Application:Timezone`	`Asia/Kolkata` (options fallback `UTC`, `SettingKeys.FallbackTimezone`)	Seeds the `Application.Timezone` setting; fallback while the row is missing
`Reminders:SchedulerEnabled`	`true`	Turns the background reminder scheduler on/off
`Reminders:TickSeconds`	60	Seconds between reminder passes
`Mail:AttachmentsPath`	`mail-attachments` (relative → under storage root)	Mail attachment files
`Mail:AppBaseUrl`	not set (`null`)	Base URL for task links in reminder mails

Key	Default (`appsettings.json` / options class)	Purpose
`Cors:AllowedOrigins`	not in `appsettings.json`; empty array when missing (no cross-origin caller allowed)	Origins for the `SpaClient` CORS policy (any header/method, credentials allowed)
`Proxy:KnownProxies`	`[]` (only loopback proxies trusted)	Proxies trusted for `X-Forwarded-For`
`Mfa:Issuer`	`TaskFlow`	Name shown in authenticator apps
`Mfa:Required`	`true`	Mandatory MFA gate (`MfaSetupGateMiddleware`); `false` only as break-glass
`Mfa:MaxFailedAttempts` / `Mfa:LockoutMinutes`	5 / 5 (`MfaOptions`)	Code lockout
`Mfa:BackupCodeCount`	10 (`MfaOptions`)	Backup codes issued
`Mfa:ChallengeMinutes`	5 (`MfaOptions`)	Validity of the post-password sign-in challenge
`AiProviders:KeyRingPath`	`dp-keys` (relative → under storage root)	Data Protection key ring (AI keys, MFA secrets)
`AiProviders:TimeoutSeconds`	60	Timeout of one AI provider call
`Seed:Enabled`	`false` (`SeedOptions`)	Create the three demo sign-ins
`Seed:AdministratorPassword` / `Seed:ManagerPassword` / `Seed:EmployeePassword`	`<secret>`	Passwords for the demo sign-ins
`AllowedHosts`	`*`	Host filtering

18.3 Runtime settings (`ApplicationSettings` table)

Inserted with defaults by `SettingsInitializer` on every start when missing; existing rows are never overwritten (`SettingKeys`).

Key	Category	Default	Edited on
`Application.Timezone`	Application	from `Application:Timezone`	Application Settings (`/settings/application`)
`Application.Name`	Application	`TaskFlow`	Application Settings
`Application.DefaultPriority`	Application	`Medium`	Application Settings
`Reminder.Enabled`	Reminder	`true`	Reminder Settings (`/settings/reminders`)
`Reminder.DeadlineDayEnabled`	Reminder	`true`	Reminder Settings
`Reminder.DefaultFirstDays`	Reminder	2 (1–30)	Reminder Settings
`Reminder.DefaultSecondDays`	Reminder	1 (0–30)	Reminder Settings
`Reminder.DefaultOverdueFrequency`	Reminder	`Daily`	Reminder Settings

Key	Category	Default	Edited on
`Notification.MailEnabled`	Notification	`true`	Reminder Settings ("Also deliver reminders to Mail")
`Notification.InAppEnabled`	Notification	`true`	–

18.4 Build and run

Step	Command / value
Prerequisites	.NET 10 SDK, Node.js 22 LTS; optional Ollama for local AI (`scripts/setup-ai.ps1`, `scripts/setup-ai.sh`)
Frontend packages (first time)	`npm install` in `TaskFlow Proj/frontend`
Run API + SPA	`dotnet run --project src/TaskFlow.Api` in `TaskFlow Proj/backend` (or F5 on `TaskFlow.sln`). `Microsoft.AspNetCore.SpaProxy` starts the Angular dev server (`SpaProxyLaunchCommand` = `npm start`, `SpaProxyServerUrl` = `http://localhost:4200`)
Addresses	Web app `http://localhost:4200`; API `https://localhost:7055` (the SPA calls `/api/*` relative in the `local` environment)
Dev certificate	`dotnet dev-certs https --trust`
Frontend production build	`ng build` (`npm run build`)
API base URL of the SPA	`config/api-base-url.ts`: one `ACTIVE` environment (`local` = `''` relative calls via `proxy.conf.json`; `develop`, `qa`, `production` = absolute API URLs), prefixed by `api-base-url.interceptor.ts`

> ⚠️ To be confirmed: `proxy.conf.json` and `launchSettings.json` are not in the snapshot (ports taken from the README). `Program.cs` has no static-file middleware, so the built SPA is hosted separately from the API.

18.5 Startup sequence and seeding rules

#	Step (`Program.cs`)	Rule
1	`StorageRoot.Resolve` + `EnsureAvailable`	Creates the data folder and proves it is writable; otherwise startup stops with `StorageUnavailableException` naming the folder
2	`StorageRoot.FindLegacyDatabase`	Warns when an old `database/taskflow.db` exists in the solution but the data folder has none (data is not migrated)
3	`KeyRingProtectionCheck.Check` (Windows)	Warns about key-ring keys stored without DPAPI protection
4	`Database.MigrateAsync()`	Applies all EF Core migrations (§7)

#	Step (`Program.cs`)	Rule
5	`SettingsInitializer`	Inserts missing `ApplicationSettings` rows (always)
6	`AccessControlSeeder`	Ensures system roles, inserts missing catalog permissions, grants every permission row to Administrator, first-time grants for Manager/Employee (always; Appendix C)
7	`AiProviderSeeder`	Built-in AI models and one row per provider; on a brand-new install Ollama is enabled with `llama3.2:3b` at `http://localhost:11434` (always; existing settings never overwritten)
8	`DatabaseSeeder`	Creates `admin@taskflow.com`, `manager@taskflow.com`, `employee@taskflow.com` only when `Seed:Enabled` is `true`; existing accounts are never changed
9	Pipeline	`UseForwardedHeaders` → `RequestLoggingMiddleware` → `UseExceptionHandler` → `UseHttpsRedirection` → `UseCors(SpaClient)` → `UseAuthentication` → `MfaSetupGateMiddleware` → `UseAuthorization` → `MapControllers`; OpenAPI is mapped in Development only

EF Core migrations are the schema scripts (BRD deliverable §9); demo data is created by `DatabaseSeeder` instead of a seed script folder.

---

19. Backup & Recovery (NFR §5.7)

19.1 What lives in the data folder

Item (under `Storage:Root`, default `C:\TaskFlow`)	Content	Back up
`taskflow.db`	All tables (§7)	Yes, together with `mail-attachments`
`mail-attachments\` (`Mail:AttachmentsPath`)	Attachment files under generated names; `MailAttachment.StorageKey` points at them	Yes, in the same backup as the database (rows and files must match)
`dp-keys\` (`AiProviders:KeyRingPath`)	Data Protection key ring: decrypts AI provider API keys (`AiProviderConfig.EncryptedApiKey`) and MFA secrets (`User.MfaSecretProtected`)	Yes, but **separately** from the database backup, and never committed

The key ring is kept apart so a copied database alone cannot be used to generate MFA codes or read API keys. It must still be kept: without it every stored AI key has to be re-entered in TFA Settings, and every user with MFA needs an administrator reset (`employees.mfa.reset`). On Windows the key ring is DPAPI-protected for the account running the API, so restoring it on another machine or account has the same effect.

19.2 Procedures

Task	Steps
Backup	Stop the API (or take a consistent snapshot with `VACUUM INTO 'backup-<date>.db'`), copy `taskflow.db` and `mail-attachments\` to the backup location; copy `dp-keys\` to a separate protected location
Restore	Stop the API, replace `taskflow.db` and `mail-attachments\` from the same backup, put back `dp-keys\` (same machine/account on Windows), start the API; pending migrations are applied automatically on start

Task	Steps
Upgrade	Back up first. Some migrations are irreversible, e.g. `20261019173718_NormalizeEmailDomain` rewrites every email to `@taskflow.com` and the original domains cannot be restored
Move the data folder	Stop the API, copy the whole folder, set `Storage:Root` (or `Storage__Root`) to the new path, start
Legacy install	The old `database/` folder is not migrated; copy `taskflow.db` (and `dp-keys`, `mail-attachments`) into the data folder, or point `Storage:Root` at the old folder (startup warns, §18.5)

> ⚠️ To be confirmed: no scheduled backup job exists in code; backups are an operational task.

---

20. Testing Strategy

20.1 Test projects

Project	Tooling	Test classes	Scope
`TaskFlow.UnitTests`	xUnit	81 (`*Tests.cs`)	Pure rules, services with fakes, validators, seeders/migrations on test databases, integrations, architecture rules
`TaskFlow.IntegrationTests`	xUnit + `WebApplicationFactory` (`TaskFlowApiFactory`)	43	HTTP endpoints end-to-end through the real pipeline, authentication, permissions, ProblemDetails
Frontend	Jasmine + Karma (`ng test`), preceded by `npm run check:structure`	–	Component/service specs and folder-structure rules

The 150 test files also include shared fixtures in `TestSupport/` (e.g. `FakeTimeProvider`, `FakeHttpHandler`, `ScriptedChatProvider`, `MfaTestSupport`, `TestAccessControl`, `WindowsOnlyFactAttribute` for DPAPI-dependent tests) and `GlobalUsings.cs`. Test folders mirror the code modules (§5). There is no end-to-end browser test tool (Playwright is not installed).

> ⚠️ To be confirmed: no `*.spec.ts` files are in the frontend snapshot, and the test projects' package references (e.g. a mocking library) were not supplied.

20.2 Coverage by area

Area	Unit tests (examples)	Integration tests (examples)
Architecture	`ArchitectureTests`	–
Auth / MFA / recovery	`AuthServiceLoginTests`, `AuthServiceRefreshAndLogoutTests`, `AuthServiceMfaTests`, `MfaServiceTests`, `PasswordRecoveryServiceTests`, `AttemptLimiterTests`, `TotpTests`, `JwtTokenServiceTests`	`AuthEndpointsTests`, `MfaSignInEndpointsTests`, `MfaSetupGateEndpointsTests`, `MfaEnrolmentEndpointsTests`, `MfaStepUpEndpointsTests`, `MfaRecoveryEndpointsTests`, `MfaAdminResetEndpointsTests`, `PasswordRecoveryEndpointsTests`

Area	Unit tests (examples)	Integration tests (examples)
Permissions	`PermissionCatalogTests`, `CustomRolePermissionTests`, `CustomRoleWorkloadTests`, `RolesMigrationTests`	`PermissionAuthorizationTests`, `ModuleBasedPermissionEndpointsTests`, `RolesEndpointsTests`, `RoleAssignmentSafeguardTests`
Tasks	`TaskTrackingDomainTests`, `TaskServiceTests`, `TaskManagementServiceTests`, `TaskValidatorsTests`, `CreateTaskRequestValidatorTests`, `TaskDeletionTests`, `TaskLogHoursTests`, `ProjectTaskTests`, `TaskCommentActivityNotificationTests`	`TasksEndpointsTests`, `WorkloadEndpointsTests`
Reminders / notifications	`ReminderCalculatorTests`, `ReminderIdempotencyTests`, `ReminderSchedulerServiceTests`, `ReminderServiceTests`, `NotificationServiceTests`	`NotificationsEndpointsTests`, `SettingsEndpointsTests`
Mail	`MailSendTests`, `MailDraftAndThreadTests`, `MailboxTests`, `MailAttachmentTests`	`MailEndpointsTests`
Dashboard / reports	`DashboardServiceTests`, `*AnalyticsServiceTests`, `AnalyticsScopeTests`, `ManagerScopeTests`, `ReportServiceTests`, `ReportCsvTests`	`DashboardEndpointsTests`, `*AnalyticsEndpointsTests`, `EmployeeAnalyticsIsolationTests`, `ManagerAnalyticsIsolationTests`, `ReportEndpointsTests`
Management	`UserAllocationServiceTests`, `ManagerAssignmentTests`, `UserDeletionTests`, `DepartmentRequestValidatorsTests`	`UsersEndpointsTests`, `LastAdministratorTests`, `LastAdministratorStatusTests`, `AdminDeleteEndpointsTests`, `DepartmentsEndpointsTests`, `ProjectTeamEndpointsTests`
Assistant / AI	`AssistantServiceTests`, `AssistantActionServiceTests`, `AssistantActionConcurrencyTests`, `AssistantToolGateTests`, `SecretScreenTests`, `ChatModelGatewayTests`, `ProviderFailureHandlingTests`, `AiKeyProtectorTests`	`AssistantEndpointsTests`, `AssistantProviderEndpointsTests`, `AiDefaultsOnFirstRunTests`, `ProviderRedirectTests`
Platform	`StorageRootTests`, `SettingsInitializerTests`, `DatabaseSeederTests`, `AppClockTests`, `AppExceptionTests`, `EmailDomainTests`	`ApiConventionsTests`, `GlobalExceptionHandlerTests`, `DependencyInjectionTests`, `ForwardedHeadersTests`, `KeyRingStartupWarningTests`, `AuthenticationWiringTests`

20.3 Critical scenarios

Scenario	Covered by
Repeated scheduler passes never duplicate a reminder	`ReminderIdempotencyTests`, `ReminderServiceTests`
Completed/cancelled tasks stop reminders; deadline change recalculates	`ReminderCalculatorTests`, `ReminderServiceTests`
Illegal status transitions are refused (incl. InQA)	`TaskTrackingDomainTests`
Users cannot read others' tasks or another team's analytics	`EmployeeAnalyticsIsolationTests`, `ManagerAnalyticsIsolationTests`, `TasksEndpointsTests`
Every endpoint refuses callers without its permission key	`PermissionAuthorizationTests`, `ModuleBasedPermissionEndpointsTests`
No access before MFA setup when `Mfa:Required`	`MfaSetupGateEndpointsTests`
The last Administrator cannot be demoted, deactivated or deleted	`LastAdministratorTests`, `LastAdministratorStatusTests`
TFA acts only after confirmation and within the caller's permissions	`AssistantActionServiceTests`, `AssistantToolGateTests`

Commands: `dotnet test` in `TaskFlow Proj/backend`; `npm test` (structure check + Karma) or `npm run check:structure` in `TaskFlow Proj/frontend`.

---

21. Acceptance Criteria Mapping (BRD §8)

The first column is the original BRD §8 criterion; the second the matching criterion of the approved overall BRD (§14 of `BRD-TaskFlow-Overall.md`), whose sub-module criteria (`AC-<CODE>-nn`) are listed there.

Acceptance criterion	Approved BRD	Where satisfied
Users log in with appropriate roles	AC-01	§9 (login, MFA, role landing), §8.2, Screen 1
(added) Generic sign-in failure message	AC-02	§9.2, §9.6
(added) Two-step verification at sign-in	AC-03	§9.3, §9.6
(added) Sessions: reload, renewal, role changes, sign-out	AC-04	§9.4
(added) Password recovery with an authenticator code	AC-05	§9.5, §9.6
(added) Mandatory Security Setup with backup codes	AC-06	§9.3, §13.2
Admins manage employee accounts	AC-07	§8.4, §10, Screen 5
Managers create and assign tasks	AC-08	§10, §8.5, Screen 8 (`tasks.create`)
Employees view/update assigned tasks	AC-09	§9 (scopes Own/Team/Organization), §10.2, Appendix A, Screens 4/9/10
Upcoming and overdue tasks shown correctly	AC-10	§7 (time and overdue derivation), §10, §8.3
Deadline reminders generated automatically	AC-11	§11, Appendix B
Assignee and selected recipients receive reminders	AC-12	§10, §11 (recipients + creator, §23)

Acceptance criterion	Approved BRD	Where satisfied
In-app notifications work when email fails	AC-13	§11, §15 (SMTP replaced by internal Mail; bell notification always written)
Completed/cancelled tasks stop reminders	AC-14	§11, Appendix A, Appendix B
Duplicate reminders prevented	AC-15	§11.4 (unique `ReminderHistory` index), §20.3
Reassignment preserves history	AC-16	§10 (`TaskActivityLog` `Reassigned`), Screen 15
Managers view workload summaries	AC-17	§8.5, Screen 16 (`workload.view`)
Reports filterable and exportable	AC-18	§8.7, Screen 17 (`reports.view` , `reports.export`)
Unauthorized access prevented	AC-19	§9.3–9.10, §14, Appendix C, §20.3
App continues after restart with history preserved	AC-20	§11 (stateless scheduler), §11.6, §17, §19
(added) Settings changed without code changes or restart	AC-21	§11.3, §18.3, §9.9
(added) Private internal mail, nothing sent outside TaskFlow	AC-22	§15
(added) TFA actions only after confirmation, within permissions, audited	AC-23	§16.6, §16.7
(added) Legal documents reachable from the footer by anyone	AC-24	§12, §13.2 (Legal pages)
(added) Privacy, Terms and Disclaimer statements	AC-25	§12, §16.4 (cloud AI)
(added) Ownership footer and browser title on every screen	AC-26	§12

22. Risks, Assumptions & Open Questions

22.1 Risks

Risk	Impact	Mitigation in code / remaining action
Scheduler run on more than one instance	Extra passes	Unique `ReminderHistory` index + `ON CONFLICT DO NOTHING` ; single instance assumed
SQLite write contention	Slow writes under load	Short transactions; delivered slots read before writing; no WAL setting in code (to consider)
In-memory throttles (`AttemptLimiter` , `AssistantRateLimiter`)	Reset on restart, not shared across instances	Single instance assumed
Loss of `dp-keys`	AI keys unreadable, MFA re-enrolment for all users	Separate key-ring backup (§19)
Timezone drift client/server	Wrong due-today/overdue	Server-side UTC and one app timezone (`Application.Timezone`); report deadline-day conversion still uses the browser timezone
Cloud AI providers	Task details leave the server	Ollama default; users warned once for Cloud models; `SecretScreen` blocks passwords/keys (§16)

Risk	Impact	Mitigation in code / remaining action
Irreversible data migrations	Lost email domains	Backup before upgrade (§19)

22.2 Assumptions

- Single API instance, single SQLite file, one application timezone (default `Asia/Kolkata`).
- Reminder defaults: first reminder 2 days, second 1 day, deadline-day reminder on, overdue `Daily`; per-task values may override them at creation.
- Every employee email is on the fixed `@taskflow.com` domain.

22.3 Questions from v1.0 — answered

v1.0 question	Answer in the code
Is a Manager's scope global or per department?	Permission-based. `tasks.view.all` alone gives organization-wide tasks; a role holding `dashboard.view.team` without `dashboard.view.organization` is team-scoped: it sees its team's tasks plus tasks it created, is assigned to or receives (`TaskService.VisibilityForCallerAsync`)
Access token lifetime / single-session logout?	`Jwt:AccessTokenMinutes` = 30, `Jwt:RefreshTokenDays` = 7; refresh-token rotation, logout revokes the refresh token, and the security stamp ends all sessions on role/permission change or MFA reset (§9)
Required CSV columns?	`TaskCode, Title, Assignee, Project, Priority, Status, StartDate, Deadline, EstimatedHours, CompletedAt` (`ReportCsv`, file `taskflow-report.csv`)

22.4 Known gaps (from the module technical docs)

Area	Gap
Auth	Password sign-in (`/api/v1/auth/login`) has no attempt limit; the login form applies an email validator although the API accepts usernames; backup codes (11 chars) cannot be typed into the 6-char sign-in code box
Auth	No UI calls `/api/v1/users/me/mfa/backup-codes` (regenerate backup codes)
Tasks	`DELETE /api/v1/tasks/{id}` and `PUT /api/v1/tasks/{id}/recipients` check the permission but not the task scope; reminder intervals cannot be edited after creation
Tasks	`TaskAccessHandler` does not apply the manager team scope to comments and activity endpoints
Projects	`/api/v1/projects/{id}/tasks` needs only `projects.view` and ignores the task view scope
Board / Workload	`board.filters.use`, `workload.filters.use`, `workload.tasks.open` are enforced only in the UI; workload is organization-wide
Notifications	Reminder notifications bypass the `Notification.InAppEnabled` toggle; the frontend type list lacks `TaskComment`; `ReminderHistory.Status` is always `Sent` and `DeliveryAttempts` 0 (no retry tracking)
Mail	`mail.attachments` is not checked in the UI; `/api/v1/mail/contacts` exposes the active-user directory to any `mail.compose` holder; purge is soft (files never deleted)
Reports	Organization-wide for any `reports.view` holder (BRD asks for permitted records only); export capped at 50,000 rows without warning
Roles	`/api/v1/roles/{id}` ignores `roles.view.all`; `roles.manage` is assignable to custom roles
TFA	`propose_role_change` checks only `roles.manage`, not `employees.role.assign` / `roles.administrator.assign`
Tasks	Worked-hours correction is tied to the role name `Administrator`, not to a permission key, so a custom role cannot be given it

Area	Gap
Documentation	The README says both to keep `dp-keys` out of any backup that holds the database and to keep it with the database in backups; §19 states the real trade-off (back it up, store it separately)
Code comments	`PermissionKeys.cs` calls the mail, notification and assistant keys "always-on", but `PermissionCatalog.cs` (authoritative) makes them configurable grants (Appendix C)
Frontend	Several role-name checks (`user.role === 'Manager'` / `'Employee'` , `dashboardRouteFor(role)`) instead of permission checks affect custom roles

> ⚠️ To be confirmed: the intended Custom-role value of `roles.administrator.assign` — the permission audit matrix says Configurable, the catalog says not assignable (this document follows the catalog).

22.5 Where open items are tracked in the document set

The gaps in §22.4 are recorded with the same wording as "⚠️ To be confirmed" items in the sub-module documents, so a decision on any of them is made once in the approved BRD (through a change to that sub-module BRD) and then flows to the FRD, HLD, LLD and this TDD.

Gap area (§22.4)	Recorded in
Auth (sign-in attempt limit, backup-code entry, backup-code regeneration)	`BRD/FRD/HLD/LLD-TaskFlow-Overall.md` (Authentication), `*-My-Profile.md`
Tasks (delete / recipients scope, reminder intervals after creation, comments/activity team scope, hours correction by role name)	`*-Task-Management.md`
Projects (`/projects/{id}/tasks` scope)	`*-Project-Management.md`
Board / Workload (filter keys enforced in the UI only, organization-wide workload)	`*-Board.md` , `*-Employee-Workload.md`
Notifications (in-app toggle bypassed by reminders, `TaskComment` type, retry tracking)	`*-Notification.md` , `*-Reminder-Settings.md`
Mail (attachments permission in the UI, contacts directory, soft purge)	`*-Mail.md`
Reports (organization-wide, 50,000-row cap)	`*-Reports.md`
Roles (`roles.view.all` , `roles.manage` assignable, `roles.administrator.assign`)	`*-Role-Configuration.md`
TFA (`propose_role_change` checks only `roles.manage`)	`*-Assistant.md`

23. Deviations from BRD

BRD item	Implemented as	Reason / notes
Screen 14 Email Notification Settings (SMTP)	**Superseded** by internal Mail (`/mail` , §15); SMTP rows and delivery logs dropped in `20261017090431_AddInternalMail`	Nothing leaves TaskFlow; reminders are copied to the Inbox when `Notification.MailEnabled` ("Also deliver reminders to Mail", Screen 13)
Fixed roles Administrator / Manager / Employee	Permission-based RBAC: 70 keys (Appendix C), read-only system roles plus custom roles	Finer control; legacy broad keys still honoured

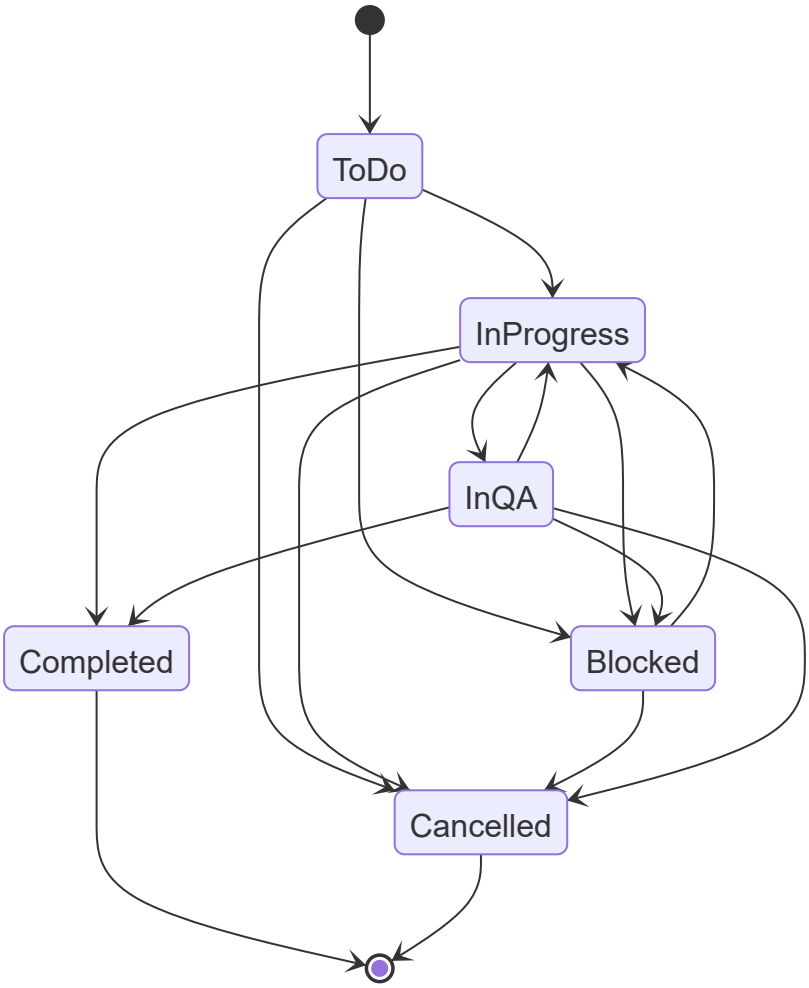
BRD item	Implemented as	Reason / notes
Screen 1 password only	Mandatory MFA (TOTP + 10 backup codes), `Mfa:Required`	Security hardening
Password reset (admin or email)	Self-service recovery with an authenticator code; Admin can only reset MFA	No email channel; no admin-known passwords
Screen 19 "require current password"	Change password (`/api/v1/users/me/password`) asks for an authenticator code, on separate route `/profile/change-password`	Code is the proof of possession
Screen 8 "Initial status"	Tasks always start as `ToDo`	Status changes go through the transition rules (Appendix A)
Status set (BRD)	Adds `InQA` ; Kanban (Screen 10) has five columns (To Do, In Progress, In QA, Blocked, Completed) and no Cancelled column	QA step in the workflow
Reminder schedule "2 days / 1 day / due day / daily overdue"	`FirstReminder` / `SecondReminder` per-task intervals (0–30 days), `DueToday` , `Overdue` `Daily` or `Weekly`	Configurable defaults in Reminder Settings
Reminder recipients = task recipients (BR-5)	Task recipients (assignee always one) **plus the creator**, active users only (§11.4)	Creator keeps track of delegated work
Reminder intervals editable with the task	Per-task intervals are set only at creation; `UpdateTaskRequest` has no reminder fields	Known gap (§22.4)
Screen 20 default reminder intervals	Held on Reminder Settings (`/settings/reminders`), linked from Application Settings	One place for reminder defaults
Screens 5, 16, 17 Department filter / column	Project filter (employee list, workload, reports); workload shows projects and allocation %	Projects and allocations added beyond the BRD
Screen 17 completion chart, counts by status	Four totals (Total, Completed, Pending, Overdue) with a progress bar	Simplified report summary
Screen 17 FR6 "only permitted records"	Organization-wide for `reports.view` holders	Known gap (§22.4)
Screens 2–4 dashboard widgets	Several cards (recent activity, due today, quick create) not implemented; analytics charts with filters and drill-down instead	See module doc 02
Screen 5 delete rule "has related task records"	Blocked only while the person has unfinished assigned tasks, then soft delete	
Not in BRD	Projects, positions and allocations; departments with managers; Role Configuration; TFA assistant and TFA Settings; internal Mail; Security Setup and Account Recovery; Legal pages; data folder and key ring	Added capabilities (§13, §15, §16)

24. Future Enhancements

Enhancement	Origin
Real-time push (SignalR/WebSockets) for notifications and mail counts (today: polling on navigation and every 15 s)	v1.0 + §22.4
Reminder retry/failure tracking (`ReminderHistory.Status`, `DeliveryAttempts`) and editable reminder intervals after creation	§22.4
Task-scope checks on delete/recipients and team scope in `TaskAccessHandler`; server-side enforcement of board/workload filter keys	§22.4
Report scoping to permitted records, department filter, per-status breakdown and chart, streamed export	Module doc 06
Profile security card: remaining backup codes and regeneration	Module doc 07
Persistent/shared rate limits and attempt limits for multi-instance hosting	§17
Audit log viewer for `AssistantAudit` and role/permission changes; retention for assistant conversations	Module docs 08, 09
Attachment clean-up job for purged mail and abandoned uploads	Module doc 05
Legal documents served from the backend with versioning and acceptance tracking	Module doc 10
File attachments on tasks, recurring tasks, calendar integration	v1.0

Appendix A — Task status transitions

Rules from `TaskStatusTransitions.Can` (`Services/Monitoring/TaskManagement/Rules`). Repeating the current status is always allowed; `Completed` and `Cancelled` are terminal. Tasks are created as `ToDo`. A change requires `tasks.status.change` plus task access, is made through `PATCH` `/api/v1/tasks/{id}/status`, writes a `StatusChanged` `TaskActivityLog` entry (plus `Completed` when completing), and sets `CompletedAt` / `CancelledAt`. Reminders stop for terminal states (Appendix B).

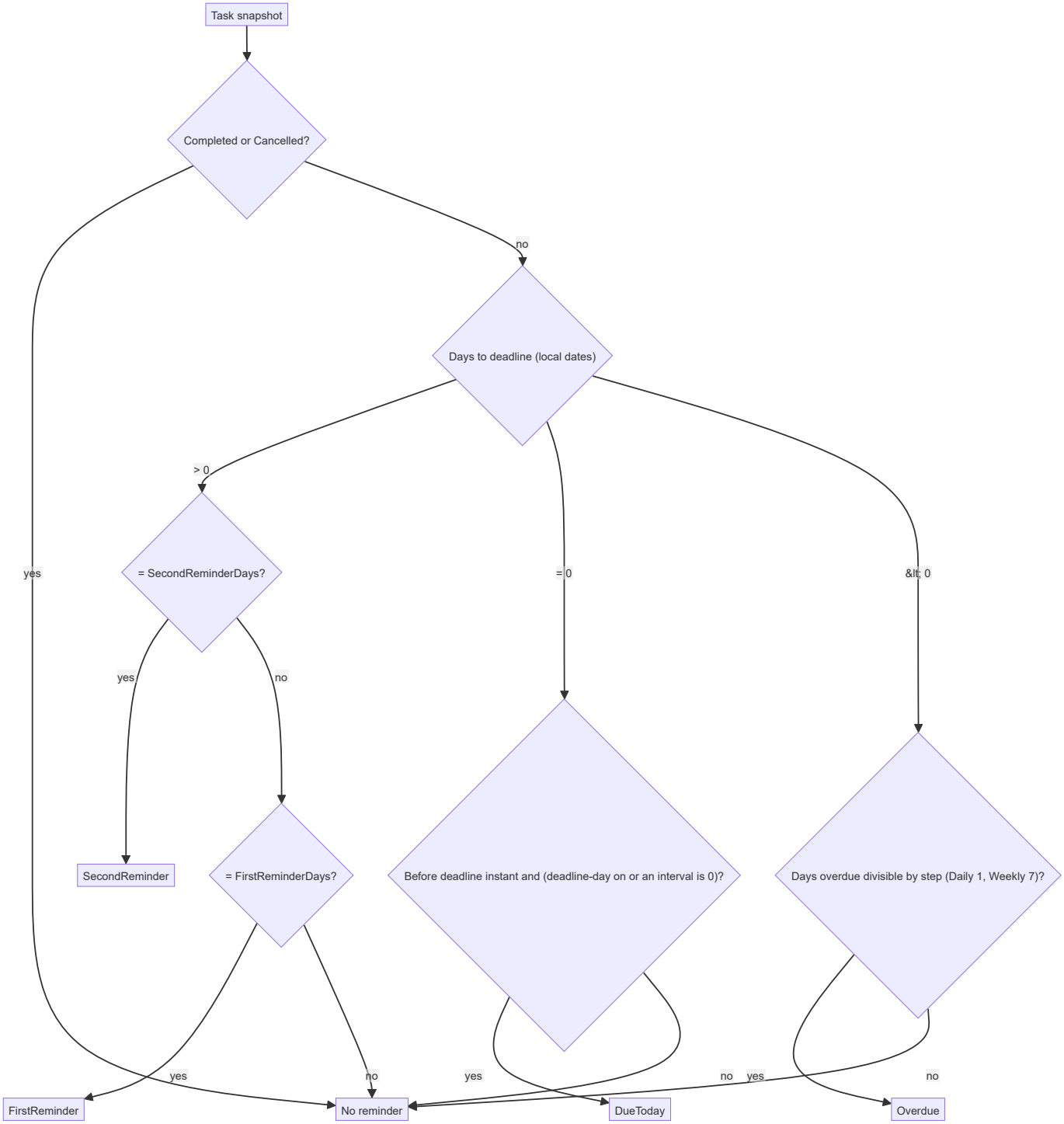


From \ To	ToDo	InProgress	Blocked	InQA	Completed	Cancelled
ToDo	✓ (same)	✓	✓	–	–	✓
InProgress	–	✓ (same)	✓	✓	✓	✓
Blocked	–	✓	✓ (same)	–	–	✓
InQA	–	✓	✓	✓ (same)	✓	✓
Completed	–	–	–	–	✓ (same)	–
Cancelled	–	–	–	–	–	✓ (same)

A refused move returns 400 (`BusinessRuleException`: "Cannot move a task from X to Y.").

Appendix B — Reminder slot calculation

`ReminderCalculator.DueReminders` decides, on each pass, which reminder (if any) a task is due for **today** in the application timezone. Inputs: `ReminderTaskSnapshot` (Status, DeadlineUtc, `FirstReminderDays`, `SecondReminderDays`, `OverdueReminderFrequency`), now (UTC), `Application.Timezone`, and `Reminder.DeadlineDayEnabled`. Nothing runs when `Reminder.Enabled` is false. Only tasks with a deadline within the next 32 days (or already past) are read.



Example. Deadline `2026-11-15 17:00` local (`Asia/Kolkata`, UTC+05:30) = `2026-11-15T11:30Z`; defaults `Reminder.DefaultFirstDays` = 2, `Reminder.DefaultSecondDays` = 1, `Reminder.DeadlineDayEnabled` = true. `ScheduledFor` is the UTC instant of local midnight of the reminder day (`LocalMidnightUtc`).

`ReminderType`	Local day	`ScheduledFor` (UTC)	`NotificationType`
`FirstReminder`	2026-11-13	2026-11-12T18:30Z	`DeadlineReminder`
`SecondReminder`	2026-11-14	2026-11-13T18:30Z	`DeadlineReminder`
`DueToday`	2026-11-15 (only before 17:00)	2026-11-14T18:30Z	`DeadlineReminder`
`Overdue` (`Daily`)	2026-11-16, 17, 18 …	2026-11-15T18:30Z, …	`Overdue`
`Overdue` (`Weekly`)	2026-11-22, 29 …	2026-11-21T18:30Z, …	`Overdue`

Rules:

- **Recipients**: the task's `TaskRecipient` users (the assignee is always one) plus the creator, de-duplicated, active and not deleted.
- **Dedup**: one `ReminderHistory` row per (TaskId, RecipientUserId, ReminderType, ScheduledFor), inserted with `ON CONFLICT DO NOTHING` only while the task is not deleted, not `Completed` / `Cancelled`, and its deadline still equals the deadline read by the pass. The notification (and the Inbox mail when `Notification.MailEnabled`) is written in the same transaction only if the row was inserted.
- **No backfill**: only today's reminder is produced; a day missed while the API was stopped is not sent later.
- **Equal intervals** collapse to one reminder (second is checked first); an interval of 0 asks for the `DueToday` reminder even when the global deadline-day reminder is off.
- **Intervals**: per task at creation, `FirstReminderDays` and `SecondReminderDays` 0–30 with `SecondReminderDays` ≤ `FirstReminderDays` unless the first is 0 (`ReminderIntervals.AreOrdered`); missing values come from Reminder Settings (defaults 1–30 and 0–30).
- **Recalculation**: a deadline change or completion takes effect on the next pass, because the calculator reads the current task and the insert re-checks the deadline.
- **DST**: a missing local midnight uses the first valid local time; an ambiguous one uses the earlier instant.

Appendix C — Permission Catalog

Source of truth: `Common/Security/PermissionCatalog.cs` (64 catalog entries) and `PermissionKeys.cs` (70 keys = 64 catalog keys + 6 legacy keys). Modules are listed in the Role Configuration tree order.

Value	Meaning
Always On	`AlwaysGranted: true`: every signed-in user has it; not stored as a grant and cannot be switched off
Granted	Held by the role. The Administrator receives every stored permission row on every start (`AccessControlSeeder`); Manager/Employee receive `GrantedToManager` / `GrantedToEmployee` entries when the permission is first seeded
Denied	Not held by that system role. System roles are read-only (database triggers), so their grants cannot be changed
Configurable	Can be ticked or unticked on a custom role in Role Configuration (`/settings/permissions`)
Not assignable	`IsAssignable: false` and not always on: `RoleService` refuses it for any role other than Administrator

Notes column: *scope rank* = levels of one selector (higher includes lower: `dashboard.view.organization` ⊃ `dashboard.view.team` ⊃ `dashboard.view.own`; `tasks.view.all` ⊃ `tasks.view.own`); *requires* = `Requires` key that must also be held (checked on save); *first seeded to …* = `GrantToAllExistingRoles` / `GrantFrom` behaviour when the key first appears, so existing custom roles kept their access.

C.1 Dashboard

Key	Permission	Screen › Section	Action	Administrator	Manager	Employee	Custom role	Notes
`dashboard.view.own`	View my dashboard	Dashboard › Overview	View	Always On	Always On	Always On	Always On	scope rank 1
`dashboard.view.team`	View team dashboard	Dashboard › Overview	View	Granted	Granted	Denied	Configurable	scope rank 2

Key	Permission	Screen › Section	Action	Administrator	Manager	Employee	Custom role	Notes
`dashboard.view.organization`	View organization dashboard	Dashboard › Overview	View	Granted	Denied	Denied	Configurable	scope rank 3
`dashboard.analytics.view`	View charts and reports	Dashboard › Analytics	Access	Granted	Granted	Granted	Configurable	first seeded to all roles existing then
`dashboard.filters.use`	Use dashboard filters	Dashboard › Analytics	Use	Granted	Granted	Granted	Configurable	first seeded to all roles existing then

C.2 TaskFlow Monitoring

Key	Permission	Screen › Section	Action	Administrator	Manager	Employee	Custom role	Notes
`tasks.view.own`	View my tasks	Task Management › Task list	View	Always On	Always On	Always On	Always On	scope rank 1
`tasks.view.all`	View all tasks	Task Management › Task list	View	Granted	Granted	Denied	Configurable	scope rank 2
`tasks.create`	Create tasks	Task Management › Task list	Create	Granted	Granted	Denied	Configurable	–
`tasks.delete`	Delete tasks	Task Management › Task list	Delete	Granted	Denied	Denied	Configurable	–
`tasks.update`	Edit tasks	Task Management › Details	Edit	Granted	Granted	Denied	Configurable	–
`tasks.reassign`	Reassign tasks	Task Management › Details	Assign	Granted	Granted	Denied	Configurable	–
`tasks.recipients.manage`	Manage task recipients	Task Management › Details	Assign	Granted	Granted	Denied	Configurable	–
`tasks.comment.create`	Post task comments	Task Management › Discussion	Create	Granted	Granted	Granted	Configurable	first seeded to all roles existing then
`tasks.comment.edit`	Edit own task comments	Task Management › Discussion	Edit	Granted	Granted	Granted	Configurable	first seeded to all roles existing then
`tasks.comment.delete`	Delete own task comments	Task Management › Discussion	Delete	Granted	Granted	Granted	Configurable	first seeded to all roles existing then
`tasks.activity.view`	View task history	Task Management › Activity	View	Granted	Granted	Granted	Configurable	first seeded to all roles existing then
`workload.view`	View workload	Employee Workload	View	Granted	Granted	Denied	Configurable	–

Key	Permission	Screen › Section	Action	Administrator	Manager	Employee	Custom role	Notes
`workload.filters.use`	Use workload filters	Employee Workload	Use	Granted	Granted	Denied	Configurable	requires `workload.view`; first seeded to holders of `workload.view`
`workload.tasks.open`	Open an employee's tasks	Employee Workload	Access	Granted	Granted	Denied	Configurable	requires `workload.view`; first seeded to holders of `workload.view`
`board.view`	View board	Board	View	Granted	Granted	Granted	Configurable	first seeded to all roles existing then
`board.filters.use`	Use board filters	Board	Use	Granted	Granted	Granted	Configurable	requires `board.view`; first seeded to all roles existing then
`tasks.status.change`	Update task status	Board	Edit	Granted	Granted	Granted	Configurable	first seeded to all roles existing then

C.3 TaskFlow Management

Key	Permission	Screen › Section	Action	Administrator	Manager	Employee	Custom role	Notes
`employees.view`	View employees	Employee Management › Employee list	View	Granted	Granted	Denied	Configurable	–
`employees.create`	Create employees	Employee Management › Employee list	Create	Granted	Denied	Denied	Configurable	requires `employees.view`
`employees.edit`	Edit employees	Employee Management › Employee list	Edit	Granted	Denied	Denied	Configurable	requires `employees.view`
`employees.status.change`	Activate or deactivate employees	Employee Management › Employee list	Edit	Granted	Denied	Denied	Configurable	requires `employees.view`
`employees.role.assign`	Assign roles	Employee Management › Employee list	Assign	Granted	Denied	Denied	Configurable	requires `employees.view`
`employees.mfa.reset`	Reset two-step verification	Employee Management › Employee list	Manage	Granted	Denied	Denied	Configurable	requires `employees.view`
`employees.delete`	Delete employees	Employee Management › Employee list	Delete	Granted	Denied	Denied	Configurable	requires `employees.view`
`projects.view`	View projects	Project Management › Project list	View	Granted	Granted	Denied	Configurable	–
`projects.create`	Create projects	Project Management › Project list	Create	Granted	Denied	Denied	Configurable	requires `projects.view`

Key	Permission	Screen › Section	Action	Administrator	Manager	Employee	Custom role	Notes
`projects.edit`	Edit projects	Project Management › Details	Edit	Granted	Denied	Denied	Configurable	requires `projects.view`
`projects.delete`	Delete projects	Project Management › Project list	Delete	Granted	Denied	Denied	Configurable	requires `projects.view`
`projects.members.manage`	Manage project members	Project Management › Members	Assign	Granted	Denied	Denied	Configurable	requires `projects.view`
`positions.create`	Create positions	Project Management › Members	Create	Granted	Denied	Denied	Configurable	requires `projects.view`
`departments.view`	View departments	Department Management › Department list	View	Granted	Granted	Denied	Configurable	–
`departments.create`	Create departments	Department Management › Department list	Create	Granted	Denied	Denied	Configurable	requires `departments.view`
`departments.edit`	Edit departments	Department Management › Department list	Edit	Granted	Denied	Denied	Configurable	requires `departments.view`
`departments.delete`	Delete departments	Department Management › Department list	Delete	Granted	Denied	Denied	Configurable	requires `departments.view`
`departments.members.manage`	Manage department members	Department Management › Members	Assign	Granted	Denied	Denied	Configurable	requires `departments.view`

C.4 Communication Management

Key	Permission	Screen › Section	Action	Administrator	Manager	Employee	Custom role	Notes
`mail.read`	Read mail	Mail › Mailbox	View	Granted	Granted	Granted	Configurable	first seeded to all roles existing then
`mail.compose`	Write and send mail	Mail › Mailbox	Create	Granted	Granted	Granted	Configurable	first seeded to all roles existing then
`mail.organize`	Organize mail	Mail › Mailbox	Edit	Granted	Granted	Granted	Configurable	first seeded to all roles existing then
`mail.attachments`	Use mail attachments	Mail › Mailbox	Use	Granted	Granted	Granted	Configurable	first seeded to all roles existing then
`notifications.view`	View notifications	Notification › Notification centre	View	Granted	Granted	Granted	Configurable	first seeded to all roles existing then
`notifications.manage`	Mark notifications read	Notification › Notification centre	Edit	Granted	Granted	Granted	Configurable	first seeded to all roles existing then

C.5 Report

Key	Permission	Screen › Section	Action	Administrator	Manager	Employee	Custom role	Notes
`reports.view`	View reports	Reports	View	Granted	Granted	Denied	Configurable	–
`reports.export`	Export reports	Reports	Export	Granted	Granted	Denied	Configurable	requires `reports.view`; first seeded to holders of `reports.view`

C.6 Settings

Key	Permission	Screen › Section	Action	Administrator	Manager	Employee	Custom role	Notes
`settings.reminders.manage`	Reminder settings	Reminder Settings	Manage	Granted	Granted	Denied	Configurable	–
`settings.application.manage`	Application settings	Application Settings	Manage	Granted	Denied	Denied	Configurable	–
`settings.ai.view`	View AI provider settings	TFA Settings › AI providers	View	Granted	Denied	Denied	Configurable	–
`settings.ai.edit`	Edit AI provider settings	TFA Settings › AI providers	Edit	Granted	Denied	Denied	Configurable	requires `settings.ai.view`
`settings.ai.test`	Test AI providers	TFA Settings › AI providers	Use	Granted	Denied	Denied	Configurable	requires `settings.ai.view`
`settings.ai.model.remove`	Remove AI models	TFA Settings › AI providers	Delete	Granted	Denied	Denied	Configurable	requires `settings.ai.view`
`roles.manage`	Manage roles and permissions	Role Configuration	Manage	Granted	Denied	Denied	Configurable	–
`roles.view.all`	View all roles	Role Configuration	View	Granted	Denied	Denied	Configurable	–
`roles.modify.access`	Modify access	Role Configuration	Edit	Granted	Denied	Denied	Configurable	–
`roles.administrator.assign`	Give the Administrator role	Role Configuration	Assign	Granted	Denied	Denied	Not assignable	Administrator only

C.7 My Profile

Key	Permission	Screen › Section	Action	Administrator	Manager	Employee	Custom role	Notes
`profile.view`	View my profile	My profile › Account	View	Always On	Always On	Always On	Always On	–
`profile.edit`	Edit my personal information	My profile › Personal information	Edit	Always On	Always On	Always On	Always On	–

Key	Permission	Screen › Section	Action	Administrator	Manager	Employee	Custom role	Notes
`profile.password.change`	Change my password	My profile › Security	Edit	Always On	Always On	Always On	Always On	–
`profile.mfa.manage`	Manage my authenticator	My profile › Security	Manage	Always On	Always On	Always On	Always On	–

C.8 TFA assistant

Key	Permission	Screen › Section	Action	Administrator	Manager	Employee	Custom role	Notes
`assistant.chat`	Chat with the assistant	Assistant	Use	Granted	Granted	Granted	Configurable	first seeded to all roles existing then
`assistant.actions.confirm`	Confirm assistant actions	Assistant	Use	Granted	Granted	Granted	Configurable	first seeded to all roles existing then; TFA can only do what the user's own permissions allow

C.9 Legacy keys

Legacy keys are not in the catalog and are never shown or assignable. A role that still holds one keeps exactly its old access: `EffectivePermissions.Expand` adds the finer keys listed below. When a role is saved, a submitted legacy key is replaced by its finer keys (`RoleService.ResolvePermissionsAsync`), so the old key is dropped.

Key (constant)	Module	Expands to	Administrator	Manager / Employee	Custom role
`dashboard.admin` (`LegacyDashboardAdmin`)	Dashboard	`dashboard.view.organization`	Granted if the row exists (all stored rows are granted)	Only if held from an earlier release	Not assignable
`dashboard.manager` (`LegacyDashboardManager`)	Dashboard	`dashboard.view.team`	Granted if the row exists	Only if held from an earlier release	Not assignable
`projects.manage` (`LegacyProjectsManage`)	TaskFlow Management	`projects.create`, `projects.edit`, `projects.delete`, `projects.members.manage`, `positions.create`	Granted if the row exists	Only if held from an earlier release	Not assignable
`departments.manage` (`LegacyDepartmentsManage`)	TaskFlow Management	`departments.create`, `departments.edit`, `departments.delete`, `departments.members.manage`	Granted if the row exists	Only if held from an earlier release	Not assignable
`employees.manage` (`LegacyEmployeesManage`)	TaskFlow Management	`employees.create`, `employees.edit`, `employees.status.change`, `employees.mfa.reset`	Granted if the row exists	Only if held from an earlier release	Not assignable

Key (constant)	Module	Expands to	Administrator	Manager / Employee	Custom role
`settings.ai.manage` (`LegacySettingsAi`)	Settings	`settings.ai.view`, `settings.ai.edit`, `settings.ai.test`, `settings.ai.model.remove`	Granted if the row exists	Only if held from an earlier release	Not assignable

> ⚠️ To be confirmed: which legacy grants a given live database still holds; the current seeder never inserts legacy rows, they exist only in databases created by earlier releases.

Totals: 8 modules; 64 catalog keys (6 Always On, 1 Not assignable, 57 Configurable) + 6 legacy keys = **70 keys**. Default grants: Administrator 58 stored + 6 Always On; Manager 32 stored; Employee 17 stored (each plus the 6 Always On). See §9.3–9.10 for how keys are enforced.

© 2026 Jojo George. All rights reserved.